claude-windows / 46eabfd2-0e71-4ff5-a68d-c76f474d4ff3

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-khelsutra-guru-badminton-highlight-indexer\46eabfd2-0e71-4ff5-a68d-c76f474d4ff3.jsonl`
- SHA-256: `8dee64ffd48b9fe577a978cbc787c9cb07d7322b026c28fd5e4b704cbb9b51cf`
- Source modified: `2026-07-08T09:46:16+00:00`
- Imported at: `2026-07-08T15:59:19+00:00`
- project: `C--Users-avidu-Projects-khelsutra-guru-badminton-highlight-indexer`
- session_id: `46eabfd2-0e71-4ff5-a68d-c76f474d4ff3`

Transcript

1. user (2026-07-08T05:53:55.965Z)

Follow-up to #521 (PR #525, merged compile-on-L4 + NVENC). Part 3 of #521 was deferred: make the branding watermark CHEAPER so the per-clip stitch isn't a full 4K re-encode.

Context: `backend/pipeline/stitcher/compiler.py::VideoCompiler` burns the watermark via a `drawtext`/`movie` `-vf` filtergraph during a per-clip re-encode. #521 already added an `encoder` knob (NVENC on the L4, libx264 fallback) via `_select_encoder`, which hardware-accelerates the ENCODE ? but every clip is still fully re-encoded because the watermark filter forces it. The 2026-07-08 CUJ (§9 of docs/COMPUTE_DECOUPLED_SERVING/runlogs/DEPLOY_REPORT_cloud-serving_validation-latency_2026-07-07.md) showed the watermark re-encode was the dominant stitch cost.

Task: investigate a path where the stitch is NOT a full re-encode:
- Option A: pre-render the watermark to a transparent PNG overlay ONCE (per resolution) and `overlay` it, still cheaper than drawtext-per-frame; or
- Option B: apply the watermark only at the CONCAT stage (single pass over the final reel) instead of per-clip, so per-clip trims can stream-copy (`-c copy`) ? the big win, since trims dominate. This changes the two-stage codec contract in `compile_highlights` (per-clip trims currently re-encode so the concat can `-c copy`); flipping it means trims `-c copy` and the concat re-encodes once with the watermark. Watch: `-ss`/`-to` stream-copy cuts at keyframes (frame-accuracy loss) ? may need a hybrid (re-encode only the boundary GOPs) or keep the current approach for frame-accurate trims.
- Keep libx264 the default + parity-safe; measure on a real 4K clip. Respect D5/D16(b) parity notes in docs/COMPUTE_DECOUPLED_SERVING/README.md (reel-encode parity is already relaxed under NVENC opt-in).

Document the chosen approach + measured before/after in docs/COMPUTE_DECOUPLED_SERVING/PHASE_PLACEMENT.md §3 (which already flags this as a tracked follow-up). Branch off origin/master; one PR referencing #521.

2. assistant (2026-07-08T05:54:04.426Z)

I'll start by ramping up on the current state. Let me pull the repo and read the key files in parallel, following the repo-freshness and handoff conventions.

3. user (2026-07-08T05:54:09.106Z)

```
On branch codex/issue-523-sd-reel-plan
Your branch is up to date with 'origin/master'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   SESSION_HANDOFF.md

Untracked files:
```

```
  (use "git add <file>..." to include in what will be committed)
        score_golden_gate.py
        score_local.py

no changes added to commit (use "git add" and/or "git commit -a")
---BRANCH---
codex/issue-523-sd-reel-plan
---FETCH---
```

4. user (2026-07-08T05:54:09.149Z)

```
1       # Session Handoff
2
3       ## You are here
4
5       This checkout is on `master` at `bf97700` after a successful `git pull --ff-only`
6       on 2026-07-08. The top-level workspace folder is not a Git repo; the relevant
7       engine repo is `badminton-highlight-indexer/`. There are pre-existing untracked
8       local files `score_golden_gate.py` and `score_local.py`; do not stage or delete
9       them.
10
11      A planning pass for the optional cheap SD reel request is complete. Tracking bug:
12      https://github.com/Khelsutra/badminton-highlight-indexer/issues/523
13
14      ## Next steps / open threads
15
16      1. Engine PR for #523: add an opt-in compile rendition field (`original` default,
17         `sd` optional), persist it through `jobs.params`, and make `_execute_compile`
18         call `VideoCompiler.compile_highlights(...)` exactly once using the requested
19         profile.
20      2. SPA PR in sibling `khelsutra/web`: expose the SD option near Build, send the
21         compile rendition only when selected, and use a distinct `-sd.mp4` filename.
22      3. Bundle follow-up: rebuild/import the SPA snapshot into `backend/ui` after the
23         source SPA PR lands, if bundling remains separate.
24      4. Link each PR back to #523 and add verification notes/artifacts there.
25
26      ## Ramp-up kit
27
28      - `backend/main.py::CompileRequest` and `backend/main.py::compile_highlights`
29      - `backend/orchestration.py::_resolve_compile` and `::_execute_compile`
30      - `backend/infrastructure/database_jobs.py::create_compile_job`
31      - `backend/pipeline/stitcher/compiler.py::VideoCompiler`
32      - `tests/test_api_endpoints.py` compile tests
33      - `tests/test_compiler.py`
34      - Sibling repo `../khelsutra/web/src/api/client.ts`
35      - Sibling repo `../khelsutra/web/src/App.tsx`
36      - Sibling repo `../khelsutra/web/src/components/SelectionFooter.tsx`
37
38      ## Key decisions
39
40      - SD is a replacement compile mode, not an additional output: if selected, build
41        and deliver only the SD reel.
42      - Backward compatibility matters: old compile payloads should behave as original
43        quality.
44      - Use the existing async compile job shape; no DB migration is needed because
45        compile-specific params already live in `jobs.params`.
46      - Apply the SD profile before any per-rally ffmpeg trim so temporary clips are
47        consistently encoded and the final concat can remain stream-copy.
48      - Prefer typed, tunable stitching SD settings over hard-coded encode constants.
49
```

5. user (2026-07-08T05:54:09.167Z)

```
1       import logging
2       import os
```

```
3       import subprocess
4       import tempfile
5       from pathlib import Path
6       from typing import Any, Dict, List, Optional, Tuple, Union
7
8       from backend.utils.ffmpeg import (  # P2a: single ffmpeg/ffprobe resolver
9           ffmpeg_bin,
10          ffmpeg_timeout_sec,
11          ffprobe_bin,
12          ffprobe_timeout_sec,
13          timeout_label,
14      )
15
16      logger = logging.getLogger(__name__)
17
18      # Bundled, commercially-clean default watermark font (Apache-2.0). Shipped so the
overlay
19      # renders out-of-the-box without a system fontconfig setup (a common gap on Windows,
where
20      # `font='Sans'` fails). Overridable via stitching.watermark.font_path. This module lives
at
21      # backend/pipeline/stitcher/, so parents[2] == backend/.
22      _BUNDLED_FONT = (
23          Path(__file__).resolve().parents[2] / "assets" / "fonts" / "RobotoSlab-Regular.ttf"
24      )
25
26
27      class VideoCompiler:
28          def __init__(
29              self,
30              output_dir: str,
31              end_padding: float = 1.0,
32              begin_padding: float = 0.5,
33              watermark: Optional[Any] = None,
34          ):
35              self.output_dir = output_dir
36              self.end_padding = end_padding
37              self.begin_padding = begin_padding
38              # Optional branding overlay. Accepts a WatermarkConfig model or a plain dict;
39              # normalized to a dict (or None) so this module stays config-package-agnostic.
40              self.watermark = self._normalize_watermark(watermark)
41              os.makedirs(output_dir, exist_ok=True)
42
43          @staticmethod
44          def _normalize_watermark(watermark: Optional[Any]) -> Optional[Dict[str, Any]]:
45              if watermark is None:
46                  return None
47              if hasattr(watermark, "model_dump"):
48                  return watermark.model_dump()
49              if isinstance(watermark, dict):
50                  return dict(watermark)
51              return None
52
53          @staticmethod
54          def _ff_quote(path: str) -> str:
55              """Make a filesystem path safe to embed inside single quotes in an ffmpeg
56              filtergraph operand (drawtext fontfile/textfile, movie= source): forward
57              slashes (Windows-friendly), escaped single quotes, AND escaped colons.
58
59              The colon MUST be escaped even inside single quotes: ffmpeg's filtergraph
60              option parser treats ':' as the option separator, so a Windows drive-letter
61              path like ``C:/dir/font.ttf`` otherwise truncates the operand at ``C`` and
62              the encode fails with "Invalid argument". This silently disabled the
63              on-by-default bundled-font watermark (and absolute logo paths) on Windows."""
64              return path.replace("\\", "/").replace("'", r"\'").replace(":", r"\:")
```

```

        @staticmethod
        def _parse_time(val: Union[int, float, str]) -> float:
            """Normalizes a timestamp value to float seconds.
            Handles: float/int (pass-through), plain numeric strings ("12.4"),
            and colon-separated strings ("MM:SS", "HH:MM:SS", "H:MM:SS.ms").
            This mirrors the parse_time logic used in gemini.py to ensure
            the stitcher can always consume whatever format the indexer produced."""
            if isinstance(val, (int, float)):
                return float(val)
            if isinstance(val, str):
                val = val.strip()
                if ":" in val:
                    # Handle MM:SS, HH:MM:SS, etc.
                    parts = val.split(":")
                    parts.reverse()
                    total = 0.0
                    for i, p in enumerate(parts):
                        try:
                            total += float(p) * (60**i)
                        except ValueError:
                            pass
                    return total
                try:
                    return float(val)
                except ValueError:
                    logger.warning(
                        f"Could not parse timestamp value '{val}' as a number. Defaulting to
0.0."
                    )
                    return 0.0
            logger.warning(
                f"Unexpected timestamp type {type(val).__name__} for value '{val}'.
Defaulting to 0.0."
            )
            return 0.0

        @staticmethod
        def _merge_overlapping(
            segments: List[Tuple[float, float]],
        ) -> List[Tuple[float, float]]:
            """Normalize, sort, and merge overlapping/duplicate (start, end) ranges.

            A highlight reel must never replay the same footage. The same rally can land in
            the segment list more than once ? e.g. two detector runs accumulated for one
            video (add_video is an UPSERT, so re-running without a prune leaves both sets),
or
            chunk-overlap duplicates near boundaries. Stitching those verbatim plays the
rally
            twice. Merging any range that overlaps or abuts the previous one collapses true
            duplicates/overlaps while preserving genuinely distinct (gapped) rallies.
            Degenerate rows (end <= start) are dropped. Timestamps may be float/int/str.
            """
            parsed = []
            for raw_s, raw_e in segments:
                s = VideoCompiler._parse_time(raw_s)
                e = VideoCompiler._parse_time(raw_e)
                if e > s:
                    parsed.append((s, e))
            parsed.sort()
            merged: List[Tuple[float, float]] = []
            for s, e in parsed:
                if merged and s <= merged[-1][1]:  # overlaps/abuts previous -> extend it
                    merged[-1] = (merged[-1][0], max(merged[-1][1], e))
                else:
```

```
                    merged.append((s, e))
            return merged

        def _get_video_duration(self, video_path: str) -> float:
            """Probes the video file to get its total duration in seconds.
            Returns 0.0 if it cannot be determined."""
            try:
                timeout = ffprobe_timeout_sec()
                result = subprocess.run(
                    [
                        ffprobe_bin(),
                        "-v",
                        "error",
                        "-show_entries",
                        "format=duration",
                        "-of",
                        "default=noprint_wrappers=1:nokey=1",
                        video_path,
                    ],
                    capture_output=True,
                    text=True,
                    check=True,
                    timeout=timeout,
                )
                return float(result.stdout.strip())
            except subprocess.TimeoutExpired:
                logger.warning(
                    "Could not determine video duration via ffprobe: timed out after %s",
                    timeout_label(ffprobe_timeout_sec()),
                )
                return 0.0
            except Exception as e:
                logger.warning(f"Could not determine video duration via ffprobe: {e}")
                return 0.0

        def _watermark_filter(self, tmp_dir: str) -> Optional[str]:
            """Build the ffmpeg -vf filtergraph that burns the configured watermark (text
            and/or logo PNG) into a clip during the per-clip re-encode. Returns None when
            watermarking is disabled or has nothing to draw. The concat stage stays -c copy
            because the mark is already baked into every clip identically."""
            wm = self.watermark
            if not wm or not wm.get("enabled"):
                return None

            text = (wm.get("text") or "").strip()
            logo_path = (wm.get("logo_path") or "").strip()
            if logo_path and not os.path.exists(logo_path):
                logger.warning(
                    f"[watermark] logo not found: {logo_path} ? skipping logo overlay."
                )
                logo_path = ""
            if not text and not logo_path:
                return None

            margin = int(wm.get("margin", 24))
            opacity = float(wm.get("opacity", 0.85))
            position = str(wm.get("position", "bottom-right"))
            # drawtext uses lowercase w/h (main) + tw/th (text); overlay uses W/H (main) +
w/h (logo).
            dt_xy = {
                "bottom-right": f"x=w-tw-{margin}:y=h-th-{margin}",
                "bottom-left": f"x={margin}:y=h-th-{margin}",
                "top-right": f"x=w-tw-{margin}:y={margin}",
                "top-left": f"x={margin}:y={margin}",
            }.get(position, f"x=w-tw-{margin}:y=h-th-{margin}")
```

```
            ov_xy = {
                "bottom-right": f"W-w-{margin}:H-h-{margin}",
                "bottom-left": f"{margin}:H-h-{margin}",
                "top-right": f"W-w-{margin}:{margin}",
                "top-left": f"{margin}:{margin}",
            }.get(position, f"W-w-{margin}:H-h-{margin}")

            drawtext = ""
            if text:
                ratio = float(wm.get("font_size_ratio") or 0.035)
                divisor = max(1, round(1.0 / ratio))
                # textfile= sidesteps the notorious drawtext text= escaping for arbitrary
strings.
                tf_path = os.path.join(tmp_dir, "watermark.txt")
                with open(tf_path, "w", encoding="utf-8") as fh:
                    fh.write(text)
                # Font precedence: explicit font_path ? bundled Apache-2.0 default (renders
                # without fontconfig) ? fontconfig family (last-resort, environment-
dependent).
                font_path = (wm.get("font_path") or "").strip()
                if not font_path and _BUNDLED_FONT.exists():
                    font_path = str(_BUNDLED_FONT)
                if font_path:
                    font_opt = f"fontfile='{self._ff_quote(font_path)}'"
                else:
                    font_opt = f"font='{wm.get('font', 'Sans')}'"
                box = ""
                if wm.get("box", True):
                    box = f":box=1:boxcolor=black@{min(opacity, 0.5):.2f}:boxborderw=10"
                drawtext = (
                    f"drawtext=textfile='{self._ff_quote(tf_path)}':{font_opt}"
                    f":fontcolor=white@{opacity:.2f}:fontsize=h/{divisor}:{dt_xy}{box}"
                )

            if logo_path and drawtext:
                return
f"movie='{self._ff_quote(logo_path)}'[wm];[in][wm]overlay={ov_xy},{drawtext}"
            if logo_path:
                return f"movie='{self._ff_quote(logo_path)}'[wm];[in][wm]overlay={ov_xy}"
            return drawtext

        def _trim_one_segment(
            self,
            idx: int,
            raw_start: Union[int, float, str],
            raw_end: Union[int, float, str],
            segments: List[Tuple[float, float]],
            video_duration: float,
            tmp_dir: str,
            raw_video_path: str,
            watermark_filter: Optional[str],
        ) -> Tuple[Optional[str], Optional[Tuple[int, float, float, str]]]:
            """Trim a single segment into its own clip file (extracted verbatim from the
            per-segment body of compile_highlights). Returns (clip_path, skip_record): on
            success the clip path and None; on a skip (invalid/out-of-range/ffmpeg failure)
            None and the (idx, start, end, reason) record to record in skipped_segments."""
            # Normalize timestamps to float seconds ? handles float, int,
            # plain numeric strings, and colon-separated formats like "MM:SS"
            start = self._parse_time(raw_start)
            end = self._parse_time(raw_end)
            segment_label = (
                f"Segment #{idx + 1}/{len(segments)} [{start:.2f}s - {end:.2f}s]"
            )

            # Validate the segment times
```

```
            if start < 0:
                logger.warning(
                    f"{segment_label}: start_time is negative ({start:.2f}s). Clamping to
0."
                )
                start = 0.0

            if start >= end:
                logger.error(
                    f"{segment_label}: start_time ({start:.2f}s) >= end_time ({end:.2f}s).
Skipping invalid segment."
                )
                return None, (idx, start, end, "start >= end")

            # Check if segment extends beyond video duration
            if video_duration > 0:
                if start >= video_duration:
                    logger.error(
                        f"{segment_label}: start_time ({start:.2f}s) is beyond the video
duration ({video_duration:.2f}s). "
                        f"This segment cannot be extracted ? the video ran out. Skipping."
                    )
                    return None, (idx, start, end, "start beyond video end")

                if end > video_duration:
                    original_end = end
                    end = video_duration
                    logger.warning(
                        f"{segment_label}: end_time ({original_end:.2f}s) exceeds video
duration ({video_duration:.2f}s). "
                        f"Clamping end_time to {video_duration:.2f}s. The segment may be
truncated."
                    )

            clip_path = os.path.join(tmp_dir, f"clip_{idx}.mp4")

            # Precise sub-second trim using fast re-encoding to preserve stream headers
            padded_start = max(0.0, start - self.begin_padding)
            padded_end = end + self.end_padding

            # Clamp padded_end to video duration if known
            if video_duration > 0 and padded_end > video_duration:
                padded_end = video_duration
                logger.info(
                    f"{segment_label}: Padded end ({end + self.end_padding:.2f}s) exceeds
video duration. "
                    f"Clamping to {video_duration:.2f}s (end padding partially applied)."
                )

            trim_duration = padded_end - padded_start
            logger.info(
                f"Trimming {segment_label} -> [{padded_start:.3f}s - {padded_end:.3f}s]
(duration: {trim_duration:.2f}s)"
            )

            base_cmd = [
                ffmpeg_bin(),
                "-y",
                "-ss",
                str(round(padded_start, 3)),
                "-to",
                str(round(padded_end, 3)),
                "-i",
                raw_video_path,
                "-c:v",
```

```
                "libx264",
                "-preset",
                "superfast",
                "-crf",
                "22",
                "-c:a",
                "aac",
                "-b:a",
                "128k",
            ]
            vf_args = ["-vf", watermark_filter] if watermark_filter else []

            # Try with the watermark first; if that encode fails (e.g. a bad font/logo
            # path), retry once WITHOUT it so a branding misconfig can never nuke the
            # whole reel. Clips stay codec-identical either way, so concat -c copy holds.
            attempts = [vf_args, []] if vf_args else [[]]
            produced = False
            last_exc: Optional[subprocess.CalledProcessError] = None
            cmd = base_cmd + [clip_path]
            timeout = ffmpeg_timeout_sec()
            for attempt_vf in attempts:
                cmd = base_cmd + attempt_vf + [clip_path]
                try:
                    subprocess.run(
                        cmd, capture_output=True, check=True, timeout=timeout
                    )
                except subprocess.TimeoutExpired:
                    reason = f"ffmpeg timeout after {timeout_label(timeout)}"
                    logger.error(f"{segment_label}: FFmpeg trim timed out.")
                    print(f"  Command: {' '.join(cmd)}")
                    print(f"  {reason}")
                    return None, (idx, start, end, reason)
                except subprocess.CalledProcessError as e:
                    last_exc = e
                    if attempt_vf and vf_args:
                        _err = (
                            e.stderr.decode(errors="replace").strip()[-300:]
                            if getattr(e, "stderr", None)
                            else ""
                        )
                        logger.warning(
                            f"{segment_label}: watermark encode failed; retrying without it.
"
                            f"ffmpeg: {_err or '(no stderr captured)'}"
                        )
                        continue
                    break
                if os.path.exists(clip_path) and os.path.getsize(clip_path) > 0:
                    produced = True
                    if not attempt_vf and vf_args:
                        logger.warning(
                            f"{segment_label}: encoded WITHOUT the watermark (the watermark
filter failed)."
                        )
                    break
                last_exc = None  # ffmpeg succeeded but produced nothing
                break

            if produced:
                return clip_path, None
            elif last_exc is not None:
                stderr_msg = last_exc.stderr.decode(errors="replace").strip()
                logger.error(f"{segment_label}: FFmpeg trim failed.")
                print(f"  Command: {' '.join(cmd)}")
                print(
```

```
373                     f"  FFmpeg stderr: {stderr_msg[-500:]}"
374                 )  # Last 500 chars to avoid flooding
375                 return None, (idx, start, end, f"ffmpeg error: {stderr_msg[-200:]}")
376             else:
377                 logger.error(
378                     f"{segment_label}: FFmpeg exited successfully but output clip is missing
or empty. Skipping."
379                 )
380                 return None, (idx, start, end, "empty output clip")
381 
382         def compile_highlights(
383             self,
384             raw_video_path: str,
385             segments: List[Tuple[float, float]],
386             output_filename: str,
387         ) -> str:
388             """
389             Extracts selected segments from a raw video and stitches them together
390             into a seamless highlights file, applying end_padding to prevent abrupt endings.
391 
392             .. warning::
393                This method uses a two-stage codec contract: per-clip re-encodes must use
394                IDENTICAL encoder settings (codec, preset, CRF, pixel format) so the final
395                concat can stream-copy (``-c copy``) without re-encoding. Changing the
396                clip encode settings without updating the concat assumptions will produce
397                broken output.
398             """
399             if not segments:
400                 raise ValueError("No highlight segments provided to compile.")
401 
402             # Never stitch the same footage twice: collapse overlapping/duplicate ranges
403             # (e.g. accumulated re-run segments or chunk-overlap dupes) before trimming.
404             original_count = len(segments)
405             segments = self._merge_overlapping(segments)
406             if len(segments) < original_count:
407                 logger.info(
408                     "Merged %d overlapping/duplicate segment(s) before stitching: %d ->
%d.",
409                     original_count - len(segments),
410                     original_count,
411                     len(segments),
412                 )
413             if not segments:
414                 raise ValueError("No valid (start < end) highlight segments after merge.")
415 
416             if not os.path.exists(raw_video_path):
417                 raise FileNotFoundError(
418                     f"[COMPILER ERROR] Source video file not found: {raw_video_path}"
419                 )
420 
421             # Defense-in-depth: never let the output name escape output_dir (the API also
422             # validates this at the request boundary).
423             output_path = os.path.join(self.output_dir, os.path.basename(output_filename))
424 
425             # Probe video duration so we can detect out-of-range segments
426             video_duration = self._get_video_duration(raw_video_path)
427             if video_duration > 0:
428                 logger.info(f"Source video duration: {video_duration:.2f}s")
429             else:
430                 logger.warning(
431                     "Video duration unknown. Cannot validate segment boundaries against
video length."
432                 )
433 
434             # We will create temporary clips and stitch them using FFmpeg concat demuxer
```

```
            temp_clips = []
            skipped_segments = []

            # Create a temp directory within output_dir
            with tempfile.TemporaryDirectory(dir=self.output_dir) as tmp_dir:
                logger.info(
                    f"Trimming {len(segments)} segments
(begin_padding={self.begin_padding}s, end_padding={self.end_padding}s)..."
                )
                # Build the (optional) watermark filtergraph once; it is applied during
every
                # per-clip re-encode so the mark is baked into each clip identically.
                watermark_filter = self._watermark_filter(tmp_dir)
                if watermark_filter:
                    logger.info("[watermark] applying branding overlay to each clip.")
                for idx, (raw_start, raw_end) in enumerate(segments):
                    clip_path, skip_record = self._trim_one_segment(
                        idx,
                        raw_start,
                        raw_end,
                        segments,
                        video_duration,
                        tmp_dir,
                        raw_video_path,
                        watermark_filter,
                    )
                    if clip_path is not None:
                        temp_clips.append(clip_path)
                    if skip_record is not None:
                        skipped_segments.append(skip_record)

                # Report summary of skipped segments
                if skipped_segments:
                    logger.info(
                        f"\n[SUMMARY] {len(skipped_segments)} out of {len(segments)}
segments were SKIPPED:"
                    )
                    for seg_idx, seg_start, seg_end, reason in skipped_segments:
                        logger.info(
                            f"  - Segment #{seg_idx + 1} [{seg_start:.2f}s -
{seg_end:.2f}s]: {reason}"
                        )
                    logger.info("")

                if not temp_clips:
                    raise RuntimeError(
                        f"[COMPILER ERROR] Failed to generate ANY valid highlight clips. "
                        f"All {len(segments)} segments were skipped. "
                        f"Check the errors above for details. "
                        f"Common causes: video file is corrupted, segments are out of range,
or ffmpeg is not installed."
                    )

                logger.info(
                    f"Successfully trimmed {len(temp_clips)}/{len(segments)} clips.
Preparing to stitch..."
                )

                # Create a list file for FFmpeg concat demuxer
                list_file_path = os.path.join(tmp_dir, "clips.txt")
                # encoding pinned: a non-ASCII clip path would crash on Windows (cp1252
default);
                # ffmpeg's concat demuxer reads the list as UTF-8 on every OS.
                with open(list_file_path, "w", encoding="utf-8") as f:
                    for clip in temp_clips:
```

```
493                         # The concat demuxer resolves relative entries against the list
file's
494                         # directory. Use absolute paths so a temp-dir clip does not become
495                         # output/tmp.../output/tmp.../clip_0.mp4 on Windows.
496                         safe_path = Path(clip).resolve().as_posix().replace("'", "'\\''")
497                         f.write(f"file '{safe_path}'\n")
498
499                 logger.info("Stitching clips together...")
500                 # Stitch them together without re-encoding (since they were encoded with
identical settings)
501                 concat_cmd = [
502                     ffmpeg_bin(),
503                     "-y",
504                     "-f",
505                     "concat",
506                     "-safe",
507                     "0",
508                     "-i",
509                     list_file_path,
510                     "-c",
511                     "copy",
512                     output_path,
513                 ]
514
515                 try:
516                     timeout = ffmpeg_timeout_sec()
517                     subprocess.run(
518                         concat_cmd, capture_output=True, check=True, timeout=timeout
519                     )
520
521                     # Final verification
522                     if not os.path.exists(output_path) or os.path.getsize(output_path) == 0:
523                         raise RuntimeError(
524                             f"[COMPILER ERROR] FFmpeg concatenation appeared to succeed, "
525                             f"but output file is missing or empty: {output_path}"
526                         )
527
528                     output_size_mb = os.path.getsize(output_path) / (1024 * 1024)
529                     logger.info(
530                         f"Dynamic highlight reel saved: {output_path} ({output_size_mb:.1f}
MB)"
531                     )
532                     return output_path
533                 except subprocess.CalledProcessError as e:
534                     stderr_msg = e.stderr.decode(errors="replace").strip()
535                     raise RuntimeError(
536                         f"[COMPILER ERROR] FFmpeg concatenation failed.\n"
537                         f"  Command: {' '.join(concat_cmd)}\n"
538                         f"  FFmpeg stderr: {stderr_msg[-500:]}"
539                     ) from e
540                 except subprocess.TimeoutExpired as e:
541                     raise RuntimeError(
542                         f"[COMPILER ERROR] FFmpeg concatenation timed out after
{timeout_label(timeout)}.\n"
543                         f"  Command: {' '.join(concat_cmd)}"
544                     ) from e
545
```

6. assistant (2026-07-08T05:54:20.528Z)

The compiler.py I read has hardcoded `libx264` ? no `_select_encoder`/`encoder` knob that the task attributes to #521/#525. Let me reconcile: check whether origin/master has moved past my local HEAD, and find where `_select_encoder` actually lives.

7. user (2026-07-08T05:54:25.262Z)

```
===HEAD vs origin/master===
bf97700 docs(runlogs): record live SPA CUJ closing the 2 owner-gated validation-latency cells
(§9) (#522)
bf97700 docs(runlogs): record live SPA CUJ closing the 2 owner-gated validation-latency cells
(§9) (#522)

===commits on origin/master not in HEAD===

===search _select_encoder===
---local working tree---
```

8. user (2026-07-08T05:54:25.285Z)

```
File does not exist. Note: your current working directory is C:\Users\avidu\Projects\khelsutra-
guru\badminton-highlight-indexer.
```

9. assistant (2026-07-08T05:54:37.986Z)

```
Key finding: `_select_encoder` doesn't exist anywhere in origin/master, and `PHASE_PLACEMENT.md`
doesn't exist at that path. HEAD == origin/master == bf97700. The task's premises (PR #525
merged NVENC/`_select_encoder`; PHASE_PLACEMENT.md) may not be in master yet. Let me establish
ground truth.
```

10. user (2026-07-08T05:54:41.967Z)

```
===COMPUTE_DECOUPLED_SERVING dir===
architecture.svg
README.md
runlogs
TESTING_STRATEGY.md

===runlogs dir===
DEPLOY_REPORT_cloud-serving_2026-07-04.md
DEPLOY_REPORT_cloud-serving_a1-cuj_2026-07-04.md
DEPLOY_REPORT_cloud-serving_a1-e2e_2026-07-04.md
DEPLOY_REPORT_cloud-serving_alpha-all1v1_2026-07-06.md
DEPLOY_REPORT_cloud-serving_cp-rebuild_2026-07-05.md
DEPLOY_REPORT_cloud-serving_gpu-resident-nvdec_2026-07-07.md
DEPLOY_REPORT_cloud-serving_validation-latency_2026-07-07.md
README.md
RUNLOG_cloud-serving_exec-status-fastfail_2026-07-07.md
RUNLOG_truly-local_TEMPLATE.md

===PHASE_PLACEMENT anywhere===

===git log for #521 #525===
8866ffe perf(serving): run compile/stitch on the L4 worker; config-driven phase?machine
placement (#521)
e99d0c3 feat(stitch): localize the reel watermark to the UI locale (+ OFL Noto fonts) (#304)
2afbadf fix(dryrun): install the watermark font in bootstrap + track the UI-trigger asks (#292)
8736e2d fix(runner): honor stitching watermark/padding + add --detect-from for hi-res reels
(#205)
ffdfde0 Merge pull request #103 from avidullu:claude/watermark-bundled-font
112e75d feat(stitcher): bundle Apache-2.0 default watermark font + log fallback reason
10864f3 feat(stitcher): configurable branding watermark on highlight reels (#101)
```

11. assistant (2026-07-08T05:55:01.825Z)

```
#521 is merged (8866ffe "run compile/stitch on the L4 worker; config-driven phase?machine
placement") but there's **no #525, no PHASE_PLACEMENT.md, no `_select_encoder`, no NVENC**
anywhere in the tree. The task describes a state ahead of master. Let me verify #521's actual
contents and check for any NVENC/encoder work in unmerged branches.
```

12. user (2026-07-08T05:55:06.990Z)

```
===is 8866ffe(#521) ancestor of HEAD?===
NO - #521 not in HEAD

===what #521 changed===
8866ffe perf(serving): run compile/stitch on the L4 worker; config-driven phase?machine
placement (#521)
 backend/compute/__init__.py                       |   2 +
 backend/compute/base.py                           |  34 +++
 backend/compute/cloud_run.py                      | 100 ++++++-
 backend/config/__init__.py                        |  16 ++
 backend/config/models.py                          |  52 ++++
 backend/config/placement.py                       |  59 ++++
 backend/config/presets.py                         |  10 +-
 backend/orchestration.py                          | 322 ++++++++++++++++++---
 backend/pipeline/stitcher/compiler.py             |  94 ++++++-
 backend/worker/__main__.py                        | 139 ++++++++++
 docs/COMPUTE_DECOUPLED_SERVING/PHASE_PLACEMENT.md | 169 ++++++++++++
 docs/COMPUTE_DECOUPLED_SERVING/README.md          |   1 +
 tests/test_api_cloud_dispatch.py                  |  70 +++++
 tests/test_compile_dispatch.py                    | 204 ++++++++++++++
 tests/test_compiler.py                            |  64 +++++
 tests/test_compute_backend.py                     | 111 ++++++++
 tests/test_config_deployment.py                   |   5 +
 tests/test_phase_placement.py                     | 138 ++++++++++
 tests/test_worker.py                              | 175 ++++++++++++
 19 files changed, 1710 insertions(+), 55 deletions(-)

===NVENC/h264_nvenc anywhere in all branches===
8866ffe2757e8fb87b84dcdb4c4158fa1f42f98f:backend/compute/base.py
8866ffe2757e8fb87b84dcdb4c4158fa1f42f98f:backend/compute/cloud_run.py
8866ffe2757e8fb87b84dcdb4c4158fa1f42f98f:backend/config/models.py
8866ffe2757e8fb87b84dcdb4c4158fa1f42f98f:backend/config/placement.py
8866ffe2757e8fb87b84dcdb4c4158fa1f42f98f:backend/config/presets.py
8866ffe2757e8fb87b84dcdb4c4158fa1f42f98f:backend/orchestration.py
8866ffe2757e8fb87b84dcdb4c4158fa1f42f98f:backend/pipeline/stitcher/compiler.py
8866ffe2757e8fb87b84dcdb4c4158fa1f42f98f:backend/worker/__main__.py
8866ffe2757e8fb87b84dcdb4c4158fa1f42f98f:deploy/cloudrun/Dockerfile
8866ffe2757e8fb87b84dcdb4c4158fa1f42f98f:deploy/digitalocean/gpu_setup.sh
(end nvenc search)
```

13. user (2026-07-08T05:55:07.825Z)

```
===all remote branches===
  origin/HEAD -> origin/master
  origin/chore/revendor-hosted-compute-picker
  origin/docs/runlog-cloud-run-exec-status-fastfail
  origin/docs/validation-latency-live-cuj-2026-07-08
  origin/feat/521-compile-on-l4
  origin/feat/validation-result-cache
  origin/fix/cloud-run-exec-status-fastfail
  origin/fix/hosted-local-detection-guardrail
  origin/master

===local branches===
  chore/revendor-hosted-compute-picker
+ claude/compile-reliability
+ claude/wasb-config-validation
* codex/issue-523-sd-reel-plan
+ feat/521-compile-on-l4
+ feat/524-l4-primary
  feat/submit-md5-cache-reuse
  fix/cloud-run-exec-status-fastfail
  fix/hosted-local-detection-guardrail
  master
  perf/validation-shared-decode-pass
```

14. assistant (2026-07-08T05:55:47.755Z)

Now the picture is clear. **#521 is unmerged** ? it lives on `origin/feat/521-compile-on-l4` (commit 8866ffe), which is where `PHASE_PLACEMENT.md`, `_select_encoder`, and the NVENC work actually are. There is no #525 in git. The task treats #521 as merged, but master (bf97700) doesn't contain it. Let me establish the branch relationship and read the real base files from the #521 branch.

15. user (2026-07-08T05:55:52.023Z)

```
===is master(bf97700) ancestor of feat/521-compile-on-l4?===
NO - #521 branch is behind master

===merge-base===
699d85bc0abd6e35d3196a4566dcec23f649394e

===commits on feat/521 not in master===
8866ffe perf(serving): run compile/stitch on the L4 worker; config-driven phase?machine
placement (#521)

===commits on master not in feat/521===
bf97700 docs(runlogs): record live SPA CUJ closing the 2 owner-gated validation-latency cells
(§9) (#522)
```

16. user (2026-07-08T05:55:54.472Z)

```
===git remote===
origin  https://github.com/Khelsutra/badminton-highlight-indexer.git (fetch)
origin  https://github.com/Khelsutra/badminton-highlight-indexer.git (push)

===#521 branch compiler.py: _select_encoder + watermark region===
26:#: Per-encoder availability memo (#521): probing ffmpeg is a subprocess, so a run's first
clip pays
27:#: it and the rest reuse the verdict. Keyed by encoder name; cleared only across processes.
31:def _probe_encoder_available(encoder: str) -> bool:
32:    """Can this ffmpeg build actually RUN ``encoder`` (not merely list it)? Runs a tiny
1-frame
34:    real test encode ? not ``-encoders`` grep ? is what distinguishes "usable" from "will
fail at
35:    runtime". Memoized. Any error / timeout ? unavailable (caller falls back to libx264)."""
36:    if encoder in _ENCODER_AVAILABLE:
37:        return _ENCODER_AVAILABLE[encoder]
40:    # budget, or 30 s when unbounded) so a wedged encoder can't hang the whole stitch.
54:                "-c:v",
55:                encoder,
66:    _ENCODER_AVAILABLE[encoder] = ok
77:        encoder: str = "libx264",
85:        # Video encoder for the per-clip trim re-encode (#521). "libx264" (CPU) is the
default and
86:        # works everywhere; "h264_nvenc"/"hevc_nvenc" use the L4's HARDWARE encoder for the
4K
87:        # re-encode the watermark forces. The requested encoder is PROBED once per run
88:        # (_select_encoder) and falls back to libx264 if this ffmpeg build can't run it.
89:        self.encoder = (encoder or "libx264").strip() or "libx264"
210:    def _watermark_filter(self, tmp_dir: str) -> Optional[str]:
211:        """Build the ffmpeg -vf filtergraph that burns the configured watermark (text
213:        watermarking is disabled or has nothing to draw. The concat stage stays -c copy
277:    def _select_encoder(self) -> Tuple[List[str], str]:
278:        """Resolve the ``-c:v ?`` args used for EVERY per-clip trim re-encode this run, plus
the
279:        chosen encoder name. Chosen ONCE (not per clip) so all clips share one codec and the
final
280:        concat can stay ``-c copy``. NVENC (hardware) is the L4 win on the 4K re-encode; if
the
281:        requested NVENC encoder can't run in this ffmpeg build (CI / non-GPU box), fall back
```

```
to
282:        libx264 so a misconfig never nukes the reel."""
283:        enc = self.encoder
284:        if enc in ("h264_nvenc", "hevc_nvenc") and not _probe_encoder_available(enc):
286:                "[compile] encoder %r is not runnable in this ffmpeg build ? falling back to
"
287:                "libx264 (CPU). On the L4 worker this should be available.",
290:            enc = "libx264"
291:        if enc == "libx264":
293:            return ["-c:v", "libx264", "-preset", "superfast", "-crf", "22"], enc
297:            ["-c:v", enc, "-preset", "p4", "-cq", "23", "-pix_fmt", "yuv420p"],
310:        watermark_filter: Optional[str],
374:        # Default to libx264 (the historical settings) when a caller invokes
_trim_one_segment
375:        # directly without resolving an encoder ? keeps this method's old behaviour intact.
379:            else ["-c:v", "libx264", "-preset", "superfast", "-crf", "22"]
396:        vf_args = ["-vf", watermark_filter] if watermark_filter else []
400:        # whole reel. Clips stay codec-identical either way, so concat -c copy holds.
458:    def compile_highlights(
470:           IDENTICAL encoder settings (codec, preset, CRF, pixel format) so the final
471:           concat can stream-copy (``-c copy``) without re-encoding. Changing the
521:            watermark_filter = self._watermark_filter(tmp_dir)
522:            if watermark_filter:
524:            # Resolve the video encoder ONCE (probe NVENC / fall back) so every clip shares
one codec
525:            # and the final concat stays -c copy (#521).
526:            video_codec_args, chosen_encoder = self._select_encoder()
527:            if chosen_encoder != "libx264":
528:                logger.info("[compile] encoding clips with %s (hardware).", chosen_encoder)
538:                    watermark_filter,
```

17. assistant (2026-07-08T05:56:34.044Z)

Now the full picture is clear, and it's important:

- `origin/feat/521-compile-on-l4` (commit 8866ffe, **unmerged**) has diverged from master: it was cut from 699d85b, and master's `bf97700` (#522, which added the §9 CUJ the task cites) landed *after*. So each branch has one commit the other lacks.
- The base this task depends on ? `_select_encoder`, the `encoder` knob, `PHASE_PLACEMENT.md` ? exists **only** on the #521 branch, not on master.

Let me read the real base files before deciding how to proceed. I'll dump the #521-branch versions to scratchpad and read them alongside master's runlog §9 and README.

18. user (2026-07-08T05:56:40.510Z)

```
dumped OK
  626 C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-khelsutra-guru-badminton-
highlight-indexer/46eabfd2-0e71-4ff5-a68d-c76f474d4ff3/scratchpad/compiler_521.py
  169 C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-khelsutra-guru-badminton-
highlight-indexer/46eabfd2-0e71-4ff5-a68d-c76f474d4ff3/scratchpad/PHASE_PLACEMENT_521.md
  795 total
```

19. user (2026-07-08T05:56:40.652Z)

```
1       # DEPLOY REPORT ? cloud-serving, validation-latency closure (#512 shared-decode + #513
worker skip-revalidation), 2026-07-07
2
3       > Durable **closure record** for the two validation-latency fixes now live on the
`cloud-serving` path: **#512** (`perf(validators)`: decode `scene_cut`/`blur`/`relevance` in ONE
shared pass instead of ~125 random per-sample seeks across 3 captures) and **#513**
(`fix(serving)`: the worker skips its redundant re-validation when the control-plane already
validated + dispatched, via `--skip-validation`). Both shipped in the live image `rally-
worker@sha256:8045f54e?` (built from `origin/master` @ `f5c9dd9`; #512 `20862d7`, #513 `23c1868`
```

are ancestors). Measures the before/after on 3 clips spread across codecs/resolutions, all against the real serving infra. Project `gen-lang-client-0306026826`, region `asia-southeast1`. Honest-limits (incl. the owner-gated live CUJ) in §6.

1. TL;DR

Stage	Result
Live image carries #512+#513	? control-plane rev `00018-qvz` + worker Job both on `sha256:8045f54e?` (= `f5c9dd9`, ancestors `20862d7`/`23c1868` verified)
Baseline captured (pre-fix)	? from prod logs on rev `00017-qbd` (image `09d2edf7` = `66756ad`, pre-#512/#513) ? no re-run needed
#513 worker skip-revalidation	? real dispatch-style exec: Video 1 validation **SKIPPED (0 s)**, runs to **SUCCESS / 13 rallies** ? the blur-reject that failed the live CUJ is **gone**
#512 shared-decode speedup (real L4)	? Video 1 re-validation **169.3 s ? 69.7 s (2.43×)** on identical worker hardware, decision byte-identical (sharpness 11.9 < 20.0 both)
#512 speedup (off-box A/B, breadth)	? 1080p H.264 **1.95×**, 4K HEVC **3.10×**, 5.3K 10-bit HEVC **3.23×**; every validator decision unchanged OLD?NEW
Live control-plane "after" number	? **live-corroborated 2026-07-08** (§9): rev `00019` validated a fresh 1.83 GiB / 174 s clip in **183 s** (in the 2?3 min band) + reel compiled end-to-end. Video-1's exact 412 s?~170 s A/B stays a projection (the live CUJ used a different, heavier clip).

Bottom line: the two fixes do exactly what they claim on live serving hardware. **#513** turns the soft 5.3K clip from a **hard FAIL** (worker re-validated at its default `min_blur=20`, rejected sharpness 11.9, `rc2`, no reel) into a **PASS with 13 rallies** by trusting the control-plane's gate. **#512** cuts the frame-sampling validators ~**2?3×** (2.43× measured on the L4; 1.95?3.23× off-box across codecs) with **zero decision drift** OLD?NEW. The only number still owner-pending is the live control-plane validation wall on rev 00018 ? its components are measured and it projects to ~2?3 min from the ~6m52s baseline.

2. Resources

- **Control-plane:** `rally-control-plane` Cloud Run **service**, rev **`rally-control-plane-00018-qvz`** (100% traffic), image `asia-southeast1-docker.pkg.dev/gen-lang-client-0306026826/rally/rally-worker@sha256:8045f54ea9fcbf?` (built from `f5c9dd9`), `DEPLOYMENT_PROFILE=cloud-serving`, CPU-only **2 vCPU**, edge-auth ON (`edge_auth_enabled=true`, `cf_team_domain=khelsutra.cloudflareaccess.com`).
- **Worker:** `rally-worker` Cloud Run **Job**, same image `sha256:8045f54e?`, 8 vCPU / 32 GiB / 1× NVIDIA **L4**, `WASB_WEIGHTS=/gcs/models/wasb_badminton_best.pth.tar`, `WASB_DECODE_BACKEND=nvdec_resident`, GCSFuse mount of `gs://khelsutra-rally-serving`.
- **Baseline (pre-fix) surface:** control-plane rev **`00017-qbd`** / worker image `sha256:09d2edf7?` (= `66756ad` = #507+#508+#509, both **pre-#512/#513**) ? the source of the prod baseline logs below (2026-07-07, before the 16:15 rev-00018 cutover).
- **Corpus:** Video 1 `gs://khelsutra-rally-serving/videos/uploads/702636eed76145e88aa59c3000c7e415/Video 1 - Badminton.MP4` (1.41 GiB, 5312×2988 yuv420p10le HEVC, 3019 frames, MD5 `8fe6b719603dd7a7b2b6ef543139a4a3`); `gs://khelsutra-rally-corpus/videos/golden_1080p/Boxhill_Doubles.mp4` (1080p H.264); `gs://khelsutra-rally-corpus/videos/golden/GX010128.mp4` (2.18 GiB, 4K HEVC, 45298 frames).

3. Exact commands

```bash
# --- Provenance: prove the live image carries both fixes ---
gcloud run services describe rally-control-plane --region asia-southeast1 \
  --format="value(status.latestReadyRevisionName,
spec.template.spec.containers[0].image)"   # 00018-qvz  ?@sha256:8045f54e?
git merge-base --is-ancestor 20862d7 f5c9dd9 && echo "#512 in live image"   # yes
git merge-base --is-ancestor 23c1868 f5c9dd9 && echo "#513 in live image"   # yes

# --- #513 proof: worker A/B on the LIVE image (Video 1 already staged in the serving bucket) ---
V1="gs://khelsutra-rally-serving/videos/uploads/702636eed76145e88aa59c3000c7e415/Video 1 - Badminton.MP4"
# Run A ? WITH --skip-validation (exactly what orchestration._maybe_dispatch_remote emits on dispatch):
```

```
      gcloud run jobs execute rally-worker --region asia-southeast1 --async \
        --args="^@@^infer@@--video-uri=$V1@@--out-uri=gs://khelsutra-rally-
serving/_valclose/v1-skipval@@--segmenter=wasb_hybrid@@--skip-validation@@--
set=deployment.compute_target=local_gpu@@--set=indexing.detector_impl=native@@--
set=indexing.skip_ai_handoff=true@@--set=indexing.wasb.batch_size=64@@--
set=indexing.wasb.prefetch_batches=4@@--set=indexing.wasb.timeout_sec=0"
      # Run B ? WITHOUT --skip-validation (worker re-validates with the NEW #512 validators ?
still blur-rejects):
      gcloud run jobs execute rally-worker --region asia-southeast1 --async \
        --args="^@@^infer@@--video-uri=$V1@@--out-uri=gs://khelsutra-rally-serving/_valclose/v
1-reval@@--segmenter=wasb_hybrid@@--set=deployment.compute_target=local_gpu@@--
set=indexing.detector_impl=native@@--set=indexing.skip_ai_handoff=true"

      # --- #512 timing: bracket "pulled ? -> in.mp4"  ?  "validation FAILED/SKIPPED"  ?
"Phase 2 (WASB) completed" ---
      gcloud logging read 'resource.type="cloud_run_job" AND
labels."run.googleapis.com/execution_name"="rally-worker-<exec>"' \
        --freshness 1d --format="value(timestamp, textPayload)"

      # --- #512 off-box A/B (isolated clones; run_all decodes the SAME sample indices in both
trees) ---
      git clone --local --no-hardlinks <repo> repo_old && git -C repo_old checkout 23c1868   #
pre-#512 (per-sample seeks)
      git -C repo_old worktree add --detach repo_new f5c9dd9
# live image src (shared pass)
      python valbench.py <repo_old|repo_new> <video> <label> 2   # times
registry.run_all(video, load_config())
      ```

      ## 4. Run results

      ### 4a. Video 1 (5312×2988 10-bit HEVC, 3019 frames) ? the clip that motivated #512/#513

      | Metric | BEFORE ? pre-fix (rev `00017`, img `09d2edf7`) | AFTER ? #512+#513 live (rev
`00018`, img `8045f54e`) |
      |---|---|---|
      | Control-plane validation | **411?412 s** (6m52s) ? passed (baked `min_blur=8.0`) |
projected ~170 s (2?3 min); **live-corroborated 2026-07-08 ? 183 s** on rev `00019` (§9 ? a
fresh 1.83 GiB clip, not a Video-1 re-run) |
      | Worker (re-)validation | **169.3 s** ? ? **BLUR-REJECT** (sharpness 11.9 < default
20.0) | **0 s ? SKIPPED** (`--skip-validation`, #513) |
      | ? counterfactual: worker re-val if NOT skipped | *(169.3 s, OLD
validators)* | **69.7 s** (NEW #512 validators, **2.43×**, same 11.9<20 decision) |
      | End-to-end | ~10m31s wall ? **FAILED** (0 rallies, no reel) | worker ? **SUCCESS** in
~2m38s exec ? **13 rallies**; **reel live-confirmed 2026-07-08** (§9 ? fresh-clip CUJ compiled
an 11-clip reel end-to-end) |
      | Rallies produced | **0** (rejected before detection) | **13** (`report.json`
`status=success`, 13 segments) |
      | Pass / fail | ? **FAIL** | ? **PASS** |

      Prod baseline evidence (rev 00017, 2026-07-07, unmodified): control-plane `13:49:18
Running validations ? 13:56:10 Video passed checks` = 411.4 s (and `14:03:56 ? 14:10:48` = 411.9
s). Worker exec `rally-worker-2lbz2` (dispatched by the 14:03 run, OLD image): `pulled 14:11:35
? validation FAILED: blur_check ? sharpness 11.9 below 20.0 @14:14:24 ? exit(2)` ? a
successful GPU dispatch that failed after the fact, the exact CUJ #513 fixes.

      After evidence (live image `8045f54e`, 2026-07-07):
      - `rally-worker-k29rk` (**Run A, --skip-validation**): `pulled 17:09:24 ? [worker]
validation SKIPPED (--skip-validation) 17:09:27 ? native WASB (torch 2.6.0+cu124, cuda True) ?
Phase 2 (WASB) completed in 100.9 s, 13 candidate windows (1510/3019 visible) ? emitted 13
rallies ? exit(0)`. Exec wall 157.7 s; `gpu_active`?100.9 s (WASB);
`bytes_out`=`report.json`+`intervals.csv`. Output
`gs://?/_valclose/v1-skipval/outputs/8fe6b719?/`, `status=success`, 13 segments.
      - `rally-worker-t8llx` (**Run B, re-validation on the NEW image**): `pulled 17:09:29 ?
validation FAILED: blur_check ? sharpness 11.9 below 20.0 @17:10:39 ? exit(2)`. Re-val wall
```
```

69.7 s vs the OLD image's 169.3 s = **2.43× on identical L4 hardware**, and the **decision is byte-identical** (11.9 < 20.0) ? #512 is pure speed, zero behavior change. (This run also shows *why* #513 is needed: absent the skip, the worker's default `min_blur=20` still rejects the clip the control-plane passed at 8.0.)

4b. Off-box #512 A/B ? codec/resolution breadth (16-core local box, cv2 4.13.0, warm, `run_all`)

Isolated clones at `23c1868` (OLD, per-sample seeks) vs `f5c9dd9` (NEW, shared pass); identical sample indices ? a fair A/B. **Every validator decision is identical OLD?NEW** on every clip (no PASS/FAIL drift from the fix).

Clip	codec / res	OLD (warm)	NEW (warm)	Speedup	decisions OLD?NEW
Boxhill_Doubles	1080p H.264	10.91 s	5.60 s	**1.95×**	6/6 identical (all PASS)
GX010128	4K HEVC (45298 fr)	35.76 s	11.54 s	**3.10×**	6/6 identical (all PASS)
Video 1	5.3K 10-bit HEVC	173.20 s	53.62 s	**3.23×**	6/6 identical (all PASS?)

? On this Windows/cv2 box the 10-bit-HEVC decode yields a Laplacian variance **above** 20 so blur PASSES off-box, whereas the Linux serving stack measures **11.9** and rejects. That platform decode-stack difference is **orthogonal to #512** (which changes only *which frames are sampled*, identically in both trees); the authoritative blur verdict and the real speedup come from the L4/prod (§4a). Off-box contributes only the OLD?NEW **ratio**, which is decode-stack-independent.

The win grows with per-frame decode cost (random-seek re-decode is what #512 removes), and it **triangulates** with the real-L4 Video 1 A/B (2.43×): the off-box ratios bound the control-plane projection.

4c. Control-plane projection (2-vCPU)

Baseline **~412 s**. Applying the L4-measured **2.43×** ? **~170 s (~2m50s)**; the off-box ratios (1.95?3.23×) bracket **~2?3 min**. Matches the pre-fix design estimate (~6.5 min ? ~2 min). Exact live number is owner-gated (§6).

5. Rollout state

- **Already deployed ? this is a measurement/closure record, not a deploy.** #512+#513 went live with the `8045f54e` image cutover (control-plane rev `00018-qvz`, worker Job same digest) on 2026-07-07. No infra change was made for this report.
- **Rollback (pre-existing levers):** control-plane `gcloud run services update-traffic rally-control-plane --region asia-southeast1 --to-revisions rally-control-plane-00017-qbd=100`; worker `gcloud run jobs update rally-worker --region asia-southeast1 --image ?/rally-worker@sha256:09d2edf7?`. Both revert to the pre-#512/#513 image.
- **Scratch outputs** written under `gs://khelsutra-rally-serving/_valclose/` (isolated prefix; safe to delete).

6. Honest limits / not-verified

- **Live end-to-end CUJ is owner-gated.** ? **CLOSED 2026-07-08 ? see §9** (owner drove one live SPA upload on rev `00019`; both ? cells now measured). Original limit, retained for context: `POST /api/process` on rev 00018 requires a Cloudflare-Access JWT (`edge_auth_enabled=true`; a direct call returns **401** `missing Cf-Access-Jwt-Assertion header`). So the two ? cells ? the **live control-plane validation wall on rev 00018** and the **stitched reel** ? need the owner to drive one upload (or hand over a CF-Access JWT / service token). Everything else here is measured on the real serving infra. **Recipe to close them:** upload "Video 1 - Badminton.MP4" via the SPA (compute=cloud), then `gcloud logging read '?service_name="rally-control-plane"?("Running validations" OR "passed checks")'` for the wall, and confirm the dispatched worker exec logs `validation SKIPPED` + emits rallies + the reel compiles. Expect CP validation ~2?3 min and a PASS.
- **#513's worker win is shown via a dispatch-faithful direct exec**, not a control-plane-initiated dispatch (same auth gate). The `--skip-validation` flag, its emission in `orchestration._maybe_dispatch_remote`, and its handling in `worker.cmd_infer` are unit-tested on master; Run A exercises the exact worker code path with the exact flag. ? **UPDATE 2026-07-08 (§9): now also proven on a genuine control-plane-initiated dispatch** (`rally-worker-64bhj`):

`_maybe_dispatch_remote` emitted `--skip-validation` on a live SPA run and the worker honored it (`validation SKIPPED`, 0 re-validation).
- **Rallies, not a reel.** Run A used `skip_ai_handoff=true` (secret-free) so the 13 "rallies" are local candidate windows (no Gemini refine, no stitched mp4) ? this validates decode+detect+windowing end-to-end and that the clip **passes**; the reel stitch is the owner-CUJ step above. The 13 segments are its inputs.
- **Off-box absolute times are from a Windows 16-core box (cv2/FFmpeg), not the 2-vCPU Linux control-plane** ? so the **ratios** transfer (the change is algorithmic: seeks?shared forward-hybrid pass), the absolutes don't, and (per ?) the 10-bit blur *verdict* can differ; the control-plane absolute is anchored to the prod 412 s baseline, not the local numbers. Video 1 off-box is a cross-check of the authoritative real-L4 2.43×.
- **#515** (crash-before-marker fast-fail) is NOT in this image (`ef41473` post-dates `f5c9dd9`); it's unrelated to validation latency and out of scope for this closure.

7. Cost

- 2 L4 worker execs (Run A 157.7 s + Run B 109.6 s ? 268 s of L4 time) ? **~$0.5?0.7**. Off-box A/B: **$0** (local). No VM, no image build. GCS read for the 3 downloaded clips (~5.1 GiB) is intra-project.

8. Closure verdict

? **CLOSED ? #512 and #513 are proven on live cloud-serving hardware.** #513 converts the motivating soft-focus 5.3K clip from a post-dispatch FAIL into a 13-rally PASS by eliminating the worker's redundant re-validation (169 s + wrongful blur-reject ? 0 s); #512 speeds the frame-sampling validators **2.43× on the real L4** and **1.95?3.23× off-box across codecs**, with **zero decision drift** OLD?NEW on every clip. The last owner-gated residual is now **closed live 2026-07-08 (§9)**: an owner-driven SPA CUJ on rev `00019` validated a fresh 1.83 GiB / 174 s clip in **183 s** (in the projected 2?3 min band) and compiled an 11-clip reel end-to-end ? with the worker `validation SKIPPED` on a genuine control-plane dispatch. (The Video-1-specific 412 s?~170 s A/B stays a projection by design ? the live CUJ used a different, heavier clip.) Validation-latency work is done.

9. Update (2026-07-08) ? live SPA CUJ closes the two owner-gated cells (on rev `00019`)

The two ? cells in §4a (the **live control-plane validation wall** and the **end-to-end reel**) were owner-gated on the Cloudflare-Access `POST /api/process`. On **2026-07-08** the owner drove one real SPA upload (compute=**Cloud**) end-to-end, so both are now measured on live serving infra. Mirrors the gpu-resident report's "§5 Update" append ? the body above is unchanged; this section records what the live run added.

Two honest deltas from the recipe (state moved after this report's 2026-07-07 image cutover):

1. **Live rev rolled `00018` ? `00019`.** Traffic is 100% on **`rally-control-plane-00019-96f`**, image `rally-worker@sha256:c6393d27b4ae?` (tag `:latest`), deployed 2026-07-07T17:06Z ? ~51 min after the `00018`/`8045f54e` cutover §2 documents; the worker Job is on the same `:latest`=`c6393d27` digest (both surfaces match). It's an undocumented forward-bump, but #512 (`20862d7`) and #513 (`23c1868`) are ancestors of `origin/master`, `f5c9dd9` (the `00018` src) is an ancestor of master, and the run **empirically** shows both fixes live (one-pass validation wall + worker `validation SKIPPED`). Measured on what is actually serving.
2. **A fresh clip, not Video 1.** The upload was **"First CUJ.MP4"** ? **1.83 GiB** (1,960,043,219 B), source duration **174.17 s** (?84 Mbps, 4K-class ? *heavier* than Video 1's 1.41 GiB), video_id `2f31b45e?`. So the live wall **corroborates** the Video-1 projection on a comparably-heavy clip; it is **not** a re-measurement of Video 1's 412 s?~170 s A/B (a different clip has a different absolute wall).

Full phase breakdown (UTC 2026-07-08, rev `00019`):

#	Phase	Window	Wall	Machine	Notes
1	Upload (client?server, 80-part multipart)	04:25:56 ? 04:28:51	**~2m55s**	client uplink	1.83 GiB @ ~84 Mbps ? owner bandwidth, not infra
2	Accept ? validation start (stage)	04:28:52 ? 04:29:22	~30 s	control-plane	stage upload to serving bucket
3	**Validation (#512 shared-decode)** ?	04:29:22 ? 04:32:25	**183 s (3m03s)**	control-plane 2 vCPU	one-pass scene_cut/blur/relevance; PASSED

```
120     | 4 | Dispatch ? worker ready (cold start) | 04:32:25 ? 04:33:20 | ~55 s | Cloud Run Job
| image pull + GCSFuse mount + torch/cuda init |
121     | 5 | **Worker skip-validation (#513)** ? | 04:33:20 ? 04:33:24 | ~4 s | L4 |
`validation SKIPPED`, 0 re-validation |
122     | 6 | Detection / segment (WASB) | 04:33:24 ? 04:36:20 | **161.6 s** | L4 (cuda) | 15
candidate windows, 2561 visible pts |
123     | 7 | Ingest + finalize | 04:36:20 ? 04:36:24 | ~4 s | control-plane | intervals?DB,
`status=success` |
124     | ? | (user reviews rallies, clicks Compile) | 04:36:24 ? 04:37:26 | ~62 s idle | ? |
human-in-the-loop |
125     | 8 | **Compile / stitch** ? slow | 04:37:26 ? 04:50:21 | **~12m55s** | control-plane 2
vCPU | 11 clips, ~67 s/clip 4K re-encode+watermark; concat ~2 s; mirror ~11 s |
126
127     Server pipeline (process?success, excl. upload+idle): **7m32s**. Full CUJ (upload?reel):
**~24m25s** wall.
128
129     **Cell 1 ? control-plane validation AFTER (live):** **183 s (3m03s)** on rev `00019`,
PASSED ? the #512 shared-decode validators on the real 2-vCPU serving path, in the projected
**2?3 min** band on a clip *heavier* than Video 1 (pre-#512 per-sample seeks were the
~6m52s-class baseline). Evidence: `Running validations 04:29:22.789 ? Video passed checks
04:32:25.780`.
130
131     **Cell 2 ? end-to-end reel (live):** `/api/compile` stitched **11 of the 15** detected
rallies (0.5 s pad, per-clip branding watermark) from the 174.17 s source ? **`gs://khelsutra-
rally-serving/highlights/2f31b45e?/first-cuj-ks-highlight-reel.mp4`** (**581.84 MiB**, saved
04:50:10 *"Dynamic highlight reel saved (581.8 MB)"*, mirrored 04:50:21, served via
`/api/highlights` 307). **Reel ? produced end-to-end on the live control-plane** ? source fetch
? per-clip trim+watermark ? concat ? mirror-upload, all exercised.
132
133     **Bonus ? #513 proof upgraded** from the §6 dispatch-faithful *direct exec* to a genuine
**control-plane-initiated dispatch** (`rally-worker-64bhj`): `_maybe_dispatch_remote` emitted
`--skip-validation` on a live SPA run and the worker honored it (0 re-validation).
134
135     **? Finding ? stitch is the scaling bottleneck (follow-ups filed).** The compile ran
**~13 min on the CPU-only 2-vCPU control-plane** (~45?90 s per ~7 s clip: the watermark forces a
full 4K re-encode, no NVENC). With validation (183 s) *also* on the control-plane and serialized
by `_LOCAL_COMPUTE_LANE`, one run pins the control-plane for ~16 min of heavy CPU ? it won't
scale past a few concurrent users. Filed: **#521** (move compile/stitch to the L4 worker + make
phase?machine placement config-driven) and **#520** (step-level debug tracing, server+UI, to a
low-latency sink ? this breakdown had to be hand-correlated across 3 log streams).
136
137     **Cost (heuristic):** one L4 detect exec (`rally-worker-64bhj`, ~222 s) ? **$0.15?0.55**
(rate-dependent, dominant) + control-plane CPU for validation+stitch (~960 s @ 2 vCPU) ?
**~$0.05** + reel egress (581 MB, if downloaded to internet) ? **~$0.07** + storage/ops **<
$0.02** + Gemini **$0** (skip_ai_handoff). **? $0.3?0.7 total**, ~one L4 exec as projected ? but
the stitch's real cost is **concurrency** (~16 min of control-plane heavy CPU), not dollars.
138
139     **Net:** both owner-gated cells are now live-measured on rev `00019` ? validation wall
**183 s** (corroborating the 2?3 min projection) and the reel compiles end-to-end. #512 one-pass
validation and #513 skip-validation-on-real-dispatch are now both directly observed in prod. The
Video-1 412 s?~170 s A/B remains a projection by design (different, heavier clip).
140
```

20. assistant (2026-07-08T05:56:54.058Z)

Now I have the CUJ data: §9 row 8 shows **compile/stitch ~12m55s** (11 clips × ~67s/clip 4K re-encode forced by the watermark; concat only ~2s). That's the cost this task targets. Let me read the #521-branch compiler (the real base) and PHASE_PLACEMENT.md §3.

21. user (2026-07-08T05:56:56.970Z)

```
1       import logging
2       import os
3       import subprocess
4       import tempfile
```

```
5       from pathlib import Path
6       from typing import Any, Dict, List, Optional, Tuple, Union
7
8       from backend.utils.ffmpeg import (  # P2a: single ffmpeg/ffprobe resolver
9           ffmpeg_bin,
10          ffmpeg_timeout_sec,
11          ffprobe_bin,
12          ffprobe_timeout_sec,
13          timeout_label,
14      )
15
16      logger = logging.getLogger(__name__)
17
18      # Bundled, commercially-clean default watermark font (Apache-2.0). Shipped so the
overlay
19      # renders out-of-the-box without a system fontconfig setup (a common gap on Windows,
where
20      # `font='Sans'` fails). Overridable via stitching.watermark.font_path. This module lives
at
21      # backend/pipeline/stitcher/, so parents[2] == backend/.
22      _BUNDLED_FONT = (
23          Path(__file__).resolve().parents[2] / "assets" / "fonts" / "RobotoSlab-Regular.ttf"
24      )
25
26      #: Per-encoder availability memo (#521): probing ffmpeg is a subprocess, so a run's
first clip pays
27      #: it and the rest reuse the verdict. Keyed by encoder name; cleared only across
processes.
28      _ENCODER_AVAILABLE: Dict[str, bool] = {}
29
30
31      def _probe_encoder_available(encoder: str) -> bool:
32          """Can this ffmpeg build actually RUN ``encoder`` (not merely list it)? Runs a tiny
1-frame
33          lavfi test encode to null. NVENC is often *compiled in* on boxes without an NVIDIA
GPU, so a
34          real test encode ? not ``-encoders`` grep ? is what distinguishes "usable" from
"will fail at
35          runtime". Memoized. Any error / timeout ? unavailable (caller falls back to
libx264)."""
36          if encoder in _ENCODER_AVAILABLE:
37              return _ENCODER_AVAILABLE[encoder]
38          ok = False
39          # The probe is a tiny 1-frame encode ? bound it to a short timeout (? the configured
ffmpeg
40          # budget, or 30 s when unbounded) so a wedged encoder can't hang the whole stitch.
41          configured = ffmpeg_timeout_sec()
42          probe_timeout = min(30.0, float(configured)) if configured else 30.0
43          try:
44              proc = subprocess.run(
45                  [
46                      ffmpeg_bin(),
47                      "-hide_banner",
48                      "-f",
49                      "lavfi",
50                      "-i",
51                      "color=c=black:s=64x64:d=0.1",
52                      "-frames:v",
53                      "1",
54                      "-c:v",
55                      encoder,
56                      "-f",
57                      "null",
58                      "-",
59                  ],
```

```
                capture_output=True,
                timeout=probe_timeout,
            )
            ok = proc.returncode == 0
        except Exception:  # noqa: BLE001 ? a probe failure/timeout just means "not usable ?
fall back"
            ok = False
        _ENCODER_AVAILABLE[encoder] = ok
        return ok


    class VideoCompiler:
        def __init__(
            self,
            output_dir: str,
            end_padding: float = 1.0,
            begin_padding: float = 0.5,
            watermark: Optional[Any] = None,
            encoder: str = "libx264",
        ):
            self.output_dir = output_dir
            self.end_padding = end_padding
            self.begin_padding = begin_padding
            # Optional branding overlay. Accepts a WatermarkConfig model or a plain dict;
            # normalized to a dict (or None) so this module stays config-package-agnostic.
            self.watermark = self._normalize_watermark(watermark)
            # Video encoder for the per-clip trim re-encode (#521). "libx264" (CPU) is the
default and
            # works everywhere; "h264_nvenc"/"hevc_nvenc" use the L4's HARDWARE encoder for
the 4K
            # re-encode the watermark forces. The requested encoder is PROBED once per run
            # (_select_encoder) and falls back to libx264 if this ffmpeg build can't run it.
            self.encoder = (encoder or "libx264").strip() or "libx264"
            os.makedirs(output_dir, exist_ok=True)

        @staticmethod
        def _normalize_watermark(watermark: Optional[Any]) -> Optional[Dict[str, Any]]:
            if watermark is None:
                return None
            if hasattr(watermark, "model_dump"):
                return watermark.model_dump()
            if isinstance(watermark, dict):
                return dict(watermark)
            return None

        @staticmethod
        def _ff_quote(path: str) -> str:
            """Make a filesystem path safe to embed inside single quotes in an ffmpeg
            filtergraph operand (drawtext fontfile/textfile, movie= source): forward
            slashes (Windows-friendly), escaped single quotes, AND escaped colons.

            The colon MUST be escaped even inside single quotes: ffmpeg's filtergraph
            option parser treats ':' as the option separator, so a Windows drive-letter
            path like ``C:/dir/font.ttf`` otherwise truncates the operand at ``C`` and
            the encode fails with "Invalid argument". This silently disabled the
            on-by-default bundled-font watermark (and absolute logo paths) on Windows."""
            return path.replace("\\", "/").replace("'", r"\'").replace(":", r"\:")

        @staticmethod
        def _parse_time(val: Union[int, float, str]) -> float:
            """Normalizes a timestamp value to float seconds.
            Handles: float/int (pass-through), plain numeric strings ("12.4"),
            and colon-separated strings ("MM:SS", "HH:MM:SS", "H:MM:SS.ms").
            This mirrors the parse_time logic used in gemini.py to ensure
            the stitcher can always consume whatever format the indexer produced."""
```

```
122             if isinstance(val, (int, float)):
123                 return float(val)
124             if isinstance(val, str):
125                 val = val.strip()
126                 if ":" in val:
127                     # Handle MM:SS, HH:MM:SS, etc.
128                     parts = val.split(":")
129                     parts.reverse()
130                     total = 0.0
131                     for i, p in enumerate(parts):
132                         try:
133                             total += float(p) * (60**i)
134                         except ValueError:
135                             pass
136                     return total
137                 try:
138                     return float(val)
139                 except ValueError:
140                     logger.warning(
141                         f"Could not parse timestamp value '{val}' as a number. Defaulting to
0.0."
142                     )
143                     return 0.0
144             logger.warning(
145                 f"Unexpected timestamp type {type(val).__name__} for value '{val}'.
Defaulting to 0.0."
146             )
147             return 0.0
148 
149         @staticmethod
150         def _merge_overlapping(
151             segments: List[Tuple[float, float]],
152         ) -> List[Tuple[float, float]]:
153             """Normalize, sort, and merge overlapping/duplicate (start, end) ranges.
154 
155             A highlight reel must never replay the same footage. The same rally can land in
156             the segment list more than once ? e.g. two detector runs accumulated for one
157             video (add_video is an UPSERT, so re-running without a prune leaves both sets),
or
158             chunk-overlap duplicates near boundaries. Stitching those verbatim plays the
rally
159             twice. Merging any range that overlaps or abuts the previous one collapses true
160             duplicates/overlaps while preserving genuinely distinct (gapped) rallies.
161             Degenerate rows (end <= start) are dropped. Timestamps may be float/int/str.
162             """
163             parsed = []
164             for raw_s, raw_e in segments:
165                 s = VideoCompiler._parse_time(raw_s)
166                 e = VideoCompiler._parse_time(raw_e)
167                 if e > s:
168                     parsed.append((s, e))
169             parsed.sort()
170             merged: List[Tuple[float, float]] = []
171             for s, e in parsed:
172                 if merged and s <= merged[-1][1]:  # overlaps/abuts previous -> extend it
173                     merged[-1] = (merged[-1][0], max(merged[-1][1], e))
174                 else:
175                     merged.append((s, e))
176             return merged
177 
178         def _get_video_duration(self, video_path: str) -> float:
179             """Probes the video file to get its total duration in seconds.
180             Returns 0.0 if it cannot be determined."""
181             try:
182                 timeout = ffprobe_timeout_sec()
```

```
                result = subprocess.run(
                    [
                        ffprobe_bin(),
                        "-v",
                        "error",
                        "-show_entries",
                        "format=duration",
                        "-of",
                        "default=noprint_wrappers=1:nokey=1",
                        video_path,
                    ],
                    capture_output=True,
                    text=True,
                    check=True,
                    timeout=timeout,
                )
                return float(result.stdout.strip())
            except subprocess.TimeoutExpired:
                logger.warning(
                    "Could not determine video duration via ffprobe: timed out after %s",
                    timeout_label(ffprobe_timeout_sec()),
                )
                return 0.0
            except Exception as e:
                logger.warning(f"Could not determine video duration via ffprobe: {e}")
                return 0.0

        def _watermark_filter(self, tmp_dir: str) -> Optional[str]:
            """Build the ffmpeg -vf filtergraph that burns the configured watermark (text
            and/or logo PNG) into a clip during the per-clip re-encode. Returns None when
            watermarking is disabled or has nothing to draw. The concat stage stays -c copy
            because the mark is already baked into every clip identically."""
            wm = self.watermark
            if not wm or not wm.get("enabled"):
                return None

            text = (wm.get("text") or "").strip()
            logo_path = (wm.get("logo_path") or "").strip()
            if logo_path and not os.path.exists(logo_path):
                logger.warning(
                    f"[watermark] logo not found: {logo_path} ? skipping logo overlay."
                )
                logo_path = ""
            if not text and not logo_path:
                return None

            margin = int(wm.get("margin", 24))
            opacity = float(wm.get("opacity", 0.85))
            position = str(wm.get("position", "bottom-right"))
            # drawtext uses lowercase w/h (main) + tw/th (text); overlay uses W/H (main) +
w/h (logo).
            dt_xy = {
                "bottom-right": f"x=w-tw-{margin}:y=h-th-{margin}",
                "bottom-left": f"x={margin}:y=h-th-{margin}",
                "top-right": f"x=w-tw-{margin}:y={margin}",
                "top-left": f"x={margin}:y={margin}",
            }.get(position, f"x=w-tw-{margin}:y=h-th-{margin}")
            ov_xy = {
                "bottom-right": f"W-w-{margin}:H-h-{margin}",
                "bottom-left": f"{margin}:H-h-{margin}",
                "top-right": f"W-w-{margin}:{margin}",
                "top-left": f"{margin}:{margin}",
            }.get(position, f"W-w-{margin}:H-h-{margin}")

            drawtext = ""
```

```
247             if text:
248                 ratio = float(wm.get("font_size_ratio") or 0.035)
249                 divisor = max(1, round(1.0 / ratio))
250                 # textfile= sidesteps the notorious drawtext text= escaping for arbitrary
strings.
251                 tf_path = os.path.join(tmp_dir, "watermark.txt")
252                 with open(tf_path, "w", encoding="utf-8") as fh:
253                     fh.write(text)
254                 # Font precedence: explicit font_path ? bundled Apache-2.0 default (renders
255                 # without fontconfig) ? fontconfig family (last-resort, environment-
dependent).
256                 font_path = (wm.get("font_path") or "").strip()
257                 if not font_path and _BUNDLED_FONT.exists():
258                     font_path = str(_BUNDLED_FONT)
259                 if font_path:
260                     font_opt = f"fontfile='{self._ff_quote(font_path)}'"
261                 else:
262                     font_opt = f"font='{wm.get('font', 'Sans')}'"
263                 box = ""
264                 if wm.get("box", True):
265                     box = f":box=1:boxcolor=black@{min(opacity, 0.5):.2f}:boxborderw=10"
266                 drawtext = (
267                     f"drawtext=textfile='{self._ff_quote(tf_path)}':{font_opt}"
268                     f":fontcolor=white@{opacity:.2f}:fontsize=h/{divisor}:{dt_xy}{box}"
269                 )
270
271             if logo_path and drawtext:
272                 return
f"movie='{self._ff_quote(logo_path)}'[wm];[in][wm]overlay={ov_xy},{drawtext}"
273             if logo_path:
274                 return f"movie='{self._ff_quote(logo_path)}'[wm];[in][wm]overlay={ov_xy}"
275             return drawtext
276
277         def _select_encoder(self) -> Tuple[List[str], str]:
278             """Resolve the ``-c:v ?`` args used for EVERY per-clip trim re-encode this run,
plus the
279             chosen encoder name. Chosen ONCE (not per clip) so all clips share one codec and
the final
280             concat can stay ``-c copy``. NVENC (hardware) is the L4 win on the 4K re-encode;
if the
281             requested NVENC encoder can't run in this ffmpeg build (CI / non-GPU box), fall
back to
282             libx264 so a misconfig never nukes the reel."""
283             enc = self.encoder
284             if enc in ("h264_nvenc", "hevc_nvenc") and not _probe_encoder_available(enc):
285                 logger.warning(
286                     "[compile] encoder %r is not runnable in this ffmpeg build ? falling
back to "
287                     "libx264 (CPU). On the L4 worker this should be available.",
288                     enc,
289                 )
290                 enc = "libx264"
291             if enc == "libx264":
292                 # Unchanged historical CPU settings (byte-compatible with the pre-#521
stitch).
293                 return ["-c:v", "libx264", "-preset", "superfast", "-crf", "22"], enc
294             # NVENC (H.264/HEVC): hardware encode. -cq is the CRF-equivalent constant-
quality knob; p4 is
295             # a balanced speed/quality preset. yuv420p keeps the output broadly playable.
296             return (
297                 ["-c:v", enc, "-preset", "p4", "-cq", "23", "-pix_fmt", "yuv420p"],
298                 enc,
299             )
300
301         def _trim_one_segment(
```

```
            self,
            idx: int,
            raw_start: Union[int, float, str],
            raw_end: Union[int, float, str],
            segments: List[Tuple[float, float]],
            video_duration: float,
            tmp_dir: str,
            raw_video_path: str,
            watermark_filter: Optional[str],
            video_codec_args: Optional[List[str]] = None,
        ) -> Tuple[Optional[str], Optional[Tuple[int, float, float, str]]]:
            """Trim a single segment into its own clip file (extracted verbatim from the
            per-segment body of compile_highlights). Returns (clip_path, skip_record): on
            success the clip path and None; on a skip (invalid/out-of-range/ffmpeg failure)
            None and the (idx, start, end, reason) record to record in skipped_segments."""
            # Normalize timestamps to float seconds ? handles float, int,
            # plain numeric strings, and colon-separated formats like "MM:SS"
            start = self._parse_time(raw_start)
            end = self._parse_time(raw_end)
            segment_label = (
                f"Segment #{idx + 1}/{len(segments)} [{start:.2f}s - {end:.2f}s]"
            )

            # Validate the segment times
            if start < 0:
                logger.warning(
                    f"{segment_label}: start_time is negative ({start:.2f}s). Clamping to
0."
                )
                start = 0.0

            if start >= end:
                logger.error(
                    f"{segment_label}: start_time ({start:.2f}s) >= end_time ({end:.2f}s).
Skipping invalid segment."
                )
                return None, (idx, start, end, "start >= end")

            # Check if segment extends beyond video duration
            if video_duration > 0:
                if start >= video_duration:
                    logger.error(
                        f"{segment_label}: start_time ({start:.2f}s) is beyond the video
duration ({video_duration:.2f}s). "
                        f"This segment cannot be extracted ? the video ran out. Skipping."
                    )
                    return None, (idx, start, end, "start beyond video end")

                if end > video_duration:
                    original_end = end
                    end = video_duration
                    logger.warning(
                        f"{segment_label}: end_time ({original_end:.2f}s) exceeds video
duration ({video_duration:.2f}s). "
                        f"Clamping end_time to {video_duration:.2f}s. The segment may be
truncated."
                    )

            clip_path = os.path.join(tmp_dir, f"clip_{idx}.mp4")

            # Precise sub-second trim using fast re-encoding to preserve stream headers
            padded_start = max(0.0, start - self.begin_padding)
            padded_end = end + self.end_padding

            # Clamp padded_end to video duration if known
```

```
            if video_duration > 0 and padded_end > video_duration:
                padded_end = video_duration
                logger.info(
                    f"{segment_label}: Padded end ({end + self.end_padding:.2f}s) exceeds
video duration. "
                    f"Clamping to {video_duration:.2f}s (end padding partially applied)."
                )

            trim_duration = padded_end - padded_start
            logger.info(
                f"Trimming {segment_label} -> [{padded_start:.3f}s - {padded_end:.3f}s]
(duration: {trim_duration:.2f}s)"
            )

            # Default to libx264 (the historical settings) when a caller invokes
_trim_one_segment
            # directly without resolving an encoder ? keeps this method's old behaviour
intact.
            codec_args = (
                video_codec_args
                if video_codec_args is not None
                else ["-c:v", "libx264", "-preset", "superfast", "-crf", "22"]
            )
            base_cmd = [
                ffmpeg_bin(),
                "-y",
                "-ss",
                str(round(padded_start, 3)),
                "-to",
                str(round(padded_end, 3)),
                "-i",
                raw_video_path,
                *codec_args,
                "-c:a",
                "aac",
                "-b:a",
                "128k",
            ]
            vf_args = ["-vf", watermark_filter] if watermark_filter else []

            # Try with the watermark first; if that encode fails (e.g. a bad font/logo
            # path), retry once WITHOUT it so a branding misconfig can never nuke the
            # whole reel. Clips stay codec-identical either way, so concat -c copy holds.
            attempts = [vf_args, []] if vf_args else [[]]
            produced = False
            last_exc: Optional[subprocess.CalledProcessError] = None
            cmd = base_cmd + [clip_path]
            timeout = ffmpeg_timeout_sec()
            for attempt_vf in attempts:
                cmd = base_cmd + attempt_vf + [clip_path]
                try:
                    subprocess.run(
                        cmd, capture_output=True, check=True, timeout=timeout
                    )
                except subprocess.TimeoutExpired:
                    reason = f"ffmpeg timeout after {timeout_label(timeout)}"
                    logger.error(f"{segment_label}: FFmpeg trim timed out.")
                    print(f"  Command: {' '.join(cmd)}")
                    print(f"  {reason}")
                    return None, (idx, start, end, reason)
                except subprocess.CalledProcessError as e:
                    last_exc = e
                    if attempt_vf and vf_args:
                        _err = (
                            e.stderr.decode(errors="replace").strip()[-300:]
```

```
                            if getattr(e, "stderr", None)
                            else ""
                        )
                        logger.warning(
                            f"{segment_label}: watermark encode failed; retrying without it.
"
                            f"ffmpeg: {_err or '(no stderr captured)'}"
                        )
                        continue
                    break
                if os.path.exists(clip_path) and os.path.getsize(clip_path) > 0:
                    produced = True
                    if not attempt_vf and vf_args:
                        logger.warning(
                            f"{segment_label}: encoded WITHOUT the watermark (the watermark
filter failed)."
                        )
                    break
                last_exc = None  # ffmpeg succeeded but produced nothing
                break

            if produced:
                return clip_path, None
            elif last_exc is not None:
                stderr_msg = last_exc.stderr.decode(errors="replace").strip()
                logger.error(f"{segment_label}: FFmpeg trim failed.")
                print(f"  Command: {' '.join(cmd)}")
                print(
                    f"  FFmpeg stderr: {stderr_msg[-500:]}"
                )  # Last 500 chars to avoid flooding
                return None, (idx, start, end, f"ffmpeg error: {stderr_msg[-200:]}")
            else:
                logger.error(
                    f"{segment_label}: FFmpeg exited successfully but output clip is missing
or empty. Skipping."
                )
                return None, (idx, start, end, "empty output clip")

        def compile_highlights(
            self,
            raw_video_path: str,
            segments: List[Tuple[float, float]],
            output_filename: str,
        ) -> str:
            """
            Extracts selected segments from a raw video and stitches them together
            into a seamless highlights file, applying end_padding to prevent abrupt endings.

            .. warning::
               This method uses a two-stage codec contract: per-clip re-encodes must use
               IDENTICAL encoder settings (codec, preset, CRF, pixel format) so the final
               concat can stream-copy (``-c copy``) without re-encoding. Changing the
               clip encode settings without updating the concat assumptions will produce
               broken output.
            """
            if not segments:
                raise ValueError("No highlight segments provided to compile.")

            # Never stitch the same footage twice: collapse overlapping/duplicate ranges
            # (e.g. accumulated re-run segments or chunk-overlap dupes) before trimming.
            original_count = len(segments)
            segments = self._merge_overlapping(segments)
            if len(segments) < original_count:
                logger.info(
                    "Merged %d overlapping/duplicate segment(s) before stitching: %d ->
```

```
%d.",
                    original_count - len(segments),
                    original_count,
                    len(segments),
                )
            if not segments:
                raise ValueError("No valid (start < end) highlight segments after merge.")

            if not os.path.exists(raw_video_path):
                raise FileNotFoundError(
                    f"[COMPILER ERROR] Source video file not found: {raw_video_path}"
                )

            # Defense-in-depth: never let the output name escape output_dir (the API also
            # validates this at the request boundary).
            output_path = os.path.join(self.output_dir, os.path.basename(output_filename))

            # Probe video duration so we can detect out-of-range segments
            video_duration = self._get_video_duration(raw_video_path)
            if video_duration > 0:
                logger.info(f"Source video duration: {video_duration:.2f}s")
            else:
                logger.warning(
                    "Video duration unknown. Cannot validate segment boundaries against
video length."
                )

            # We will create temporary clips and stitch them using FFmpeg concat demuxer
            temp_clips = []
            skipped_segments = []

            # Create a temp directory within output_dir
            with tempfile.TemporaryDirectory(dir=self.output_dir) as tmp_dir:
                logger.info(
                    f"Trimming {len(segments)} segments
(begin_padding={self.begin_padding}s, end_padding={self.end_padding}s)..."
                )
                # Build the (optional) watermark filtergraph once; it is applied during
every
                # per-clip re-encode so the mark is baked into each clip identically.
                watermark_filter = self._watermark_filter(tmp_dir)
                if watermark_filter:
                    logger.info("[watermark] applying branding overlay to each clip.")
                # Resolve the video encoder ONCE (probe NVENC / fall back) so every clip
shares one codec
                # and the final concat stays -c copy (#521).
                video_codec_args, chosen_encoder = self._select_encoder()
                if chosen_encoder != "libx264":
                    logger.info("[compile] encoding clips with %s (hardware).",
chosen_encoder)
                for idx, (raw_start, raw_end) in enumerate(segments):
                    clip_path, skip_record = self._trim_one_segment(
                        idx,
                        raw_start,
                        raw_end,
                        segments,
                        video_duration,
                        tmp_dir,
                        raw_video_path,
                        watermark_filter,
                        video_codec_args,
                    )
                    if clip_path is not None:
                        temp_clips.append(clip_path)
                    if skip_record is not None:
```

```
544                         skipped_segments.append(skip_record)
545
546                 # Report summary of skipped segments
547                 if skipped_segments:
548                     logger.info(
549                         f"\n[SUMMARY] {len(skipped_segments)} out of {len(segments)}
segments were SKIPPED:"
550                     )
551                     for seg_idx, seg_start, seg_end, reason in skipped_segments:
552                         logger.info(
553                             f"  - Segment #{seg_idx + 1} [{seg_start:.2f}s -
{seg_end:.2f}s]: {reason}"
554                         )
555                     logger.info("")
556
557                 if not temp_clips:
558                     raise RuntimeError(
559                         f"[COMPILER ERROR] Failed to generate ANY valid highlight clips. "
560                         f"All {len(segments)} segments were skipped. "
561                         f"Check the errors above for details. "
562                         f"Common causes: video file is corrupted, segments are out of range,
or ffmpeg is not installed."
563                     )
564
565                 logger.info(
566                     f"Successfully trimmed {len(temp_clips)}/{len(segments)} clips.
Preparing to stitch..."
567                 )
568
569                 # Create a list file for FFmpeg concat demuxer
570                 list_file_path = os.path.join(tmp_dir, "clips.txt")
571                 # encoding pinned: a non-ASCII clip path would crash on Windows (cp1252
default);
572                 # ffmpeg's concat demuxer reads the list as UTF-8 on every OS.
573                 with open(list_file_path, "w", encoding="utf-8") as f:
574                     for clip in temp_clips:
575                         # The concat demuxer resolves relative entries against the list
file's
576                         # directory. Use absolute paths so a temp-dir clip does not become
577                         # output/tmp.../output/tmp.../clip_0.mp4 on Windows.
578                         safe_path = Path(clip).resolve().as_posix().replace("'", "'\\''")
579                         f.write(f"file '{safe_path}'\n")
580
581                 logger.info("Stitching clips together...")
582                 # Stitch them together without re-encoding (since they were encoded with
identical settings)
583                 concat_cmd = [
584                     ffmpeg_bin(),
585                     "-y",
586                     "-f",
587                     "concat",
588                     "-safe",
589                     "0",
590                     "-i",
591                     list_file_path,
592                     "-c",
593                     "copy",
594                     output_path,
595                 ]
596
597                 try:
598                     timeout = ffmpeg_timeout_sec()
599                     subprocess.run(
600                         concat_cmd, capture_output=True, check=True, timeout=timeout
601                     )
```

```
602
603                     # Final verification
604                     if not os.path.exists(output_path) or os.path.getsize(output_path) == 0:
605                         raise RuntimeError(
606                             f"[COMPILER ERROR] FFmpeg concatenation appeared to succeed, "
607                             f"but output file is missing or empty: {output_path}"
608                         )
609
610                     output_size_mb = os.path.getsize(output_path) / (1024 * 1024)
611                     logger.info(
612                         f"Dynamic highlight reel saved: {output_path} ({output_size_mb:.1f}
MB)"
613                     )
614                     return output_path
615                 except subprocess.CalledProcessError as e:
616                     stderr_msg = e.stderr.decode(errors="replace").strip()
617                     raise RuntimeError(
618                         f"[COMPILER ERROR] FFmpeg concatenation failed.\n"
619                         f"  Command: {' '.join(concat_cmd)}\n"
620                         f"  FFmpeg stderr: {stderr_msg[-500:]}"
621                     ) from e
622                 except subprocess.TimeoutExpired as e:
623                     raise RuntimeError(
624                         f"[COMPILER ERROR] FFmpeg concatenation timed out after
{timeout_label(timeout)}.\n"
625                         f"  Command: {' '.join(concat_cmd)}"
626                     ) from e
627
```

22. user (2026-07-08T05:56:57.310Z)

```
1       # Phase ? machine placement + compile-on-L4 (#521)
2
3       > **Status:** ? **SHIPPED (code)** · **Owner:** Avi · **Created:** 2026-07-08 ·
**Issue:** [#521](https://github.com/Khelsutra/badminton-highlight-indexer/issues/521)
4       > **Relation:** extends [COMPUTE_DECOUPLED_SERVING](./README.md) (the `ComputeBackend`
dispatch seam) and **refines D16(b)** (NVENC-encode-deferred-behind-a-config-knob) now that the
2026-07-08 live CUJ supplies the cost data D16(b) said to revisit on.
5       > **Evidence:** `runlogs/DEPLOY_REPORT_cloud-serving_validation-latency_2026-07-07.md`
§9 (2026-07-08 live CUJ, rev `rally-control-plane-00019`).
6
7       ---
8
9       ## 0. TL;DR
10
11      The compile/stitch phase used to run **in-process on the CPU-only 2-vCPU control-
plane**, where the
12      2026-07-08 live CUJ measured **~12m55s** for an 11-clip / ~80 s reel from a 1.83 GiB 4K
source
13      (~45?90 s per ~7 s clip ? the per-clip branding watermark forces a full 4K re-encode
with **no
14      NVENC**). Validation (183 s) runs there too; both serialize on `_LOCAL_COMPUTE_LANE`, so
one run pins
15      the control-plane for ~16 min ? the API can't scale past a handful of concurrent users.
16
17      #521 makes **where each pipeline phase runs a config edit, not a code change**, and
moves the stitch
18      to the **L4 worker (NVENC + 8 vCPU)** the way detection already dispatches:
19
20      1. **`deployment.phase_placement`** ? a per-phase knob (`validation` / `segment` /
`compile`) with
21         values `control_plane | cloud_run_l4`. Unset ? **today's placement, byte-identical**.
22      2. **compile ? L4** ? a `kind='compile'` Cloud Run dispatch mirroring the detect path:
the
23         control-plane resolves the reel's segments, stages a compile-spec to the bucket,
```

```
dispatches
24         `python -m backend.worker compile` on the L4, bucket-polls a done-marker, and returns
the same
25         `{status, filepath, download_url}` shape the SPA already expects.
26      3. **NVENC on the L4** ? `deployment.cloud_stitch_encoder` (default `h264_nvenc`) is
forwarded onto
27         the worker's `stitching.encoder`; libx264 stays the everywhere-safe fallback (auto-
probed).
28
29      The cloud-serving preset opts compile onto the L4
(`phase_placement.compile=cloud_run_l4`).
30
31      ---
32
33      ## 1. The placement model
34
35      `resolve_phase_placement(cfg, phase)` (in `backend/config/placement.py`) is the **single
source of
36      truth** ? orchestration and the worker never re-derive the mapping. Precedence: an
explicit
37      `deployment.phase_placement.<phase>` wins; otherwise the **legacy default** reproduces
pre-#521
38      behaviour exactly:
39
40      | phase | knob unset (legacy default) | meaning |
41      |---|---|---|
42      | `validation` | `control_plane` | the authoritative gate (#512/#513) stays on the
control-plane |
43      | `segment` (detect) | `cloud_run_l4` **iff** `compute_target==cloud_run`, else
`control_plane` | today's DETECT dispatch |
44      | `compile` (stitch) | `control_plane` | the in-process ffmpeg stitch |
45
46      So a config with **no** `phase_placement` block behaves exactly as before this existed.
To relocate a
47      phase, set its knob:
48
49      ```jsonc
50      // Move the stitch to the L4 (what the cloud-serving preset does):
51      { "deployment": { "compute_target": "cloud_run",
52                        "phase_placement": { "compile": "cloud_run_l4" } } }
53
54      // Force detection back onto the control-plane on a cloud box (e.g. a debugging run):
55      { "deployment": { "compute_target": "cloud_run",
56                        "phase_placement": { "segment": "control_plane" } } }
57
58      // Move validation off the control-plane too (must pair with a cloud segment ? see §4):
59      { "deployment": { "compute_target": "cloud_run",
60                        "phase_placement": { "validation": "cloud_run_l4" } } }
61      ```
62
63      `control_plane` runs the phase in-process on the (CPU-only) control-plane service;
`cloud_run_l4`
64      dispatches it to the GPU/NVENC Cloud Run worker Job.
65
66      ---
67
68      ## 2. compile ? L4 dispatch (the flow)
69
70      `orchestration._execute_compile` consults `resolve_phase_placement(cfg, "compile")`.
When it resolves
71      to `cloud_run_l4` **and** cloud dispatch is available (`_cloud_available`), it calls
72      `_dispatch_compile_remote`; otherwise it stitches in-process via the shared
`_stitch_reel` core
73      (unchanged).
74
```

`_dispatch_compile_remote` (mirrors `_maybe_dispatch_remote`):

1. **Resolve** the reel's segments HERE ? the control-plane owns the DB (`_resolve_compile` ? `(video, kept, out_name)`), applying the `stitching.min_shot_count` floor as before.
2. **Stage a compile-spec** to `<output_root>/_compile/<token>/spec.json` ? the resolved `time_pairs`, the control-plane's resolved `stitching` config (watermark + padding, with `encoder` swapped to `cloud_stitch_encoder`), the source `video_uri`, `video_id`, `output_name`, `locale`. The worker therefore needs **no control-plane DB** ? everything rides the spec.
3. **Dispatch** a `compile` Cloud Run Job execution (`CompileJobSpec` ? `CloudRunCompute.run_compile`) running `python -m backend.worker compile --spec-uri ? --done-uri ? --set storage.output_root=?`, forwarding the bucket coordinates (the worker Job has no `DEPLOYMENT_PROFILE`).
4. **Bucket-poll** the done-marker at `<output_root>/_compile/<token>.done` ? the SAME `_await_marker` machinery `run_infer` uses (crash-fast on a terminal-without-marker execution, rescue-or-cancel on a poll timeout). Distinct `_compile/` prefix so a compile and a detection for one bucket never collide on a marker.
5. The worker (`cmd_compile`) reads the spec, stitches the reel via the **shared `_stitch_reel` core** (no forked pipeline), mirror-uploads it to `<output_root>/highlights/<video_id>/<name>`, and writes the marker (rc + `download_url` + `reel_uri`).
6. The control-plane returns `{status, filepath, download_url, video_id}` ? the reel is already in the bucket, so `/api/highlights` serves it via the existing signed-URL 307. On a worker rc?0 ? `status='failed'` with the worker's message (clean SPA verdict, never a crash).

Fallback: if compile is placed on the L4 but the box isn't actually cloud-wired (no output bucket / Cloud Run env), `_dispatch_compile_remote` logs and stitches in-process ? a slow reel beats no reel.

---

3. NVENC on the L4 ? and the D5/D16(b) parity note

The dominant term in the CUJ stitch was the **per-clip 4K re-encode** the branding watermark forces (~67 s/clip on 2 vCPU with libx264). The L4 has a hardware H.264/HEVC encoder, so #521 adds an encoder knob:

- `StitchingConfig.encoder` ? `libx264` (default, CPU, works everywhere incl. CI) | `h264_nvenc` | `hevc_nvenc`. `VideoCompiler` **probes** the requested encoder once per run (a 1-frame lavfi test encode) and **falls back to libx264** if this ffmpeg build can't run it, so a misconfig / non-GPU box never breaks. All clips share the one chosen encoder ? the concat stays `-c copy`.
- `deployment.cloud_stitch_encoder` (default `h264_nvenc`) ? what the compile-dispatch forwards onto the worker's `stitching.encoder` for the L4 run. Set it to `libx264` to run the L4 stitch on CPU.
- The control-plane's **own** local stitch always uses `stitching.encoder` (libx264 by default) ? the CP has no GPU.

Parity note (honest): [README.md](./README.md) **D5/D16(b)** kept stitch **encode = libx264** to

```
118     protect a byte-exact stitch parity guarantee, and explicitly **deferred NVENC encode
behind a config
119     knob, "revisit only if M7 cost data justifies relaxing parity."** The 2026-07-08 CUJ
(~13 min stitch,
120     a hard scaling wall past ~5 concurrent users) **is** that cost data. #521 lands exactly
the deferred
121     shape ? NVENC **behind a config knob**, libx264 the default, config-reversible per
phase. Enabling
122     NVENC relaxes byte-identity **at the reel ENCODE layer only** (a cosmetic, shareable
artifact); the
123     **WINDOW/rally-level parity** (the analytical output) is untouched ? detection and
windowing are
124     unchanged. If bit-exact reels are ever required, set `cloud_stitch_encoder=libx264`.
125
126     > Follow-up (tracked, not in #521's first cut): a **pre-rendered / overlay-filter
watermark** so the
127     > stitch isn't a full re-encode at all (the deeper win beyond swapping the encoder).
128
129     ---
130
131     ## 4. Validation placement + the "nobody validates" guard
132
133     `phase_placement.validation` moves the CPU-bound validator suite off the 2-vCPU control-
plane too,
134     via the existing `--skip-validation` lever (#513):
135
136     - **`control_plane`** (default): the control-plane runs the validators and is the
authoritative gate;
137       the worker **skips** its redundant re-validation (`skip_validation=True`) ? today's
#512/#513
138       behaviour, byte-identical.
139     - **`cloud_run_l4`**: when detection **actually** dispatches to the L4, the control-
plane **delegates**
140       ? it does NOT run the validator suite (`_execute_processing` skips it), and the
dispatch carries
141       **no** `--skip-validation`, so the **worker** validates and rejects a bad clip via a
non-zero rc
142       (surfaced as a failed job).
143
144     **Footgun guard:** a clip can only be validated on the worker if the worker also
**detects** it there.
145     Two protections:
146
147     1. **Config-load guard** (`DeploymentConfig` model-validator): `validation=cloud_run_l4`
requires the
148        effective segment placement to also be `cloud_run_l4` (explicit, or via
`compute_target=cloud_run`)
149        ? else config load fails with an actionable error.
150     2. **Runtime guard** (`_will_detect_on_cloud`): if a per-request `compute='local'`
forces detection
151        in-process (no worker), the control-plane **still validates** ? never leave a clip
unvalidated
152        because nobody ran the gate.
153
154     ---
155
156     ## 5. Files & tests
157
158     | Area | Code | Tests |
159     |---|---|---|
160     | Placement resolver + config | `backend/config/placement.py`,
`backend/config/models.py` (`PhasePlacementConfig`, `DeploymentConfig.phase_placement` /
`cloud_stitch_encoder`, guard) | `tests/test_phase_placement.py`,
`tests/test_config_deployment.py` |
161     | Preset | `backend/config/presets.py` (`cloud-serving` ?
```

`phase_placement.compile=cloud_run_l4`) | `tests/test_phase_placement.py` |
162 | compile dispatch | `backend/orchestration.py` (`_execute_compile`,
`_dispatch_compile_remote`, `_stitch_reel`, `_will_detect_on_cloud`) |
`tests/test_compile_dispatch.py` |
163 | compute backend | `backend/compute/base.py` (`CompileJobSpec`, `run_compile`),
`backend/compute/cloud_run.py` (`run_compile`, `_await_marker`) |
`tests/test_compute_backend.py` |
164 | worker | `backend/worker/__main__.py` (`compile` subcommand, `cmd_compile`) |
`tests/test_worker.py` |
165 | NVENC encoder | `backend/pipeline/stitcher/compiler.py` (`encoder`, `_select_encoder`,
`_probe_encoder_available`) | `tests/test_compiler.py` |
166 | validation delegation | `backend/orchestration.py` (`_maybe_dispatch_remote` skip
flag, `_execute_processing`) | `tests/test_api_cloud_dispatch.py` |
167
168   All GPU-free + cloud-free: a fake `CloudRunJobClient` simulates the worker (reads the
staged spec,
169   writes the reel + done-marker into a local "bucket"), mirroring
`tests/test_api_cloud_dispatch.py`.
170

23. assistant (2026-07-08T05:57:56.011Z)

I now have full context. Before writing any code, I need to surface a material discrepancy between the task's premise and the actual repo state, because it determines the base branch ? and that's a fork I shouldn't pick silently given the repo's "branch off master / NO PR stacking" rule.

What I found:
- **#521 is *not* merged to master.** It lives on `origin/feat/521-compile-on-l4` (commit 8866ffe), which has *diverged* from master: it was cut from `699d85b`, and master's `bf97700` (#522) landed after. Each branch has one commit the other lacks.
- **There is no PR #525 / no NVENC / no `_select_encoder` anywhere in git history** except that one unmerged branch. The `encoder` knob, `_select_encoder`, `_probe_encoder_available`, and `docs/.../PHASE_PLACEMENT.md` (whose §3 I'm told to update, and which already declares this exact "pre-rendered / overlay-filter watermark" follow-up) **exist only on `feat/521-compile-on-l4`** ? none are on master.
- So the task's literal instruction ("branch off `origin/master`") is incoherent as stated: master has neither the code this builds on nor the doc §3 to update. The 2026-07-08 CUJ §9 the task cites *is* on master (added by #522), and it confirms the target: stitch ran **~12m55s** (11 clips × ~67s/clip, watermark forcing a full 4K re-encode; concat only ~2s).

The work only makes sense on top of #521. That means stacking on an unmerged branch ? which your CLAUDE.md explicitly discourages ("NO PR stacking") ? so I want your call rather than picking it myself.

24. user (2026-07-08T06:00:13.933Z)

Your questions have been answered: "This task ("Part 3 of #521") builds on #521's encoder knob and updates PHASE_PLACEMENT.md §3 ? both of which exist ONLY on the unmerged, diverged branch origin/feat/521-compile-on-l4, not on origin/master. How should I base the work?"="The PR is merged so sync and build". You can now continue with these answers in mind.

25. assistant (2026-07-08T06:00:23.254Z)

Understood ? #521 merged on the remote after my initial fetch (local was lagging, exactly the parallel-slate scenario your CLAUDE.md warns about). Let me re-sync and confirm master now carries #521.

26. user (2026-07-08T06:00:31.807Z)

From https://github.com/Khelsutra/badminton-highlight-indexer
   bf97700..dac6736 master   -> origin/master
===origin/master now===
dac6736 perf(serving): run compile/stitch on the L4 worker; config-driven phase?machine

```
placement (#521) (#525)
bf97700 docs(runlogs): record live SPA CUJ closing the 2 owner-gated validation-latency cells
(§9) (#522)
699d85b feat(validation): cache validation verdict by video_id to skip re-validation on reupload
(#518)

===does origin/master contain #521 (8866ffe)?===
NO - #521 still not in master

===_select_encoder + PHASE_PLACEMENT on origin/master?===
PHASE_PLACEMENT.md EXISTS on master
_select_encoder count on master: 3
```

27. assistant (2026-07-08T06:04:14.362Z)

The user was right ? origin/master now advanced `bf97700 ? dac6736` = "perf(serving): run compile/stitch on the L4 worker ? (#521) (#525)". Master now has `PHASE_PLACEMENT.md` and `_select_encoder` (count 3). My earlier fetch was simply stale. The dilemma is resolved: I branch off `origin/master`, which now literally contains #521.

Let me mark this chapter, set up tracking, and create the branch safely (I have unrelated dirty files ? the #523 handoff WIP and untracked `score_*.py` ? that I must not clobber or stage).

28. user (2026-07-08T06:04:15.808Z)

Chapter marked: "Ramp-up & base reconciliation". Continue your current work.

29. user (2026-07-08T06:04:20.135Z)

```
===does SESSION_HANDOFF.md differ HEAD vs origin/master?===
SAME (no conflict on checkout)

===ffmpeg + encoders===
ffmpeg version 8.1.1-full_build-www.gyan.dev Copyright (c) 2000-2026 the FFmpeg developers
 V....D av1_nvenc            NVIDIA NVENC av1 encoder (codec av1)
 V....D libx264              libx264 H.264 / AVC / MPEG-4 AVC / MPEG-4 part 10 (codec h264)
 V....D libx264rgb           libx264 H.264 / AVC / MPEG-4 AVC / MPEG-4 part 10 RGB (codec h264)
 V....D h264_nvenc           NVIDIA NVENC H.264 encoder (codec h264)
 V....D libx265              libx265 H.265 / HEVC (codec hevc)
 V....D hevc_nvenc           NVIDIA NVENC hevc encoder (codec hevc)
```

30. assistant (2026-07-08T06:04:29.965Z)

This box has `h264_nvenc`/`hevc_nvenc` *compiled in* (I'll probe runnability during benchmarking). Handoff won't conflict on checkout. Now I'll create the branch off `origin/master` per the branching-safety protocol, then verify HEAD is clean.

31. user (2026-07-08T06:04:33.738Z)

```
Exit code 1
error: Your local changes to the following files would be overwritten by checkout:
        backend/config/models.py
        backend/orchestration.py
        backend/pipeline/stitcher/compiler.py
        tests/test_compiler.py
Please commit your changes or stash them before you switch branches.
Aborting
```

32. assistant (2026-07-08T06:04:52.140Z)

Unexpected ? checkout is blocked by uncommitted local changes to `backend/config/models.py`, `orchestration.py`, `compiler.py`, `tests/test_compiler.py`. These weren't in my session-start `git status` (which showed only `SESSION_HANDOFF.md` + untracked `score_*.py`). This is the concurrent-shared-checkout hazard my CLAUDE.md warns about. Let me inspect carefully before

touching anything ? I must not clobber a sibling session's WIP.

33. user (2026-07-08T06:04:54.893Z)

```
===FULL git status===
On branch codex/issue-523-sd-reel-plan
Your branch is behind 'origin/master' by 1 commit, and can be fast-forwarded.
  (use "git pull" to update your local branch)

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   README.md
        modified:   SESSION_HANDOFF.md
        modified:   backend/config/models.py
        modified:   backend/infrastructure/database_jobs.py
        modified:   backend/main.py
        modified:   backend/orchestration.py
        modified:   backend/pipeline/stitcher/compiler.py
        modified:   config.json
        modified:   tests/test_api_endpoints.py
        modified:   tests/test_async_jobs.py
        modified:   tests/test_compiler.py
        modified:   tests/test_watermark.py

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        score_golden_gate.py
        score_local.py

no changes added to commit (use "git add" and/or "git commit -a")

===stat of the 4 flagged files vs HEAD (bf97700)===
warning: in the working copy of 'backend/config/models.py', LF will be replaced by CRLF the next
time Git touches it
warning: in the working copy of 'backend/orchestration.py', LF will be replaced by CRLF the next
time Git touches it
warning: in the working copy of 'backend/pipeline/stitcher/compiler.py', LF will be replaced by
CRLF the next time Git touches it
warning: in the working copy of 'tests/test_compiler.py', LF will be replaced by CRLF the next
time Git touches it
 backend/config/models.py              | 24 ++++++++++
 backend/orchestration.py              | 30 ++++++++++++
 backend/pipeline/stitcher/compiler.py | 86 +++++++++++++++++++++++++++++++-----
 tests/test_compiler.py                | 64 +++++++++++++++++++++++++++
 4 files changed, 193 insertions(+), 11 deletions(-)
```

34. assistant (2026-07-08T06:05:52.584Z)

This is the concurrent-shared-checkout hazard my CLAUDE.md explicitly warns about. A **sibling session is actively implementing #523 (the SD-reel work)** in the shared tree right now ? the modified set (`main.py`, `orchestration.py`, `database_jobs.py`, `compiler.py`, `tests/test_compiler.py`, `test_watermark.py`, `config.json`, `README.md`) matches the #523 ramp-up kit from the handoff exactly. That WIP includes `compiler.py` ? my target file. I must **not** clobber it, so I won't force the checkout.

The clean, non-destructive way to work concurrently is a **separate git worktree** off `origin/master`. Let me verify the branch doesn't already exist and set one up in isolation.

35. user (2026-07-08T06:05:56.882Z)

```
===does branch exist?===
does not exist (good)
```

```
===existing worktrees===
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer
bf97700 [codex/issue-523-sd-reel-plan]
C:/kg-wt521
8866ffe [feat/521-compile-on-l4]
C:/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-
indexer/6e9d0958-a2ec-4a8e-bb2d-6cce9833c527/scratchpad/cfg-wt        20b2d5a [claude/wasb-
config-validation]
C:/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-
indexer/6e9d0958-a2ec-4a8e-bb2d-6cce9833c527/scratchpad/compile-wt    b09a1d4 [claude/compile-
reliability]
C:/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-
indexer/6e9d0958-a2ec-4a8e-bb2d-6cce9833c527/scratchpad/pr458-pre-wt  355dad6 (detached HEAD)
C:/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-
indexer/6e9d0958-a2ec-4a8e-bb2d-6cce9833c527/scratchpad/pr458-wt      5de1ef2 (detached HEAD)
C:/Users/avidu/AppData/Local/Temp/claude/wt524-969
bf97700 [feat/524-l4-primary]

===create isolated worktree off origin/master===
Preparing worktree (new branch 'perf/521-cheap-watermark')
branch 'perf/521-cheap-watermark' set up to track 'origin/master'.
HEAD is now at dac6736 perf(serving): run compile/stitch on the L4 worker; config-driven
phase?machine placement (#521) (#525)
```

36. assistant (2026-07-08T06:06:18.183Z)

Isolated worktree created at `C:/Users/avidu/Projects/khelsutra-guru/bhi-wt-cheap-watermark` on `perf/521-cheap-watermark` at `dac6736` (origin/master, includes #521). The worktree list confirms this is a heavily parallel environment (kg-wt521, wt524, compile-reliability, etc.) ? isolation was the right call. All my edits will happen in this worktree, leaving the sibling's #523 WIP untouched.

Let me confirm the worktree is clean and read the config surface + test patterns I'll build on.

37. user (2026-07-08T06:06:21.786Z)

```
===worktree status (should be clean)===
(clean if empty above)

===confirm #521 content present===
compiler _select_encoder refs: 5
===StitchingConfig in models.py===
37:            "recordings, and other encoders may use different GOP sizes ? tune "
332:class WatermarkConfig(BaseModel):
354:class StitchingConfig(BaseModel):
367:    # Video encoder for the per-clip trim re-encode (the branding watermark forces a re-
encode; the
369:    # ``hevc_nvenc`` are the L4 HARDWARE encoders ? dramatically faster on the 4K re-encode
that
370:    # dominated the 2026-07-08 CUJ stitch (#521). The compiler probes the requested encoder
ONCE and
372:    # cloud compile-dispatch forwards ``deployment.cloud_stitch_encoder`` onto this for the
L4 run;
374:    encoder: Literal["libx264", "h264_nvenc", "hevc_nvenc"] = "libx264"
477:class DeploymentConfig(BaseModel):
512:    # Video encoder the cloud compile-dispatch forwards onto ``stitching.encoder`` for the
L4 stitch
513:    # (#521). ``h264_nvenc`` uses the L4's hardware encoder for the 4K re-encode the
watermark forces;
515:    # the control-plane's own local stitch always uses ``stitching.encoder`` (libx264 by
default).
516:    cloud_stitch_encoder: Literal["libx264", "h264_nvenc", "hevc_nvenc"] = "h264_nvenc"
```

38. assistant (2026-07-08T06:06:25.935Z)

Worktree is clean with #521 content. Let me read the config classes and the test file to understand the surface I'll extend.

39. user (2026-07-08T06:06:27.042Z)

```
332     class WatermarkConfig(BaseModel):
333         """Branding overlay burned into each highlight clip during the per-clip re-encode
334         (backend/pipeline/stitcher/compiler.py). **On by default** ? set `enabled: false`
335         (under `stitching.watermark` in config.json) to turn it off. The text is fully
336         configurable. The concat stage stays stream-copy (-c copy)."""
337
338         enabled: bool = True
339         text: str = "Generated by Khelsutra.guru"
340         logo_path: str = ""  # optional transparent PNG overlay; "" = no logo
341         font_path: str = ""  # optional .ttf; "" = use the fontconfig `font` family below
342         font: str = "Sans"  # fontconfig family used when font_path is empty
343         font_size_ratio: float = Field(
344             default=0.035, gt=0.0, le=0.5
345         )  # text height as a fraction of frame height
346         opacity: float = Field(default=0.85, ge=0.0, le=1.0)
347         box: bool = True  # faint backing box behind the text for legibility
348         margin: int = Field(default=24, ge=0)  # px inset from the chosen corner
349         position: Literal["bottom-right", "bottom-left", "top-right", "top-left"] = (
350             "bottom-right"
351         )
352
353
354     class StitchingConfig(BaseModel):
355         begin_padding: float = 0.5
356         end_padding: float = 0.5
357         use_cache: bool = False
358         min_shot_count: int = 1
359         # Long-rally reel filter (seconds; 0 = OFF). Drop detected rally pairs whose
duration
360         # (end ? start) is below this from the highlight reel, BEFORE compiling. A cleaner
361         # reel-selection knob than `min_shot_count`: shot_count is unlabeled / "Unknown" on
the
362         # fully-local no-AI path, whereas duration is derived from the rally's own
start/end. The
363         # local detector over-fires and its false positives are mostly SHORT (motion blips,
364         # background-court fragments), so this keeps the reel to the eye-catching long
rallies.
365         # OFF by default ? the detector output is unchanged, only which rallies reach the
reel.
366         min_rally_duration: float = Field(default=0.0, ge=0.0)
367         # Video encoder for the per-clip trim re-encode (the branding watermark forces a re-
encode; the
368         # concat stays -c copy). Default ``libx264`` (CPU, works everywhere incl. CI).
``h264_nvenc`` /
369         # ``hevc_nvenc`` are the L4 HARDWARE encoders ? dramatically faster on the 4K re-
encode that
370         # dominated the 2026-07-08 CUJ stitch (#521). The compiler probes the requested
encoder ONCE and
371         # falls back to libx264 if this ffmpeg build can't run it, so a non-GPU box never
breaks. The
372         # cloud compile-dispatch forwards ``deployment.cloud_stitch_encoder`` onto this for
the L4 run;
373         # the control-plane's own local stitch keeps libx264.
374         encoder: Literal["libx264", "h264_nvenc", "hevc_nvenc"] = "libx264"
375         watermark: WatermarkConfig = Field(default_factory=WatermarkConfig)
376
377
378     class LoggingConfig(BaseModel):
379         """Logging knobs for the run entrypoints (see backend/utils/logging_setup.py)."""
380
```

```
        # Third-party HTTP/RPC client loggers (httpx, httpcore, urllib3, google-genai,
        # google.auth, grpc) emit one INFO line per request ? dozens per Gemini run, which
        # buries our own [rally]/[compile] lines in run.log during manual QA. Default raises
        # them to WARNING; set true to restore full chatter for request-flow debugging.
        verbose_http: bool = False


    class SecurityConfig(BaseModel):
        # When set, /api/process video_path must resolve under this directory (hosted/tenant
        # mode). Default None preserves the single-user local 'any local file' workflow.
        allowed_video_root: Optional[str] = None

        # --- Chunked upload (B2, PR-2) ---
        # Landing zone for browser uploads (assembled from parts). Per-user subdirectories
        # arrive with PR-3 identity scoping. ? In hosted mode, set allowed_video_root to
        # contain upload_dir or /api/process will reject the uploaded paths.
        upload_dir: str = "output/uploads"
        # Whole-file cap (default 4 GB) and per-part cap. Parts stay under ~100 MB because
        # free-tier edge proxies cap request bodies there (HOSTING_PLAN §2).
        max_upload_mb: int = 4096
        max_part_mb: int = 95
```

40. user (2026-07-08T06:06:27.564Z)

```
      """VideoCompiler coverage (previously zero ? the user-facing highlight-reel output path,
      where the f-string SyntaxError lived undetected). ffmpeg/ffprobe are mocked."""

      import os
      import subprocess
      from unittest.mock import MagicMock, patch

      import pytest

      from backend.pipeline.stitcher.compiler import VideoCompiler


      def test_parse_time_formats():
          f = VideoCompiler._parse_time
          assert f(12.5) == 12.5
          assert f("12.5") == 12.5
          assert f("1:30") == 90.0
          assert f("1:00:00") == 3600.0
          assert f("garbage") == 0.0


      def test_merge_overlapping_collapses_duplicates_and_overlaps():
          m = VideoCompiler._merge_overlapping
          # Exact duplicates + near-duplicates (the "every rally twice" re-run bug) collapse
to one each.
          assert m([(28.2, 34.5), (28.5, 34.5), (40.5, 45.5), (40.5, 45.5)]) == [
              (28.2, 34.5),
              (40.5, 45.5),
          ]
          # Unsorted input is sorted; an overlapping range extends the previous.
          assert m([(100.0, 110.0), (0.0, 5.0), (104.0, 120.0)]) == [
              (0.0, 5.0),
              (100.0, 120.0),
          ]
          # Genuinely distinct (gapped) rallies are preserved, not merged.
          assert m([(0.0, 5.0), (6.0, 9.0)]) == [(0.0, 5.0), (6.0, 9.0)]
          # Degenerate rows (end <= start) are dropped; string timestamps are accepted.
          assert m([("0:10", "0:20"), (30.0, 30.0), (31.0, 29.0)]) == [(10.0, 20.0)]


      def test_compile_dedups_before_stitching(tmp_path):
```

```
        """A segment list with every rally twice must yield ONE clip per unique rally."""
        comp = VideoCompiler(str(tmp_path / "out"))
        raw = tmp_path / "raw.mp4"
        raw.write_bytes(b"x")
        trims = []

        def fake_run(cmd, *a, **k):
            if cmd[0] == "ffprobe":
                return MagicMock(stdout="100.0\n", returncode=0)
            out = cmd[-1]
            if isinstance(out, str) and out.endswith(".mp4"):
                open(out, "wb").write(b"clip")
                # count per-clip trims (the concat list file input is not an .mp4 output)
                if "-i" in cmd and cmd[cmd.index("-i") + 1] == str(raw):
                    trims.append(cmd)
            return MagicMock(returncode=0, stdout=b"", stderr=b"")

        dup_segments = [(0.0, 5.0), (0.2, 5.0), (10.0, 15.0), (10.0, 15.0), (20.0, 25.0)]
        with patch.object(subprocess, "run", side_effect=fake_run):
            comp.compile_highlights(str(raw), dup_segments, "reel.mp4")
        # 5 input segments collapse to 3 unique rallies -> 3 per-clip trims, not 5.
        assert len(trims) == 3, f"expected 3 deduped trims, got {len(trims)}"


    def test_compile_basenames_output_and_stitches(tmp_path):
        comp = VideoCompiler(str(tmp_path / "out"))
        raw = tmp_path / "raw.mp4"
        raw.write_bytes(b"x")

        calls = []

        def fake_run(cmd, *a, **k):
            calls.append(cmd)
            # Create whatever output file ffmpeg/ffprobe was told to write so existence
checks pass.
            if cmd[0] == "ffprobe":
                return MagicMock(stdout="100.0\n", returncode=0)
            out = cmd[-1]
            if isinstance(out, str) and out.endswith((".mp4",)):
                open(out, "wb").write(b"clip")
            return MagicMock(returncode=0, stdout=b"", stderr=b"")

        with patch.object(subprocess, "run", side_effect=fake_run):
            # Traversal in the name must be stripped (defense-in-depth basename).
            out_path = comp.compile_highlights(
                str(raw), [(0.0, 5.0), (10.0, 15.0)], "../reel.mp4"
            )

        assert os.path.basename(out_path) == "reel.mp4"
        assert os.path.dirname(out_path).endswith("out")
        # Per-clip trims + one concat = at least 3 ffmpeg invocations (plus an ffprobe).
        assert sum(1 for c in calls if c[0] == "ffmpeg") >= 3


    def test_compile_concat_timeout_raises_clean_error(tmp_path):
        comp = VideoCompiler(str(tmp_path / "out"))
        raw = tmp_path / "raw.mp4"
        raw.write_bytes(b"x")

        def fake_run(cmd, *a, **k):
            if cmd[0] == "ffprobe":
                return MagicMock(stdout="100.0\n", returncode=0)
            if "concat" in cmd:
                raise subprocess.TimeoutExpired(cmd, k.get("timeout"))
            out = cmd[-1]
```

```
            if isinstance(out, str) and out.endswith(".mp4"):
                open(out, "wb").write(b"clip")
            return MagicMock(returncode=0, stdout=b"", stderr=b"")

        with patch.object(subprocess, "run", side_effect=fake_run):
            with pytest.raises(RuntimeError, match="timed out after 1800s"):
                comp.compile_highlights(str(raw), [(0.0, 5.0)], "reel.mp4")


    def test_concat_list_uses_absolute_clip_paths(tmp_path):
        """Regression for Windows concat failure:

        ffmpeg resolves relative paths inside clips.txt relative to clips.txt itself. If
        clips.txt lives in output/tmp123 and contains output/tmp123/clip_0.mp4, ffmpeg
        tries output/tmp123/output/tmp123/clip_0.mp4. The list must contain absolute
        clip paths.
        """
        comp = VideoCompiler(str(tmp_path / "out"))
        raw = tmp_path / "raw.mp4"
        raw.write_bytes(b"x")
        concat_list = ""

        def fake_run(cmd, *a, **k):
            nonlocal concat_list
            if cmd[0] == "ffprobe":
                return MagicMock(stdout="100.0\n", returncode=0)
            if "-f" in cmd and "concat" in cmd:
                list_path = cmd[cmd.index("-i") + 1]
                concat_list = open(list_path, encoding="utf-8").read()
            out = cmd[-1]
            if isinstance(out, str) and out.endswith(".mp4"):
                open(out, "wb").write(b"clip")
            return MagicMock(returncode=0, stdout=b"", stderr=b"")

        with patch.object(subprocess, "run", side_effect=fake_run):
            comp.compile_highlights(str(raw), [(0.0, 5.0)], "reel.mp4")

        line = concat_list.strip()
        assert line.startswith("file '")
        clip_path = line.removeprefix("file '").removesuffix("'")
        assert os.path.isabs(clip_path)
        assert "clip_0.mp4" in clip_path


    def test_compile_raises_on_no_segments(tmp_path):
        comp = VideoCompiler(str(tmp_path / "out"))
        raw = tmp_path / "raw.mp4"
        raw.write_bytes(b"x")
        with pytest.raises(ValueError):
            comp.compile_highlights(str(raw), [], "reel.mp4")


    def test_compile_raises_when_source_missing(tmp_path):
        comp = VideoCompiler(str(tmp_path / "out"))
        with pytest.raises(FileNotFoundError):
            comp.compile_highlights(str(tmp_path / "nope.mp4"), [(0.0, 5.0)], "reel.mp4")


    # ---------------------------------------------------------- encoder selection (#521
? NVENC)


    def test_select_encoder_default_is_libx264(tmp_path):
        """Default encoder = libx264 (CPU) with the historical settings ? no probe, works
everywhere."""
```

```
168         comp = VideoCompiler(str(tmp_path / "out"))
169         args, chosen = comp._select_encoder()
170         assert chosen == "libx264"
171         assert args == ["-c:v", "libx264", "-preset", "superfast", "-crf", "22"]
172
173
174     def test_select_encoder_nvenc_when_available(tmp_path, monkeypatch):
175         """h264_nvenc resolves to the hardware encoder args when the probe reports it
runnable (the L4)."""
176         from backend.pipeline.stitcher import compiler as comp_mod
177
178         monkeypatch.setattr(comp_mod, "_probe_encoder_available", lambda enc: True)
179         comp = VideoCompiler(str(tmp_path / "out"), encoder="h264_nvenc")
180         args, chosen = comp._select_encoder()
181         assert chosen == "h264_nvenc"
182         assert args[:2] == ["-c:v", "h264_nvenc"]
183         assert "-cq" in args  # NVENC constant-quality knob (not -crf)
184
185
186     def test_select_encoder_falls_back_to_libx264_when_nvenc_unavailable(tmp_path,
monkeypatch):
187         """On a box where NVENC can't actually run (CI / no GPU), the requested encoder
falls back to
188         libx264 so a misconfig never nukes the reel."""
189         from backend.pipeline.stitcher import compiler as comp_mod
190
191         monkeypatch.setattr(comp_mod, "_probe_encoder_available", lambda enc: False)
192         comp = VideoCompiler(str(tmp_path / "out"), encoder="h264_nvenc")
193         args, chosen = comp._select_encoder()
194         assert chosen == "libx264"
195         assert args == ["-c:v", "libx264", "-preset", "superfast", "-crf", "22"]
196
197
198     def test_compile_uses_selected_encoder_in_trim(tmp_path, monkeypatch):
199         """End-to-end (ffmpeg mocked): with an available NVENC encoder, every per-clip trim
command
200         carries -c:v h264_nvenc, and the concat stays -c copy."""
201         from backend.pipeline.stitcher import compiler as comp_mod
202
203         monkeypatch.setattr(comp_mod, "_probe_encoder_available", lambda enc: True)
204         comp = VideoCompiler(str(tmp_path / "out"), encoder="h264_nvenc")
205         raw = tmp_path / "raw.mp4"
206         raw.write_bytes(b"x")
207         trims = []
208
209         def fake_run(cmd, *a, **k):
210             if cmd[0] == "ffprobe":
211                 return MagicMock(stdout="100.0\n", returncode=0)
212             out = cmd[-1]
213             if isinstance(out, str) and out.endswith(".mp4"):
214                 open(out, "wb").write(b"clip")
215                 if "-i" in cmd and cmd[cmd.index("-i") + 1] == str(raw):
216                     trims.append(cmd)
217             return MagicMock(returncode=0, stdout=b"", stderr=b"")
218
219         with patch.object(subprocess, "run", side_effect=fake_run):
220             comp.compile_highlights(str(raw), [(0.0, 5.0), (10.0, 15.0)], "reel.mp4")
221         assert trims, "no trim commands captured"
222         for cmd in trims:
223             assert "h264_nvenc" in cmd  # hardware encode on every clip
224             assert "libx264" not in cmd
225
```

41. assistant (2026-07-08T06:07:36.294Z)

I now understand the config surface (`StitchingConfig.encoder`, `WatermarkConfig`, `DeploymentConfig.cloud_stitch_encoder`) and the test patterns. Before implementing, the task says **measure on a real 4K clip** ? so I'll build an empirical benchmark to settle Option A vs B rather than theorize. First, probe whether NVENC actually *runs* on this box (it's compiled in), since the L4's default cloud encoder is `h264_nvenc` ? that makes the benchmark far more representative.

42. user (2026-07-08T06:07:43.286Z)

```
===probe h264_nvenc runnable?===
h264_nvenc: NOT runnable (no GPU here)
hevc_nvenc: NOT runnable

===cpu count===
cpus: 16
===nvidia-smi?===
NVIDIA GeForce RTX 2070 Super with Max-Q Design
```

43. assistant (2026-07-08T06:07:56.985Z)

A GPU exists (RTX 2070 Super Max-Q) but NVENC failed to initialize. Let me see the actual error ? if it's a trivial probe issue I can use NVENC for a more L4-representative benchmark; if it's environmental, I'll benchmark on libx264 (the default anyway) and flag NVENC/L4 as owner-gated.

44. user (2026-07-08T06:08:00.167Z)

```
===h264_nvenc error (realistic probe)===
    encoder         : Lavf62.12.101
  Stream #0:0: Video: h264 (Main), yuv420p(tv, progressive), 1280x720 [SAR 1:1 DAR 16:9],
q=2-31, 2000 kb/s, 25 fps, 25 tbn
    Metadata:
      encoder         : Lavc62.28.101 h264_nvenc
    Side data:
      CPB properties: bitrate max/min/avg: 0/0/2000000 buffer size: 4000000 vbv_delay: N/A
[out#0/null @ 000001c452747b40] video:87KiB audio:0KiB subtitle:0KiB other streams:0KiB global
headers:0KiB muxing overhead: unknown
frame=    3 fps=0.0 q=22.0 Lsize=N/A time=00:00:00.00 bitrate=N/A speed=   0x elapsed=0:00:00.18
```

45. assistant (2026-07-08T06:08:21.350Z)

Interesting and important: `h264_nvenc` **works** at 1280x720 ? it only failed at 64x64. That means NVENC *is* usable on this box (RTX 2070 Super), and I can make the benchmark representative of the L4's actual `h264_nvenc` path. Let me confirm the exact #521 probe command's behavior vs a 4K encode.

46. user (2026-07-08T06:08:26.517Z)

```
===EXACT #521 probe (64x64)===
[out#0/null @ 0000020b1c847700] Nothing was written into output file, because at least one of
its streams received no packets.
frame=    0 fps=0.0 q=0.0 Lsize=       0KiB time=N/A bitrate=N/A speed=N/A elapsed=0:00:00.26
Conversion failed!
rc=0

===h264_nvenc at 4K (real use case)===
h264_nvenc 4K: OK
```

47. assistant (2026-07-08T06:09:29.465Z)

Confirmed two things:
1. **The exact #521 probe (`color=...:s=64x64`) fails** ? 64×64 is below NVENC's minimum encode size, so `_probe_encoder_available` *under-reports* NVENC on this box (and potentially others). That's a latent bug in #521's probe, separate from my task ? I'll flag it, not fix it here.
2. **h264_nvenc works at 4K** ? so I can benchmark the L4's actual production encoder, not just

libx264.

Let me kick off generation of a realistic 4K 10-bit HEVC source (mimicking the corpus decode cost) in the background, and verify the bundled font while it runs.

48. user (2026-07-08T06:09:32.119Z)

```
total 1536
drwxr-xr-x 1 avidu 197609      0 Jul  8 11:35 .
drwxr-xr-x 1 avidu 197609      0 Jul  8 11:35 ..
-rw-r--r-- 2 avidu 197609  11560 Jul  8 11:35 LICENSE-Apache-2.0.txt
-rw-r--r-- 2 avidu 197609   4476 Jul  8 11:35 LICENSE-OFL-1.1.txt
-rw-r--r-- 2 avidu 197609 647144 Jul  8 11:35 NotoSansDevanagari-Variable.ttf
-rw-r--r-- 2 avidu 197609 641372 Jul  8 11:35 NotoSansKannada-Variable.ttf
-rw-r--r-- 2 avidu 197609   2100 Jul  8 11:35 README.md
-rw-r--r-- 2 avidu 197609 251880 Jul  8 11:35 RobotoSlab-Regular.ttf
---font exists check---
FONT OK
```

49. user (2026-07-08T06:09:37.353Z)

Command running in background with ID: bxcfpizw4. Output is being written to: C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-khelsutra-guru-badminton-highlight-indexer\46eabfd2-0e71-4ff5-a68d-c76f474d4ff3\tasks\bxcfpizw4.output. You will be notified when it completes. To check interim output, use Read on that file path.

50. assistant (2026-07-08T06:10:38.921Z)

Font present; source generating in the background. Let me write the benchmark harness while it generates. It mirrors the compiler's exact ffmpeg args and measures all four strategies (Baseline / Option A / Option B / no-watermark floor) across both encoders (libx264 default + h264_nvenc L4), with a per-stage (trim vs concat) breakdown ? the crux of whether cost actually *moves* under Option B.

51. assistant (2026-07-08T06:11:21.422Z)

```
"""Watermark-stitch cost benchmark for #521 Part-3 (cheap watermark).

Compares four stitch strategies on a real 4K clip, across encoders:
  BASELINE  : per-clip trim RE-ENCODE + drawtext watermark  -> concat -c copy   (current #521)
  OPTION_A  : pre-render watermark PNG once; per-clip trim RE-ENCODE + overlay  -> concat -c
copy
  OPTION_B  : per-clip trim -c copy (stream-copy)            -> concat RE-ENCODE + drawtext
watermark
  FLOOR     : per-clip trim -c copy                          -> concat -c copy  (no watermark =
theoretical min)

Reports total wall + trim-stage vs concat-stage split, plus output validity (duration).
ffmpeg-only; mirrors backend/pipeline/stitcher/compiler.py arg construction.
"""
import os
import subprocess
import sys
import time

SP = os.path.dirname(os.path.abspath(__file__))
SRC = sys.argv[1] if len(sys.argv) > 1 else os.path.join(SP, "src_4k_hevc10.mp4")
FONT = r"C:/Users/avidu/Projects/khelsutra-guru/bhi-wt-cheap-
watermark/backend/assets/fonts/RobotoSlab-Regular.ttf"

BEGIN_PAD, END_PAD = 0.5, 0.5
RAW_SEGMENTS = [
    (2.0, 6.5), (9.0, 13.5), (15.0, 19.0), (21.5, 25.0),
    (27.0, 31.5), (33.0, 37.0), (39.0, 43.0), (44.0, 47.0),
```

```
]


def ff_quote(path: str) -> str:
    return path.replace("\\", "/").replace("'", r"\'").replace(":", r"\:")


def probe_duration(path: str) -> float:
    try:
        out = subprocess.run(
            ["ffprobe", "-v", "error", "-show_entries", "format=duration",
             "-of", "default=noprint_wrappers=1:nokey=1", path],
            capture_output=True, text=True, timeout=60,
        )
        return float(out.stdout.strip())
    except Exception:
        return 0.0


def probe_res(path: str):
    out = subprocess.run(
        ["ffprobe", "-v", "error", "-select_streams", "v:0", "-show_entries",
         "stream=width,height", "-of", "csv=p=0:s=x", path],
        capture_output=True, text=True, timeout=60,
    )
    w, h = out.stdout.strip().split("x")
    return int(w), int(h)


def drawtext_filter(tmp: str) -> str:
    tf = os.path.join(tmp, "watermark.txt")
    with open(tf, "w", encoding="utf-8") as fh:
        fh.write("Generated by Khelsutra.guru")
    # divisor = round(1/0.035) = 29 ; matches compiler _watermark_filter
    return (
        f"drawtext=textfile='{ff_quote(tf)}':fontfile='{ff_quote(FONT)}'"
        f":fontcolor=white@0.85:fontsize=h/29:x=w-tw-24:y=h-th-24"
        f":box=1:boxcolor=black@0.50:boxborderw=10"
    )


def enc_args(encoder: str):
    if encoder == "libx264":
        return ["-c:v", "libx264", "-preset", "superfast", "-crf", "22"]
    return ["-c:v", encoder, "-preset", "p4", "-cq", "23", "-pix_fmt", "yuv420p"]


def run(cmd) -> float:
    t = time.perf_counter()
    p = subprocess.run(cmd, capture_output=True)
    dt = time.perf_counter() - t
    if p.returncode != 0:
        raise RuntimeError("ffmpeg failed:\n" + " ".join(str(c) for c in cmd)
                           + "\n" + p.stderr.decode(errors="replace")[-800:])
    return dt


def padded(seg):
    s, e = seg
    return max(0.0, s - BEGIN_PAD), e + END_PAD


def clips_txt(tmp, clips):
    lp = os.path.join(tmp, "list.txt")
    with open(lp, "w", encoding="utf-8") as f:
```

```
        for c in clips:
            f.write("file '%s'\n" % c.replace("\\", "/").replace("'", "'\\''"))
    return lp


def do_baseline(tmp, enc):
    va = enc_args(enc)
    dt = drawtext_filter(tmp)
    trim_t = 0.0
    clips = []
    for i, seg in enumerate(RAW_SEGMENTS):
        ps, pe = padded(seg)
        out = os.path.join(tmp, f"b_{i}.mp4")
        trim_t += run(["ffmpeg", "-y", "-ss", str(ps), "-to", str(pe), "-i", SRC,
                       "-vf", dt, *va, "-c:a", "aac", "-b:a", "128k", out])
        clips.append(out)
    lp = clips_txt(tmp, clips)
    outp = os.path.join(tmp, "reel_baseline.mp4")
    concat_t = run(["ffmpeg", "-y", "-f", "concat", "-safe", "0", "-i", lp, "-c", "copy", outp])
    return trim_t, concat_t, outp


def do_option_a(tmp, enc):
    va = enc_args(enc)
    dt = drawtext_filter(tmp)
    W, H = probe_res(SRC)
    wm = os.path.join(tmp, "wm.png")
    # Pre-render the SAME drawtext onto a transparent RGBA canvas ONCE.
    prerender_t = run(["ffmpeg", "-y", "-f", "lavfi", "-i",
                       f"color=c=black@0.0:s={W}x{H}:r=1:d=1,format=rgba",
                       "-vf", dt, "-frames:v", "1", wm])
    ov = "W-w-24:H-h-24"
    wmq = ff_quote(wm)
    trim_t = 0.0
    clips = []
    for i, seg in enumerate(RAW_SEGMENTS):
        ps, pe = padded(seg)
        out = os.path.join(tmp, f"a_{i}.mp4")
        trim_t += run(["ffmpeg", "-y", "-ss", str(ps), "-to", str(pe), "-i", SRC,
                       "-vf", f"movie='{wmq}'[wm];[in][wm]overlay={ov}",
                       *va, "-c:a", "aac", "-b:a", "128k", out])
        clips.append(out)
    lp = clips_txt(tmp, clips)
    outp = os.path.join(tmp, "reel_optA.mp4")
    concat_t = run(["ffmpeg", "-y", "-f", "concat", "-safe", "0", "-i", lp, "-c", "copy", outp])
    return trim_t, concat_t, outp, prerender_t


def do_option_b(tmp, enc):
    va = enc_args(enc)
    dt = drawtext_filter(tmp)
    trim_t = 0.0
    clips = []
    for i, seg in enumerate(RAW_SEGMENTS):
        ps, pe = padded(seg)
        dur = pe - ps
        out = os.path.join(tmp, f"c_{i}.mp4")
        # stream-copy trim: fast seek to keyframe <= ps, copy dur seconds (frame-INACCURATE at
start).
        trim_t += run(["ffmpeg", "-y", "-ss", str(ps), "-i", SRC, "-t", str(dur),
                       "-c", "copy", "-avoid_negative_ts", "make_zero", out])
        clips.append(out)
    lp = clips_txt(tmp, clips)
    outp = os.path.join(tmp, "reel_optB.mp4")
    # single re-encode pass over the concatenated reel, applying the watermark once.
```

```
    concat_t = run(["ffmpeg", "-y", "-f", "concat", "-safe", "0", "-i", lp,
                    "-vf", dt, *va, "-c:a", "aac", "-b:a", "128k", outp])
    return trim_t, concat_t, outp


def do_floor(tmp, enc):
    trim_t = 0.0
    clips = []
    for i, seg in enumerate(RAW_SEGMENTS):
        ps, pe = padded(seg)
        dur = pe - ps
        out = os.path.join(tmp, f"f_{i}.mp4")
        trim_t += run(["ffmpeg", "-y", "-ss", str(ps), "-i", SRC, "-t", str(dur),
                       "-c", "copy", "-avoid_negative_ts", "make_zero", out])
        clips.append(out)
    lp = clips_txt(tmp, clips)
    outp = os.path.join(tmp, "reel_floor.mp4")
    concat_t = run(["ffmpeg", "-y", "-f", "concat", "-safe", "0", "-i", lp, "-c", "copy", outp])
    return trim_t, concat_t, outp


def main():
    if not os.path.exists(SRC):
        print("SOURCE MISSING:", SRC); sys.exit(1)
    sw, sh = probe_res(SRC)
    sdur = probe_duration(SRC)
    exp = sum((e + END_PAD) - max(0.0, s - BEGIN_PAD) for s, e in RAW_SEGMENTS)
    print(f"# source: {sw}x{sh}, {sdur:.1f}s | {len(RAW_SEGMENTS)} segments | expected reel
~{exp:.1f}s\n")

    encoders = ["libx264"]
    # include NVENC only if it actually runs at 4K on this box
    probe = subprocess.run(["ffmpeg", "-hide_banner", "-f", "lavfi", "-i",
                            "testsrc2=s=1920x1080:d=0.2", "-frames:v", "3",
                            "-c:v", "h264_nvenc", "-f", "null", "-"], capture_output=True)
    if probe.returncode == 0:
        encoders.append("h264_nvenc")
    print("# encoders under test:", encoders, "\n")

    import tempfile
    rows = []
    for enc in encoders:
        for name, fn in [("BASELINE", do_baseline), ("OPTION_A", do_option_a),
                         ("OPTION_B", do_option_b), ("FLOOR", do_floor)]:
            # warm run (discard) + measured run, to remove cold-cache noise
            for measured in (False, True):
                with tempfile.TemporaryDirectory(dir=SP) as tmp:
                    try:
                        res = fn(tmp, enc)
                        trim_t, concat_t, outp = res[0], res[1], res[2]
                        pre = res[3] if len(res) > 3 else 0.0
                        dur = probe_duration(outp)
                        sz = os.path.getsize(outp) / 1e6
                    except Exception as ex:
                        if measured:
                            rows.append((enc, name, None, None, None, None, None,
str(ex)[:120]))
                        break
                    if measured:
                        total = trim_t + concat_t + pre
                        rows.append((enc, name, total, trim_t, concat_t, dur, sz, ""))
                        print(f"[{enc:11s}] {name:9s} total={total:7.2f}s "
                              f"(trim={trim_t:7.2f} concat={concat_t:6.2f}"
                              f"{f' pre={pre:.2f}' if pre else ''}) "
                              f"reel={dur:5.1f}s {sz:6.1f}MB")
```

```
    print("\n\n==== SUMMARY (measured wall seconds) ====")
    print(f"{'encoder':12s} {'strategy':10s} {'total':>8s} {'trim':>8s} {'concat':>8s}
{'reel_s':>7s} {'MB':>7s}")
    for enc, name, total, trim_t, concat_t, dur, sz, err in rows:
        if total is None:
            print(f"{enc:12s} {name:10s}  ERROR: {err}")
        else:
            print(f"{enc:12s} {name:10s} {total:8.2f} {trim_t:8.2f} {concat_t:8.2f} {dur:7.1f}
{sz:7.1f}")


if __name__ == "__main__":
    main()
```

52. user (2026-07-08T06:11:21.470Z)

```
File created successfully at: C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-
khelsutra-guru-badminton-highlight-
indexer\46eabfd2-0e71-4ff5-a68d-c76f474d4ff3\scratchpad\wm_bench.py (file state is current in
your context ? no need to Read it back)
```

53. assistant (2026-07-08T06:11:26.400Z)

```
Let me check whether the source finished generating.
```

54. user (2026-07-08T06:11:30.766Z)

```
-rw-r--r-- 1 avidu 197609 65273900 Jul  8 11:40 C:/Users/avidu/AppData/Local/Temp/claude/C--
Users-avidu-Projects-khelsutra-guru-badminton-highlight-
indexer/46eabfd2-0e71-4ff5-a68d-c76f474d4ff3/scratchpad/src_4k_hevc10.mp4
---tail of gen log---
Generating 4K 10-bit HEVC source (mandelbrot, detailed/realistic decode cost, GOP 60, ~48s)...
```

55. assistant (2026-07-08T06:13:03.862Z)

```
While the source generates, let me reconsider the frame-accuracy question, since it's decisive.

**Key insight:** the watermark burns into *every* frame, so every frame must be re-encoded
regardless. That reframes both options:
- **Option A** (pre-rendered overlay, per-clip): frame-accurate (re-encode allows precise `-ss`
cuts), same encode volume as baseline, saves only per-frame `drawtext` rasterization.
- **Option B** (stream-copy trims + concat re-encode): the concat *still* re-encodes the whole
reel for the watermark, so the encode floor doesn't shrink ? it just *moves*. And stream-copy
trims start at keyframes, so each clip is keyframe-extended (encodes *more* footage) *and* loses
frame-accuracy. The "smart-cut boundary GOP" hybrid the task mentions is moot here ? you're re-
encoding every frame for the watermark anyway.

So theory predicts Option B is a wash-or-worse when the watermark is on, and the real per-clip
cost is the irreducible encode. The benchmark's `FLOOR` (stream-copy only) vs `BASELINE`
decomposition will confirm how much is overhead vs encode. Let me check the source.
```

56. user (2026-07-08T06:13:08.601Z)

```
-rw-r--r-- 1 avidu 197609 122945580 Jul  8 11:43 C:/Users/avidu/AppData/Local/Temp/claude/C--
Users-avidu-Projects-khelsutra-guru-badminton-highlight-
indexer/46eabfd2-0e71-4ff5-a68d-c76f474d4ff3/scratchpad/src_4k_hevc10.mp4
---
[mov,mp4,m4a,3gp,3g2,mj2 @ 0000025490481700] moov atom not found
C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-khelsutra-guru-badminton-
highlight-indexer/46eabfd2-0e71-4ff5-a68d-c76f474d4ff3/scratchpad/src_4k_hevc10.mp4: Invalid
data found when processing input
still writing / not finalized
```

57. assistant (2026-07-08T06:13:36.218Z)

Source still finalizing (mandelbrot 4K is CPU-heavy; ~122MB of ~300MB). I'll wait for the completion notification. Meanwhile, let me study the approach-independent wiring ? how `VideoCompiler` is constructed from config in orchestration, so I know exactly where a new knob threads through.

58. user (2026-07-08T06:13:39.651Z)

```
===VideoCompiler construction sites===
backend/main.py:1381:        compiler = VideoCompiler(
backend/orchestration.py:1215:    compiler = _main.VideoCompiler(

===_stitch_reel / compile core in orchestration===
16:``ProcessRequest``, ``compute_video_id``, ``emit_indexer_report``, ``VideoCompiler``.
1105:    and survive the ``stitching.min_shot_count`` floor (segments WITHOUT a known shot_count
are exempt
1128:            if sc is not None and int(sc) < stitching.min_shot_count:
1135:            "[compile] stitching.min_shot_count=%s: dropped %d/%d segments below the
floor.",
1136:            stitching.min_shot_count,
1144:            f"stitching.min_shot_count={stitching.min_shot_count}.",
1190:def _stitch_reel(
1202:    needs NO control-plane DB ? the worker reads them from the staged spec. Builds the
VideoCompiler
1203:    with ``stitching.encoder`` (NVENC on the L4, libx264 on the CP), fetches the source,
runs the
1207:    import backend.main as _main  # call-time seam: VideoCompiler stays patchable on
backend.main
1214:    localized_watermark = localize_watermark(stitching.watermark, locale)
1215:    compiler = _main.VideoCompiler(
1217:        begin_padding=stitching.begin_padding,
1218:        end_padding=stitching.end_padding,
1219:        watermark=localized_watermark,
1220:        encoder=getattr(stitching, "encoder", "libx264"),
1326:    stitch_dump = stitching.model_dump()
```

59. assistant (2026-07-08T06:13:44.251Z)

Let me read both `VideoCompiler` construction sites to see the exact config threading.

60. user (2026-07-08T06:13:45.774Z)

```
1190    def _stitch_reel(
1191        cfg,
1192        stitching,
1193        video_id: str,
1194        source_ref: str,
1195        time_pairs: List[Tuple[float, float]],
1196        out_name: str,
1197        locale: Optional[str],
1198        notify,
1199    ) -> Dict[str, Any]:
1200        """Stitch + mirror-upload core, shared by the control-plane compile handler AND the
L4 worker
1201        (#521). ``time_pairs`` are already RESOLVED (the caller filtered via
``_resolve_compile``), so this
1202        needs NO control-plane DB ? the worker reads them from the staged spec. Builds the
VideoCompiler
1203        with ``stitching.encoder`` (NVENC on the L4, libx264 on the CP), fetches the source,
runs the
1204        trim/watermark/concat, mirrors the reel to
``<output_root>/highlights/<video_id>/<name>``, and
1205        returns the ``{status, filepath, download_url, video_id, reel_uri}`` the SPA polls
for. Returns a
1206        ``status='failed'`` result on a stitch error (never raises for an expected ffmpeg
```

```
failure)."""
1207        import backend.main as _main  # call-time seam: VideoCompiler stays patchable on
backend.main
1208
1209        output_dir = (
1210            getattr(getattr(cfg, "output", None), "output_dir", "output") or "output"
1211        )
1212        # Localize the branding watermark to the UI locale the SPA rendered in (recorded on
the job);
1213        # unknown/absent locale keeps the configured default unchanged.
1214        localized_watermark = localize_watermark(stitching.watermark, locale)
1215        compiler = _main.VideoCompiler(
1216            output_dir,
1217            begin_padding=stitching.begin_padding,
1218            end_padding=stitching.end_padding,
1219            watermark=localized_watermark,
1220            encoder=getattr(stitching, "encoder", "libx264"),
1221        )
1222
1223        notify("stitching")
1224        local_src = ""
1225        try:
1226            # The stored `filepath` is the SOURCE ref (gs://, s3://, gdrive://, or a local
path). Resolve to
1227            # a LOCAL file the stitcher can read ? identity for a local path, fetch-to-
scratch for a URI (the
1228            # same seam the process path uses); without it, compiling a bucket-ingested
video can't open gs://.
1229            local_src = storage.acquire_local_input(
1230                source_ref, getattr(cfg, "storage", None)
1231            )
1232            # The ffmpeg stitch is CPU-heavy ? serialize it so a raised drain worker count
(for remote
1233            # dispatch) doesn't run multiple compiles at once on the box (see
_LOCAL_COMPUTE_LANE).
1234            with _LOCAL_COMPUTE_LANE:
1235                out_path = compiler.compile_highlights(local_src, time_pairs, out_name)
1236        except Exception as e:  # noqa: BLE001 ? a stitch/ffmpeg failure is a clean job
FAILURE, not a crash
1237            logger.error(
1238                "[compile] stitching failed for video_id=%s: %s", video_id, e, exc_info=True
1239            )
```

61. user (2026-07-08T06:13:46.328Z)

```
1370                out_name = out_filename
1371
1372            os.makedirs(out_dir, exist_ok=True)
1373            print(
1374                f"[CLI] Extracting debug segment from {args.trim_start}s to {args.trim_end}s
from {args.video_path}..."
1375            )
1376
1377            # global_config isn't initialized this early; load the typed config so the trim
1378            # clip carries the same configured stitching paddings + watermark as a real
1379            # highlight (watermark default-on).
1380            stitching = load_app_config().stitching
1381            compiler = VideoCompiler(
1382                out_dir,
1383                begin_padding=stitching.begin_padding,
1384                end_padding=stitching.end_padding,
1385                watermark=stitching.watermark,
1386            )
1387            try:
1388                res_path = compiler.compile_highlights(
```

```
1389                     args.video_path, [(args.trim_start, args.trim_end)], out_name
1390                 )
1391                 print(f"[SUCCESS] Trimmed segment saved to: {res_path}")
1392             except Exception as e:
1393                 print(f"[ERROR] Failed to compile trimmed segment: {e}")
1394             return
1395 
1396         if args.debug_clip is not None:
1397             if not args.video_path:
1398                 print("[ERROR] Please provide --video-path to debug a specific clip.")
1399                 return
```

62. assistant (2026-07-08T06:14:00.096Z)

Full config flow understood: a new knob on `StitchingConfig` threads through `_stitch_reel` ? `VideoCompiler` (and rides the cloud compile-spec automatically since the spec carries `stitching`). Same pattern as `encoder`. Let me check the source generation.

63. user (2026-07-08T06:14:05.060Z)

```
-rw-r--r-- 1 avidu 197609 162529324 Jul  8 11:44 C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-khelsutra-guru-badminton-highlight-indexer/46eabfd2-0e71-4ff5-a68d-c76f474d4ff3/scratchpad/src_4k_hevc10.mp4
still generating
```

64. assistant (2026-07-08T06:14:25.086Z)

162MB and climbing. Rather than poll, let me do genuinely independent work: rigorously characterize the NVENC-probe bug I found (to make the flag actionable) by testing which resolutions the probe passes/fails ? this pins down the fix.

65. user (2026-07-08T06:14:33.424Z)

```
color 64x64: FAIL
color 128x128: FAIL
color 145x49: PASS
color 256x256: PASS
color 320x240: PASS
--- with a real (testsrc2) source, same tiny sizes ---
testsrc2 64x64: FAIL
testsrc2 128x128: FAIL
testsrc2 256x256: PASS
```

66. assistant (2026-07-08T06:14:59.413Z)

This precisely characterizes a **latent bug in #521**: `_probe_encoder_available` uses a `64x64` probe, but NVENC's minimum encode size is higher ? `64x64`/`128x128` **fail** while `145x49`/`256x256`+ **pass**, even with a real source. So on any box with a working NVENC (including potentially the L4), the probe *under-reports* NVENC as unavailable and silently falls back to libx264 ? defeating #521's core optimization. It's out of scope for my watermark PR, but worth flagging with the concrete fix while it's fresh.

67. user (2026-07-08T06:15:10.730Z)

Noted (position 1, task_id: task_8a2b7d23). A chip is showing for the user ? they can start it in a fresh worktree with one click, or dismiss it. If this suggestion becomes stale or superseded, call dismiss_task with this task_id. Currently pending: task_8a2b7d23 "Fix NVENC probe: 64x64 under-reports availability". Continue your current work.

68. assistant (2026-07-08T06:15:15.848Z)

Flagged. Let me check the source status.

69. user (2026-07-08T06:15:20.754Z)

```
-rw-r--r-- 1 avidu 197609 232521772 Jul  8 11:45 C:/Users/avidu/AppData/Local/Temp/claude/C--
Users-avidu-Projects-khelsutra-guru-badminton-highlight-
indexer/46eabfd2-0e71-4ff5-a68d-c76f474d4ff3/scratchpad/src_4k_hevc10.mp4
still generating
```

70. assistant (2026-07-08T06:15:33.434Z)

232MB ? nearly done. Rather than poll, let me use the wait productively: read the existing watermark tests (clean origin/master version in my worktree) to understand the test surface I'll extend regardless of which option wins.

71. user (2026-07-08T06:15:34.156Z)

```
1       """Tests for the configurable highlight watermark (stitching.watermark).
2
3       Covers the config defaults, the filtergraph builder (text + logo, positions, font,
4       the configurable string), and the end-to-end wiring through compile_highlights with
5       ffmpeg/ffprobe mocked ? including the graceful fallback when the watermark encode fails.
6       """
7
8       import os
9       import subprocess
10      from pathlib import Path
11      from unittest.mock import MagicMock, patch
12
13      import pytest
14      from pydantic import ValidationError
15
16      from backend.config import AppConfig
17      from backend.config.models import WatermarkConfig
18      from backend.pipeline.stitcher.compiler import VideoCompiler
19
20      # --- config ------------------------------------------------------------------
21
22
23      def test_watermark_config_defaults_on_with_brand_text():
24          wm = AppConfig().stitching.watermark
25          assert wm.enabled is True  # default-on, with an off switch
26          assert wm.text == "Generated by Khelsutra.guru"
27          assert wm.position == "bottom-right"
28
29
30      def test_watermark_can_be_disabled_via_config():
31          cfg = AppConfig.model_validate({"stitching": {"watermark": {"enabled": False}}})
32          assert cfg.stitching.watermark.enabled is False
33
34
35      def test_font_size_ratio_and_opacity_bounds_enforced():
36          with pytest.raises(ValidationError):
37              WatermarkConfig(opacity=1.5)
38          with pytest.raises(ValidationError):
39              WatermarkConfig(font_size_ratio=0.0)
40
41
42      # --- filtergraph builder -----------------------------------------------------
43
44
45      def test_disabled_or_none_builds_no_filter(tmp_path):
46          assert (
47              VideoCompiler(str(tmp_path / "a"), watermark=None)._watermark_filter(
48                  str(tmp_path)
49              )
50              is None
```

```
        )
        off = VideoCompiler(str(tmp_path / "b"), watermark={"enabled": False})
        assert off._watermark_filter(str(tmp_path)) is None


    def test_text_filter_writes_configurable_string_and_builds_drawtext(tmp_path):
        custom = "My Academy 2026 ? Khelsutra.guru"
        comp = VideoCompiler(
            str(tmp_path / "o"), watermark={"enabled": True, "text": custom}
        )
        graph = comp._watermark_filter(str(tmp_path))
        assert graph is not None and graph.startswith("drawtext=")
        # The arbitrary, configurable string is written verbatim to the textfile (no
escaping pain).
        assert (tmp_path / "watermark.txt").read_text(encoding="utf-8") == custom
        assert "textfile=" in graph
        assert "fontcolor=white@0.85" in graph
        assert "fontsize=h/29" in graph  # round(1/0.035) == 29
        assert "x=w-tw-24:y=h-th-24" in graph  # bottom-right default, margin 24
        assert "box=1" in graph


    def test_position_and_margin_map_to_corner(tmp_path):
        comp = VideoCompiler(
            str(tmp_path / "o"),
            watermark={"enabled": True, "text": "X", "position": "top-left", "margin": 12},
        )
        graph = comp._watermark_filter(str(tmp_path))
        assert "x=12:y=12" in graph


    def test_explicit_font_path_uses_fontfile(tmp_path):
        comp = VideoCompiler(
            str(tmp_path / "o1"),
            watermark={"enabled": True, "text": "X", "font_path": "/fonts/Brand.ttf"},
        )
        g = comp._watermark_filter(str(tmp_path))
        assert "fontfile='/fonts/Brand.ttf'" in g


    def test_default_uses_bundled_apache_font(tmp_path):
        # No font_path ? the bundled Apache-2.0 Roboto Slab (renders without fontconfig).
        comp = VideoCompiler(str(tmp_path / "o2"), watermark={"enabled": True, "text": "X"})
        g = comp._watermark_filter(str(tmp_path))
        assert "fontfile=" in g and "RobotoSlab-Regular.ttf" in g
        assert "font='" not in g  # not the fontconfig path


    def test_bundled_font_actually_ships_and_is_apache():
        import backend.pipeline.stitcher.compiler as cmod

        assert cmod._BUNDLED_FONT.exists(), (
            "bundled default watermark font must ship in the repo"
        )
        with open(cmod._BUNDLED_FONT, "rb") as fh:
            assert fh.read(4) in (
                b"\x00\x01\x00\x00",
                b"OTTO",
                b"true",
                b"ttcf",
            )  # valid sfnt
        lic = cmod._BUNDLED_FONT.parent / "LICENSE-Apache-2.0.txt"
        assert lic.exists() and "Apache License" in lic.read_text(
            encoding="utf-8", errors="replace"
        )
```

```


    def test_fontconfig_fallback_when_bundled_missing(tmp_path, monkeypatch):
        import backend.pipeline.stitcher.compiler as cmod

        monkeypatch.setattr(cmod, "_BUNDLED_FONT", Path("/no/such/font.ttf"))
        comp = VideoCompiler(
            str(tmp_path / "o3"),
            watermark={"enabled": True, "text": "X", "font": "DejaVu Sans"},
        )
        g = comp._watermark_filter(str(tmp_path))
        assert "font='DejaVu Sans'" in g and "fontfile=" not in g


    # --- Windows drive-letter colon escaping (regression) -------------------------
    # A Windows absolute path's drive-letter colon (C:/...) must be escaped for the
    # ffmpeg filtergraph, or drawtext/movie parse ':' as an option separator and the
    # encode fails with "Invalid argument" ? which silently disabled the on-by-default
    # bundled-font watermark (and absolute logo paths) on Windows. Verified end-to-end
    # against real ffmpeg before this fix landed.


    def test_ff_quote_escapes_drive_letter_colon():
        assert VideoCompiler._ff_quote(r"C:\fonts\Brand.ttf") == r"C\:/fonts/Brand.ttf"
        # no colon -> unchanged (POSIX paths and the Linux bundled font are unaffected)
        assert (
            VideoCompiler._ff_quote("/usr/share/fonts/font.ttf")
            == "/usr/share/fonts/font.ttf"
        )
        # colon + single-quote + backslash all handled
        assert VideoCompiler._ff_quote(r"D:\a'b\f.ttf") == r"D\:/a\'b/f.ttf"


    def test_watermark_font_path_with_drive_colon_is_escaped(tmp_path):
        # Explicit font_path (literal, existence not required) -> deterministic on any OS.
        comp = VideoCompiler(
            str(tmp_path / "o"),
            watermark={"enabled": True, "text": "X", "font_path": "C:/fonts/Brand.ttf"},
        )
        g = comp._watermark_filter(str(tmp_path))
        assert r"fontfile='C\:/fonts/Brand.ttf'" in g
        assert "fontfile='C:/fonts" not in g  # the raw colon would break drawtext parsing


    def test_logo_plus_text_overlays_then_draws(tmp_path):
        logo = tmp_path / "logo.png"
        logo.write_bytes(b"\x89PNG\r\n")
        comp = VideoCompiler(
            str(tmp_path / "o"),
            watermark={"enabled": True, "text": "X", "logo_path": str(logo)},
        )
        graph = comp._watermark_filter(str(tmp_path))
        assert graph.startswith("movie=")
        assert "overlay=W-w-24:H-h-24" in graph
        assert "drawtext=" in graph


    def test_missing_logo_is_skipped_gracefully(tmp_path):
        comp = VideoCompiler(
            str(tmp_path / "o"),
            watermark={
                "enabled": True,
                "text": "X",
                "logo_path": str(tmp_path / "nope.png"),
            },
```

```
        )
        graph = comp._watermark_filter(str(tmp_path))
        assert "movie=" not in graph and graph.startswith("drawtext=")  # text still drawn


    # --- end-to-end wiring (ffmpeg/ffprobe mocked) -------------------------------


    def _fake_run_factory(calls, fail_on_vf=False):
        def fake_run(cmd, *a, **k):
            calls.append(cmd)
            if cmd[0] == "ffprobe":
                return MagicMock(stdout="100.0\n", returncode=0)
            if fail_on_vf and "-vf" in cmd:
                raise subprocess.CalledProcessError(
                    1, cmd, output=b"", stderr=b"font not found"
                )
            out = cmd[-1]
            if isinstance(out, str) and out.endswith(".mp4"):
                open(out, "wb").write(b"clip")
            return MagicMock(returncode=0, stdout=b"", stderr=b"")

        return fake_run


    def test_compile_applies_watermark_vf_to_each_clip_not_concat(tmp_path):
        raw = tmp_path / "raw.mp4"
        raw.write_bytes(b"x")
        comp = VideoCompiler(
            str(tmp_path / "out"),
            watermark={"enabled": True, "text": "Generated by Khelsutra.guru"},
        )
        calls = []
        with patch.object(subprocess, "run", side_effect=_fake_run_factory(calls)):
            comp.compile_highlights(str(raw), [(0.0, 5.0), (10.0, 15.0)], "reel.mp4")

        trim_cmds = [
            c for c in calls if c[0] == "ffmpeg" and "-i" in c and "concat" not in c
        ]
        assert trim_cmds, "expected per-clip trim invocations"
        for c in trim_cmds:
            assert "-vf" in c
            assert "drawtext=" in c[c.index("-vf") + 1]
        # The concat (-c copy) must NOT re-filter ? the mark is already baked into each
clip.
        concat_cmds = [c for c in calls if "concat" in c]
        assert concat_cmds and all("-vf" not in c for c in concat_cmds)


    def test_compile_without_watermark_adds_no_vf(tmp_path):
        raw = tmp_path / "raw.mp4"
        raw.write_bytes(b"x")
        comp = VideoCompiler(str(tmp_path / "out"))  # no watermark passed
        calls = []
        with patch.object(subprocess, "run", side_effect=_fake_run_factory(calls)):
            comp.compile_highlights(str(raw), [(0.0, 5.0)], "reel.mp4")
        assert all("-vf" not in c for c in calls)


    def test_watermark_failure_falls_back_to_unwatermarked_clip(tmp_path):
        """A bad font/logo must not nuke the reel: the clip re-encodes without the
watermark."""
        raw = tmp_path / "raw.mp4"
        raw.write_bytes(b"x")
        comp = VideoCompiler(
```

```
            str(tmp_path / "out"),
            watermark={"enabled": True, "text": "Generated by Khelsutra.guru"},
        )
        calls = []
        with patch.object(
            subprocess, "run", side_effect=_fake_run_factory(calls, fail_on_vf=True)
        ):
            out_path = comp.compile_highlights(str(raw), [(0.0, 5.0)], "reel.mp4")

        # Reel still produced despite the watermark encode failing.
        assert out_path.endswith("reel.mp4")
        assert os.path.exists(out_path)
        # For the clip, both a watermarked attempt and a plain fallback attempt were made.
        clip_cmds = [
            c for c in calls if c[0] == "ffmpeg" and "concat" not in c and "-i" in c
        ]
        assert any("-vf" in c for c in clip_cmds)  # watermark attempt
        assert any("-vf" not in c for c in clip_cmds)  # fallback attempt


    def test_watermark_failure_logs_the_ffmpeg_reason(tmp_path, caplog):
        """The fallback warning must surface ffmpeg's stderr (the *why*), not go silent ?
        e.g. the 'Fontconfig error' that tells the user to set font_path."""
        import logging as _logging

        raw = tmp_path / "raw.mp4"
        raw.write_bytes(b"x")
        comp = VideoCompiler(
            str(tmp_path / "out"), watermark={"enabled": True, "text": "Y"}
        )

        def fake_run(cmd, *a, **k):
            if cmd[0] == "ffprobe":
                return MagicMock(stdout="100.0\n", returncode=0)
            if "-vf" in cmd:
                raise subprocess.CalledProcessError(
                    1,
                    cmd,
                    output=b"",
                    stderr=b"Fontconfig error: Cannot load default config file",
                )
            out = cmd[-1]
            if isinstance(out, str) and out.endswith(".mp4"):
                open(out, "wb").write(b"clip")
            return MagicMock(returncode=0, stdout=b"", stderr=b"")

        with caplog.at_level(_logging.WARNING):
            with patch.object(subprocess, "run", side_effect=fake_run):
                comp.compile_highlights(str(raw), [(0.0, 5.0)], "reel.mp4")
        assert "Fontconfig error" in caplog.text

```

72. assistant (2026-07-08T06:15:43.367Z)

I now have the complete test surface. Critically,
`test_compile_applies_watermark_vf_to_each_clip_not_concat` locks the *current* contract
(watermark per-clip, concat `-c copy`, no `-vf` on concat) ? so any Option-B behavior must be
opt-in behind a knob with the default keeping that test green. Let me check the source.

73. user (2026-07-08T06:15:48.259Z)

```
-rw-r--r-- 1 avidu 197609 309665754 Jul  8 11:45 C:/Users/avidu/AppData/Local/Temp/claude/C--
Users-avidu-Projects-khelsutra-guru-badminton-highlight-
indexer/46eabfd2-0e71-4ff5-a68d-c76f474d4ff3/scratchpad/src_4k_hevc10.mp4
codec_name=hevc
```

```
width=3840
height=2160
pix_fmt=yuv420p10le
duration=48.000000
FINALIZED
```

74. assistant (2026-07-08T06:16:06.534Z)

Source finalized: **3840×2160, yuv420p10le (10-bit HEVC), 48s, ~310MB (~52 Mbps)** ? a realistically decode-heavy 4K clip mirroring the corpus. Now let me run the benchmark (Baseline/A/B/Floor × libx264+NVENC, warm+measured) in the background and monitor.

75. user (2026-07-08T06:16:08.373Z)

Command running in background with ID: br1xehp9b. Output is being written to: C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-khelsutra-guru-badminton-highlight-indexer\46eabfd2-0e71-4ff5-a68d-c76f474d4ff3\tasks\br1xehp9b.output. You will be notified when it completes. To check interim output, use Read on that file path.

76. assistant (2026-07-08T06:16:19.674Z)

Benchmark running. Let me check interim output to confirm it started cleanly (encoders detected, no immediate errors).

77. user (2026-07-08T06:16:45.387Z)

(Bash completed with no output)

78. assistant (2026-07-08T06:17:30.883Z)

Python buffers stdout when redirected, so interim prints won't flush until exit ? I'll get the full table on completion. Let me verify the benchmark is actively progressing (not wedged) by checking for the temp clip files it creates, rather than waiting blind.

79. user (2026-07-08T06:17:45.356Z)

```
===python running?===
python.exe                   11352 Console                    2     14,200 K
python.exe                   37588 Console                    2    243,916 K
python.exe                   21464 Console                    2     16,760 K
===active temp work dirs (evidence of progress)===
C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-khelsutra-guru-badminton-highlight-indexer/46eabfd2-0e71-4ff5-a68d-c76f474d4ff3/scratchpad/tmp87u1n3pb
===clips in newest tmp===
total 75757
drwxr-xr-x 1 avidu 197609        0 Jul  8 11:47 .
drwxr-xr-x 1 avidu 197609        0 Jul  8 11:47 ..
-rw-r--r-- 1 avidu 197609 39363597 Jul  8 11:47 b_0.mp4
-rw-r--r-- 1 avidu 197609 38199704 Jul  8 11:47 b_1.mp4
-rw-r--r-- 1 avidu 197609        0 Jul  8 11:47 b_2.mp4
-rw-r--r-- 1 avidu 197609       27 Jul  8 11:47 watermark.txt
```

80. assistant (2026-07-08T06:18:31.658Z)

The benchmark is actively progressing ? libx264 4K baseline clips are being encoded (`b_0`/`b_1` at ~38 MB each of detailed content, realistic load). Since Python buffers output until exit, I'll get the full results table on completion. This is a good checkpoint to summarize where things stand.

Status so far:
- **Base reconciled.** #521 (PR #525) is confirmed merged to `origin/master` (`dac6736`). I set up an isolated git **worktree** (`bhi-wt-cheap-watermark`) off master because the shared checkout has a *sibling session's #523 WIP* (including `compiler.py`) that I must not clobber.
- **Found a latent #521 bug** and flagged it (out of scope): `_probe_encoder_available` probes

NVENC at `64x64`, below NVENC's minimum encode size, so it *under-reports* NVENC (fails at 64/128px, passes at ?256px) ? silently falling back to slow libx264 on real GPU boxes, possibly even the L4. Chip filed with the concrete fix.
- **Built an empirical benchmark** on a real 4K 10-bit HEVC clip (mimicking the corpus decode cost), comparing Baseline / Option A (pre-rendered overlay) / Option B (stream-copy trims + concat re-encode) / Floor (no watermark), across **both libx264 and h264_nvenc** (NVENC works at 4K here, so the L4 path is representative). Running now.
- **Key theoretical finding driving the analysis:** the watermark burns into *every* frame, so the encode is an unavoidable floor ? Option B *relocates* the re-encode to the concat pass rather than eliminating it, and loses frame-accuracy (keyframe cuts). The benchmark's Floor-vs-Baseline split will quantify exactly how much is irreducible encode vs per-clip overhead, settling A-vs-B with data.

I'll continue automatically when the benchmark completes.

81. user (2026-07-08T06:26:28.218Z)

```
=== DONE ===
```

source: 3840x2160, 48.0s | 8 segments | expected reel ~40.0s

encoders under test: ['libx264', 'h264_nvenc']

```
[libx264    ] BASELINE  total=  73.10s (trim=  71.26 concat=  1.84) reel= 40.0s  519.8MB
[libx264    ] OPTION_A  total=  59.52s (trim=  57.29 concat=  2.07 pre=0.16) reel= 40.0s
566.7MB
[libx264    ] OPTION_B  total=  65.29s (trim=   1.01 concat= 64.28) reel= 48.1s  568.6MB
[libx264    ] FLOOR     total=   1.10s (trim=   0.79 concat=  0.31) reel= 48.1s  308.0MB
[h264_nvenc ] BASELINE  total=  34.74s (trim=  32.70 concat=  2.04) reel= 40.0s  369.4MB
[h264_nvenc ] OPTION_A  total=  36.02s (trim=  34.27 concat=  1.59 pre=0.15) reel= 40.0s
393.8MB
[h264_nvenc ] OPTION_B  total=  28.91s (trim=   0.98 concat= 27.93) reel= 48.1s  401.8MB
[h264_nvenc ] FLOOR     total=   1.21s (trim=   0.86 concat=  0.35) reel= 48.1s  308.0MB


==== SUMMARY (measured wall seconds) ====
encoder      strategy      total     trim   concat  reel_s      MB
libx264      BASELINE      73.10    71.26     1.84    40.0   519.8
libx264      OPTION_A      59.52    57.29     2.07    40.0   566.7
libx264      OPTION_B      65.29     1.01    64.28    48.1   568.6
libx264      FLOOR          1.10     0.79     0.31    48.1   308.0
h264_nvenc   BASELINE      34.74    32.70     2.04    40.0   369.4
h264_nvenc   OPTION_A      36.02    34.27     1.59    40.0   393.8
h264_nvenc   OPTION_B      28.91     0.98    27.93    48.1   401.8
h264_nvenc   FLOOR          1.21     0.86     0.35    48.1   308.0
```

82. assistant (2026-07-08T06:28:56.769Z)

Benchmark complete ? clean, decisive data. Let me capture the key findings:

encoder	strategy	total	trim	concat	reel_s
libx264	BASELINE	73.1s	71.3	1.8	40.0 ?
libx264	OPTION_A	**59.5s** (?18.6%)	57.3	2.1	40.0 ?
libx264	OPTION_B	65.3s (?10.7%)	1.0	64.3	**48.1 ?**
libx264	FLOOR	1.1s	0.8	0.3	48.1
nvenc	BASELINE	34.7s	32.7	2.0	40.0 ?
nvenc	OPTION_A	36.0s (**+3.7%**)	34.3	1.6	40.0 ?
nvenc	OPTION_B	**28.9s** (?16.8%)	1.0	27.9	**48.1 ?**
nvenc	FLOOR	1.2s	0.9	0.4	48.1

Reading it: FLOOR?1.1s proves the stitch is ~99% watermark *encode* (the floor is real). NVENC is the 2.1× lever (#521). **Option B is frame-INACCURATE** ? reel is 48.1s vs 40.0s

(stream-copy keyframe cuts add ~8s of unwanted lead-in); that's a content/quality regression, not a cosmetic parity relaxation. Option A is frame-accurate and ?18.6% on libx264 but *+3.7% on NVENC*.

Before deciding, there's a **fourth approach worth measuring**: a *single-pass concat-filter* (multiple `-ss` inputs ? `concat` filter ? watermark ? one encode) ? potentially frame-accurate AND faster on *both* encoders (no intermediate I/O, watermark applied once). If it wins, it beats both A and B. Let me measure it.

83. assistant (2026-07-08T06:29:23.384Z)

```
"""Focused bench: OPTION_C single-pass concat-filter (frame-accurate, watermark once) vs
BASELINE.

OPTION_C: one ffmpeg invocation, N precise `-ss/-t` inputs -> concat filter -> drawtext once ->
encode once.
No intermediate clip files. Should be frame-accurate (reel == 40.0s) unlike stream-copy Option
B.
"""
import os
import subprocess
import sys
import time
import tempfile

SP = os.path.dirname(os.path.abspath(__file__))
SRC = sys.argv[1] if len(sys.argv) > 1 else os.path.join(SP, "src_4k_hevc10.mp4")
FONT = r"C:/Users/avidu/Projects/khelsutra-guru/bhi-wt-cheap-
watermark/backend/assets/fonts/RobotoSlab-Regular.ttf"
BEGIN_PAD, END_PAD = 0.5, 0.5
RAW_SEGMENTS = [
    (2.0, 6.5), (9.0, 13.5), (15.0, 19.0), (21.5, 25.0),
    (27.0, 31.5), (33.0, 37.0), (39.0, 43.0), (44.0, 47.0),
]


def ff_quote(p): return p.replace("\\", "/").replace("'", r"\'").replace(":", r"\:")


def dur(path):
    o = subprocess.run(["ffprobe", "-v", "error", "-show_entries", "format=duration",
                        "-of", "default=nk=1:nw=1", path], capture_output=True, text=True)
    try: return float(o.stdout.strip())
    except Exception: return 0.0


def drawtext(tmp):
    tf = os.path.join(tmp, "wm.txt")
    open(tf, "w", encoding="utf-8").write("Generated by Khelsutra.guru")
    return (f"drawtext=textfile='{ff_quote(tf)}':fontfile='{ff_quote(FONT)}'"
            f":fontcolor=white@0.85:fontsize=h/29:x=w-tw-24:y=h-th-24"
            f":box=1:boxcolor=black@0.50:boxborderw=10")


def enc_args(enc):
    return (["-c:v", "libx264", "-preset", "superfast", "-crf", "22"] if enc == "libx264"
            else ["-c:v", enc, "-preset", "p4", "-cq", "23", "-pix_fmt", "yuv420p"])


def run(cmd):
    t = time.perf_counter()
    p = subprocess.run(cmd, capture_output=True)
    dt = time.perf_counter() - t
    if p.returncode != 0:
        raise RuntimeError(p.stderr.decode(errors="replace")[-1200:])
```

```
    return dt


def padded(seg):
    s, e = seg
    return max(0.0, s - BEGIN_PAD), e + END_PAD


def baseline(tmp, enc):
    va, dt = enc_args(enc), drawtext(tmp)
    t0 = time.perf_counter(); clips = []
    for i, seg in enumerate(RAW_SEGMENTS):
        ps, pe = padded(seg); out = os.path.join(tmp, f"b_{i}.mp4")
        run(["ffmpeg", "-y", "-ss", str(ps), "-to", str(pe), "-i", SRC, "-vf", dt,
             *va, "-c:a", "aac", "-b:a", "128k", out]); clips.append(out)
    lp = os.path.join(tmp, "l.txt")
    with open(lp, "w", encoding="utf-8") as f:
        for c in clips: f.write("file '%s'\n" % c.replace("\\", "/").replace("'", "'\\''"))
    outp = os.path.join(tmp, "reel_base.mp4")
    run(["ffmpeg", "-y", "-f", "concat", "-safe", "0", "-i", lp, "-c", "copy", outp])
    return time.perf_counter() - t0, outp


def single_pass(tmp, enc):
    """OPTION_C: N -ss/-t inputs -> concat filter -> drawtext once -> one encode."""
    va, dt = enc_args(enc), drawtext(tmp)
    cmd = ["ffmpeg", "-y"]
    for seg in RAW_SEGMENTS:
        ps, pe = padded(seg)
        cmd += ["-ss", str(ps), "-t", str(pe - ps), "-i", SRC]
    n = len(RAW_SEGMENTS)
    fc = []
    for i in range(n):
        fc.append(f"[{i}:v]setpts=PTS-STARTPTS[v{i}];[{i}:a]asetpts=PTS-STARTPTS[a{i}]")
    concat_in = "".join(f"[v{i}][a{i}]" for i in range(n))
    fc.append(f"{concat_in}concat=n={n}:v=1:a=1[cv][ca]")
    fc.append(f"[cv]{dt}[outv]")
    outp = os.path.join(tmp, "reel_c.mp4")
    t0 = time.perf_counter()
    run(cmd + ["-filter_complex", ";".join(fc), "-map", "[outv]", "-map", "[ca]",
               *va, "-c:a", "aac", "-b:a", "128k", outp])
    return time.perf_counter() - t0, outp


def main():
    encoders = ["libx264"]
    if subprocess.run(["ffmpeg", "-hide_banner", "-f", "lavfi", "-i",
"testsrc2=s=1920x1080:d=0.2",
                       "-frames:v", "3", "-c:v", "h264_nvenc", "-f", "null", "-"],
                      capture_output=True).returncode == 0:
        encoders.append("h264_nvenc")
    exp = sum((e + END_PAD) - max(0.0, s - BEGIN_PAD) for s, e in RAW_SEGMENTS)
    print(f"# expected frame-accurate reel ~{exp:.1f}s | encoders {encoders}\n")
    for enc in encoders:
        for name, fn in [("BASELINE", baseline), ("OPTION_C_singlepass", single_pass)]:
            for measured in (False, True):
                with tempfile.TemporaryDirectory(dir=SP) as tmp:
                    try:
                        wall, outp = fn(tmp, enc)
                        d = dur(outp); sz = os.path.getsize(outp) / 1e6
                    except Exception as ex:
                        if measured: print(f"[{enc:11s}] {name:20s} ERROR: {str(ex)[:200]}")
                        break
                    if measured:
                        acc = "ACCURATE" if abs(d - exp) < 0.6 else f"INACCURATE(+{d-exp:.1f}s)"
```

```
                        print(f"[{enc:11s}] {name:20s} total={wall:7.2f}s reel={d:5.1f}s [{acc}]
{sz:6.1f}MB")


if __name__ == "__main__":
    main()
```

84. user (2026-07-08T06:29:23.447Z)

```
File created successfully at: C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-
khelsutra-guru-badminton-highlight-
indexer\46eabfd2-0e71-4ff5-a68d-c76f474d4ff3\scratchpad\wm_bench2.py (file state is current in
your context ? no need to Read it back)
```

85. assistant (2026-07-08T06:29:30.637Z)

Let me run this focused benchmark (unbuffered so I can watch progress live).

86. user (2026-07-08T06:29:32.888Z)

```
Command running in background with ID: bzuf204va. Output is being written to:
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-khelsutra-guru-badminton-
highlight-indexer\46eabfd2-0e71-4ff5-a68d-c76f474d4ff3\tasks\bzuf204va.output. You will be
notified when it completes. To check interim output, use Read on that file path.
```

87. assistant (2026-07-08T06:29:43.721Z)

Let me wait for the single-pass results (first line appears after ~2.5 min ? baseline libx264 warm+measured).

88. user (2026-07-08T06:36:49.572Z)

```
/usr/bin/bash: line 1: [: 0
0: integer expression expected
/usr/bin/bash: line 1: [: 0
0: integer expression expected
/usr/bin/bash: line 1: [: 0
0: integer expression expected
/usr/bin/bash: line 1: [: 0
0: integer expression expected
/usr/bin/bash: line 1: [: 0
0: integer expression expected
/usr/bin/bash: line 1: [: 0
0: integer expression expected
/usr/bin/bash: line 1: [: 0
0: integer expression expected
/usr/bin/bash: line 1: [: 0
0: integer expression expected
/usr/bin/bash: line 1: [: 0
0: integer expression expected
=== COMPLETE ===
```

expected frame-accurate reel ~40.0s | encoders ['libx264', 'h264_nvenc']

```
[libx264    ] BASELINE             total=  78.01s reel= 40.0s [ACCURATE]  519.8MB
[libx264    ] OPTION_C_singlepass  total=  67.43s reel= 40.0s [ACCURATE]  490.8MB
[h264_nvenc ] BASELINE             total=  38.49s reel= 40.0s [ACCURATE]  369.4MB
[h264_nvenc ] OPTION_C_singlepass  total=  34.07s reel= 40.0s [ACCURATE]  344.8MB
```

89. assistant (2026-07-08T06:40:19.043Z)

OPTION_C (single-pass concat-filter) is the winner ? and it beats both A and B:

```
| encoder | strategy | total | reel | accuracy |
|---|---|---|---|---|
| libx264 | BASELINE | 78.0s | 40.0s | ? |
| libx264 | **OPTION_C** | **67.4s (?13.6%)** | 40.0s | **? frame-accurate** |
| nvenc | BASELINE | 38.5s | 40.0s | ? |
| nvenc | **OPTION_C** | **34.1s (?11.5%)** | 40.0s | **? frame-accurate** |

Unlike Option A, C helps **both** encoders (including the L4/NVENC primary path); unlike Option
B, C is **frame-accurate** (reel stays 40.0s). It also produces **smaller files** (one
continuous encode vs 8 keyframe-forced GOPs) and writes **no intermediate clips**. It's Option
B's "watermark once" intent done via the concat *filter* instead of the stream-copy *demuxer*.

**Decision:** implement Option C as an **opt-in** `single_pass_stitch` knob (default OFF =
current per-clip path, parity-safe, all existing tests green), with a **robust fallback to the
per-clip path** on any failure (matching the repo's "slow reel beats no reel" philosophy) and a
segment-count guard. Let me read the real worktree `compiler.py` to refactor precisely.
```

90. user (2026-07-08T06:40:20.674Z)

```
300
301         def _trim_one_segment(
302             self,
303             idx: int,
304             raw_start: Union[int, float, str],
305             raw_end: Union[int, float, str],
306             segments: List[Tuple[float, float]],
307             video_duration: float,
308             tmp_dir: str,
309             raw_video_path: str,
310             watermark_filter: Optional[str],
311             video_codec_args: Optional[List[str]] = None,
312         ) -> Tuple[Optional[str], Optional[Tuple[int, float, float, str]]]:
313             """Trim a single segment into its own clip file (extracted verbatim from the
314             per-segment body of compile_highlights). Returns (clip_path, skip_record): on
315             success the clip path and None; on a skip (invalid/out-of-range/ffmpeg failure)
316             None and the (idx, start, end, reason) record to record in skipped_segments."""
317             # Normalize timestamps to float seconds ? handles float, int,
318             # plain numeric strings, and colon-separated formats like "MM:SS"
319             start = self._parse_time(raw_start)
320             end = self._parse_time(raw_end)
321             segment_label = (
322                 f"Segment #{idx + 1}/{len(segments)} [{start:.2f}s - {end:.2f}s]"
323             )
324
325             # Validate the segment times
326             if start < 0:
327                 logger.warning(
328                     f"{segment_label}: start_time is negative ({start:.2f}s). Clamping to
0."
329                 )
330                 start = 0.0
331
332             if start >= end:
333                 logger.error(
334                     f"{segment_label}: start_time ({start:.2f}s) >= end_time ({end:.2f}s).
Skipping invalid segment."
335                 )
336                 return None, (idx, start, end, "start >= end")
337
338             # Check if segment extends beyond video duration
339             if video_duration > 0:
340                 if start >= video_duration:
341                     logger.error(
342                         f"{segment_label}: start_time ({start:.2f}s) is beyond the video
duration ({video_duration:.2f}s). "
```

```
343                         f"This segment cannot be extracted ? the video ran out. Skipping."
344                     )
345                     return None, (idx, start, end, "start beyond video end")
346 
347                 if end > video_duration:
348                     original_end = end
349                     end = video_duration
350                     logger.warning(
351                         f"{segment_label}: end_time ({original_end:.2f}s) exceeds video
duration ({video_duration:.2f}s). "
352                         f"Clamping end_time to {video_duration:.2f}s. The segment may be
truncated."
353                     )
354 
355             clip_path = os.path.join(tmp_dir, f"clip_{idx}.mp4")
356 
357             # Precise sub-second trim using fast re-encoding to preserve stream headers
358             padded_start = max(0.0, start - self.begin_padding)
359             padded_end = end + self.end_padding
360 
361             # Clamp padded_end to video duration if known
362             if video_duration > 0 and padded_end > video_duration:
363                 padded_end = video_duration
364                 logger.info(
365                     f"{segment_label}: Padded end ({end + self.end_padding:.2f}s) exceeds
video duration. "
366                     f"Clamping to {video_duration:.2f}s (end padding partially applied)."
367                 )
368 
369             trim_duration = padded_end - padded_start
370             logger.info(
371                 f"Trimming {segment_label} -> [{padded_start:.3f}s - {padded_end:.3f}s]
(duration: {trim_duration:.2f}s)"
372             )
373 
374             # Default to libx264 (the historical settings) when a caller invokes
_trim_one_segment
375             # directly without resolving an encoder ? keeps this method's old behaviour
intact.
376             codec_args = (
377                 video_codec_args
378                 if video_codec_args is not None
379                 else ["-c:v", "libx264", "-preset", "superfast", "-crf", "22"]
380             )
381             base_cmd = [
382                 ffmpeg_bin(),
383                 "-y",
384                 "-ss",
385                 str(round(padded_start, 3)),
386                 "-to",
387                 str(round(padded_end, 3)),
388                 "-i",
389                 raw_video_path,
390                 *codec_args,
391                 "-c:a",
392                 "aac",
393                 "-b:a",
394                 "128k",
395             ]
396             vf_args = ["-vf", watermark_filter] if watermark_filter else []
397 
398             # Try with the watermark first; if that encode fails (e.g. a bad font/logo
399             # path), retry once WITHOUT it so a branding misconfig can never nuke the
```

91. assistant (2026-07-08T06:40:29.809Z)

I have the exact code. Let me implement now, starting with the config knob, then the compiler refactor + single-pass path.

1. Config knob (`StitchingConfig.single_pass_stitch`, default off = parity-safe):

92. user (2026-07-08T06:40:40.051Z)

The file C:\Users\avidu\Projects\khelsutra-guru\bhi-wt-cheap-watermark\backend\config\models.py has been updated successfully. (file state is current in your context ? no need to Read it back)

93. assistant (2026-07-08T06:40:56.343Z)

2. Compiler `__init__` ? add the `single_pass` param:

94. user (2026-07-08T06:41:09.246Z)

The file C:\Users\avidu\Projects\khelsutra-guru\bhi-wt-cheap-watermark\backend\pipeline\stitcher\compiler.py has been updated successfully. (file state is current in your context ? no need to Read it back)

95. assistant (2026-07-08T06:41:46.860Z)

3. Extract `_resolve_padded_window` (shared validation/padding) and refactor `_trim_one_segment` to call it ? a verbatim move, so the per-clip path's behavior is unchanged:

96. user (2026-07-08T06:41:59.811Z)

The file C:\Users\avidu\Projects\khelsutra-guru\bhi-wt-cheap-watermark\backend\pipeline\stitcher\compiler.py has been updated successfully. (file state is current in your context ? no need to Read it back)

97. assistant (2026-07-08T06:42:06.535Z)

4. Insert `_resolve_padded_window` before `_trim_one_segment`:

98. user (2026-07-08T06:42:20.148Z)

The file C:\Users\avidu\Projects\khelsutra-guru\bhi-wt-cheap-watermark\backend\pipeline\stitcher\compiler.py has been updated successfully. (file state is current in your context ? no need to Read it back)

99. assistant (2026-07-08T06:43:42.177Z)

5. Add `_probe_has_audio` (ffprobe sibling, needed by the single-pass concat graph):

100. user (2026-07-08T06:43:47.723Z)

The file C:\Users\avidu\Projects\khelsutra-guru\bhi-wt-cheap-watermark\backend\pipeline\stitcher\compiler.py has been updated successfully. (file state is current in your context ? no need to Read it back)

101. assistant (2026-07-08T06:43:50.809Z)

6. Add `_compile_single_pass` (the core single-pass stitch) before `compile_highlights`:

102. user (2026-07-08T06:44:17.414Z)

The file C:\Users\avidu\Projects\khelsutra-guru\bhi-wt-cheap-watermark\backend\pipeline\stitcher\compiler.py has been updated successfully. (file state is current in your context ? no need to Read it back)

103. assistant (2026-07-08T06:44:25.044Z)

7. Wire the single-pass branch into `compile_highlights` (opt-in, with fall-through to the per-clip path):

104. user (2026-07-08T06:44:30.582Z)

The file C:\Users\avidu\Projects\khelsutra-guru\bhi-wt-cheap-watermark\backend\pipeline\stitcher\compiler.py has been updated successfully. (file state is current in your context ? no need to Read it back)

105. assistant (2026-07-08T06:44:37.825Z)

8. Wire config ? `VideoCompiler` in `orchestration._stitch_reel` (rides the cloud compile-spec automatically since it's on `stitching`):

106. user (2026-07-08T06:44:40.727Z)

The file C:\Users\avidu\Projects\khelsutra-guru\bhi-wt-cheap-watermark\backend\orchestration.py has been updated successfully. (file state is current in your context ? no need to Read it back)

107. assistant (2026-07-08T06:44:45.995Z)

Now let me verify the refactor didn't break the default path. First a syntax check, then the targeted compiler/watermark tests.

108. user (2026-07-08T06:44:50.959Z)

```
SYNTAX OK
===targeted tests===
..............................                                          [100%]
30 passed in 0.67s
```

109. assistant (2026-07-08T06:45:13.843Z)

All 30 existing tests pass ? the refactor and the default-off single-pass path preserve the per-clip behavior. Now the critical step: **verify the single-pass path end-to-end with real ffmpeg** on the actual 4K source (mocked tests can't prove the `filter_complex` produces a valid, frame-accurate, watermarked reel). Let me drive the real `VideoCompiler`.

110. assistant (2026-07-08T06:45:44.106Z)

```
"""End-to-end verification of the single-pass stitch against REAL ffmpeg on the 4K source.

Drives the actual VideoCompiler (single_pass=True) and confirms:
  - frame-accurate reel duration (matches per-clip path, within a frame)
  - valid 4K video + audio streams
  - watermark actually rendered (bottom-right corner differs from a no-watermark reel)
  - NVENC single-pass path also works
  - fallback to per-clip when the single pass can't run
"""
import os
import subprocess
import sys

WT = r"C:/Users/avidu/Projects/khelsutra-guru/bhi-wt-cheap-watermark"
sys.path.insert(0, WT)
SP = os.path.dirname(os.path.abspath(__file__))
SRC = os.path.join(SP, "src_4k_hevc10.mp4")
OUT = os.path.join(SP, "verify_out")
os.makedirs(OUT, exist_ok=True)

from backend.pipeline.stitcher.compiler import VideoCompiler  # noqa: E402

SEGMENTS = [(3.0, 7.0), (12.0, 16.0), (20.0, 24.0)]
WM = {"enabled": True, "text": "Generated by Khelsutra.guru"}
```

```
def dur(p):
    o = subprocess.run(["ffprobe", "-v", "error", "-show_entries", "format=duration",
                        "-of", "default=nk=1:nw=1", p], capture_output=True, text=True)
    return float(o.stdout.strip())


def streams(p):
    o = subprocess.run(["ffprobe", "-v", "error", "-show_entries",
                        "stream=codec_type,codec_name,width,height", "-of", "csv=p=0", p],
                       capture_output=True, text=True)
    return o.stdout.strip()


def corner_frame(reel, t, name):
    """Extract the bottom-right 700x180 corner at time t as a PNG (where the watermark sits)."""
    fp = os.path.join(OUT, name)
    subprocess.run(["ffmpeg", "-y", "-ss", str(t), "-i", reel, "-frames:v", "1",
                    "-vf", "crop=700:180:in_w-700:in_h-180", fp], capture_output=True)
    return fp


def mean_luma(png):
    o = subprocess.run(["ffmpeg", "-i", png, "-vf", "format=gray,signalstats", "-f", "null",
"-"],
                       capture_output=True, text=True)
    for line in o.stderr.splitlines():
        if "YAVG" in line:
            for tok in line.split():
                if tok.startswith("YAVG:"):
                    return float(tok.split(":")[1])
    return -1.0


exp = sum((e + 1.0) - max(0.0, s - 0.5) for s, e in SEGMENTS)  # VideoCompiler defaults 0.5/1.0
print(f"# expected frame-accurate reel = {exp:.2f}s\n")
```

1. per-clip (baseline) WITH watermark ? reference

```
pc = VideoCompiler(OUT, watermark=WM, single_pass=False)
r_pc = pc.compile_highlights(SRC, SEGMENTS, "reel_pc.mp4")
print(f"per-clip     : dur={dur(r_pc):.2f}s streams=[{streams(r_pc)}]")
```

2. single-pass WITH watermark (libx264)

```
spc = VideoCompiler(OUT, watermark=WM, single_pass=True)
r_sp = spc.compile_highlights(SRC, SEGMENTS, "reel_sp.mp4")
print(f"single-pass  : dur={dur(r_sp):.2f}s streams=[{streams(r_sp)}]")
```

3. single-pass WITHOUT watermark

```
spn = VideoCompiler(OUT, watermark={"enabled": False}, single_pass=True)
r_spn = spn.compile_highlights(SRC, SEGMENTS, "reel_sp_nowm.mp4")
print(f"single nowm  : dur={dur(r_spn):.2f}s streams=[{streams(r_spn)}]")
```

4. single-pass NVENC

```
try:
    spv = VideoCompiler(OUT, watermark=WM, single_pass=True, encoder="h264_nvenc")
    r_spv = spv.compile_highlights(SRC, SEGMENTS, "reel_sp_nvenc.mp4")
    print(f"single nvenc : dur={dur(r_spv):.2f}s streams=[{streams(r_spv)}]")
except Exception as ex:
    r_spv = None
    print(f"single nvenc : SKIP/ERROR {str(ex)[:80]}")
```

5. fallback: single_pass=True but a missing source path forces per-clip fallback path.

(Use a valid source but an impossible segment set that yields no windows -> returns
falls through to per-clip, which then raises RuntimeError with no clips. We assert the
single-pass returned None by observing the per-clip path ran. Simpler: force >cap se

```
spcap = VideoCompiler(OUT, watermark=WM, single_pass=True)
spcap._SINGLE_PASS_MAX_SEGMENTS = 1  # force the cap so single-pass declines -> per-clip
fallback
r_fb = spcap.compile_highlights(SRC, SEGMENTS, "reel_fallback.mp4")
print(f"fallback     : dur={dur(r_fb):.2f}s streams=[{streams(r_fb)}] (single-pass declined,
per-clip ran)")
```

--- assertions ---

```
print("\n# checks:")
ok = True

def check(name, cond):
    global ok
    print(f"  [{'PASS' if cond else 'FAIL'}] {name}")
    ok = ok and cond

check(f"single-pass duration ~ per-clip ({dur(r_sp):.2f} vs {dur(r_pc):.2f})", abs(dur(r_sp) -
dur(r_pc)) < 0.6)
check(f"single-pass frame-accurate (~{exp:.1f}s)", abs(dur(r_sp) - exp) < 0.6)
check("single-pass has video+audio", "video" in streams(r_sp) and "audio" in streams(r_sp))
check("single-pass 4K", "3840,2160" in streams(r_sp))
check("no-wm single-pass frame-accurate", abs(dur(r_spn) - exp) < 0.6)
if r_spv:
    check("nvenc single-pass frame-accurate", abs(dur(r_spv) - exp) < 0.6)
check("fallback produced a reel", os.path.exists(r_fb) and abs(dur(r_fb) - exp) < 0.6)
```

watermark visible: bottom-right corner luma differs wm vs no-wm (the white text + box

```
fp_wm = corner_frame(r_sp, 3.0, "corner_wm.png")
fp_nowm = corner_frame(r_spn, 3.0, "corner_nowm.png")
y_wm, y_nowm = mean_luma(fp_wm), mean_luma(fp_nowm)
check(f"watermark rendered (corner luma wm={y_wm:.1f} vs nowm={y_nowm:.1f}, differ)", abs(y_wm -
y_nowm) > 1.0)

print("\nRESULT:", "ALL PASS" if ok else "SOME FAILED")
print("watermarked-corner PNG for visual check:", fp_wm)
sys.exit(0 if ok else 1)
```

111. user (2026-07-08T06:45:44.145Z)

```
File created successfully at: C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-
khelsutra-guru-badminton-highlight-
indexer\46eabfd2-0e71-4ff5-a68d-c76f474d4ff3\scratchpad\verify_singlepass.py (file state is
current in your context ? no need to Read it back)
```

112. assistant (2026-07-08T06:45:50.167Z)

```
Let me run the end-to-end verification against real ffmpeg.
```

113. user (2026-07-08T06:45:52.112Z)

```
Command running in background with ID: b97k9dhwx. Output is being written to:
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-khelsutra-guru-badminton-
highlight-indexer\46eabfd2-0e71-4ff5-a68d-c76f474d4ff3\tasks\b97k9dhwx.output. You will be
notified when it completes. To check interim output, use Read on that file path.
```

114. assistant (2026-07-08T06:45:57.783Z)

```
Let me wait for the verification to complete.
```

115. user (2026-07-08T06:47:48.993Z)

```
=== DONE ===
```

expected frame-accurate reel = 16.50s

```
per-clip     : dur=16.52s streams=[h264,video,3840,2160
aac,audio]
single-pass  : dur=16.50s streams=[h264,video,3840,2160
aac,audio]
single nowm  : dur=16.50s streams=[h264,video,3840,2160
aac,audio]
[compile] encoder 'h264_nvenc' is not runnable in this ffmpeg build ? falling back to libx264
(CPU). On the L4 worker this should be available.
single nvenc : dur=16.50s streams=[h264,video,3840,2160
aac,audio]
[compile] single-pass stitch unavailable/failed; using the per-clip path.
fallback     : dur=16.52s streams=[h264,video,3840,2160
aac,audio] (single-pass declined, per-clip ran)
```

checks:

```
  [PASS] single-pass duration ~ per-clip (16.50 vs 16.52)
  [PASS] single-pass frame-accurate (~16.5s)
  [PASS] single-pass has video+audio
  [PASS] single-pass 4K
  [PASS] no-wm single-pass frame-accurate
  [PASS] nvenc single-pass frame-accurate
  [PASS] fallback produced a reel
  [FAIL] watermark rendered (corner luma wm=-1.0 vs nowm=-1.0, differ)

RESULT: SOME FAILED
watermarked-corner PNG for visual check: C:\Users\avidu\AppData\Local\Temp\claude\C--Users-
avidu-Projects-khelsutra-guru-badminton-highlight-
indexer\46eabfd2-0e71-4ff5-a68d-c76f474d4ff3\scratchpad\verify_out\corner_wm.png
```

116. assistant (2026-07-08T06:48:07.910Z)

Verification is essentially successful ? **all substantive checks pass**: single-pass is frame-accurate (16.50s vs per-clip 16.52s), produces valid 4K H.264 + AAC, the no-watermark and NVENC-requested and fallback paths all work. Two notes:
1. The "nvenc" run **fell back to libx264 via the `_select_encoder` probe** (`64x64` probe bug I flagged) ? the single-pass correctly honored the fallback and still produced a valid reel. My earlier benchmark verified the true NVENC single-pass directly (bypassing the probe): 34.1s, frame-accurate.
2. The only FAIL is my **luma-check helper** (`signalstats` YAVG parsing returned ?1.0 for *both* ? a broken probe, not a broken watermark).

Let me confirm the watermark actually rendered by visually inspecting the extracted corner frames.

117. assistant (2026-07-08T06:48:59.578Z)

The watermark renders perfectly in the single-pass output ? "...d by Khelsutra.guru" with its backing box, cropped to the bottom-right corner. So the end-to-end verification is a **success**; the only FAIL was my broken luma-parsing helper, not the watermark.

Verified with real ffmpeg: single-pass is frame-accurate (16.50s vs per-clip 16.52s), valid 4K H.264+AAC, watermark correct, no-watermark path works, per-clip fallback works, and the NVENC request correctly fell back to libx264 via the (#521-buggy) probe while still producing a valid reel.

Now let me add unit tests for the single-pass path (mocked ffmpeg ? command shape, fallback, no-audio, cap) and the extracted validator.

118. user (2026-07-08T06:49:00.843Z)

```
196
197
198     def test_compile_uses_selected_encoder_in_trim(tmp_path, monkeypatch):
199         """End-to-end (ffmpeg mocked): with an available NVENC encoder, every per-clip trim
command
200         carries -c:v h264_nvenc, and the concat stays -c copy."""
201         from backend.pipeline.stitcher import compiler as comp_mod
202
203         monkeypatch.setattr(comp_mod, "_probe_encoder_available", lambda enc: True)
204         comp = VideoCompiler(str(tmp_path / "out"), encoder="h264_nvenc")
205         raw = tmp_path / "raw.mp4"
206         raw.write_bytes(b"x")
207         trims = []
208
209         def fake_run(cmd, *a, **k):
210             if cmd[0] == "ffprobe":
211                 return MagicMock(stdout="100.0\n", returncode=0)
212             out = cmd[-1]
213             if isinstance(out, str) and out.endswith(".mp4"):
214                 open(out, "wb").write(b"clip")
215                 if "-i" in cmd and cmd[cmd.index("-i") + 1] == str(raw):
216                     trims.append(cmd)
217             return MagicMock(returncode=0, stdout=b"", stderr=b"")
218
219         with patch.object(subprocess, "run", side_effect=fake_run):
220             comp.compile_highlights(str(raw), [(0.0, 5.0), (10.0, 15.0)], "reel.mp4")
221         assert trims, "no trim commands captured"
222         for cmd in trims:
223             assert "h264_nvenc" in cmd  # hardware encode on every clip
224             assert "libx264" not in cmd
225
```

119. assistant (2026-07-08T06:50:04.985Z)

Now let me add unit tests for the single-pass path and the extracted validator, appended to `test_compiler.py`:

120. user (2026-07-08T06:50:39.921Z)

The file C:\Users\avidu\Projects\khelsutra-guru\bhi-wt-cheap-watermark\tests\test_compiler.py has been updated successfully. (file state is current in your context ? no need to Read it back)

121. assistant (2026-07-08T06:50:44.572Z)

Let me run the new tests plus the full compiler/watermark suite:

122. user (2026-07-08T06:50:49.790Z)

```
.....................................                                    [100%]
37 passed in 0.68s
```

123. assistant (2026-07-08T06:50:57.514Z)

All 37 pass (30 original + 7 new). Let me confirm the config field is tested alongside the existing `encoder` config test, mirroring that pattern:

124. user (2026-07-08T06:50:59.833Z)

```
===where is stitching.encoder config default tested?===
tests/test_compiler.py:258:def test_single_pass_defaults_off(tmp_path):
tests/test_compiler.py:259:    """single_pass defaults OFF ? the per-clip path stays the parity-
safe default."""
```

```
tests/test_compiler.py:260:    assert VideoCompiler(str(tmp_path / "o")).single_pass is False
tests/test_compiler.py:263:def
test_single_pass_fuses_reel_in_one_ffmpeg_pass_with_concat_and_watermark(tmp_path):
tests/test_compiler.py:264:    """single_pass=True: ONE ffmpeg encode via -filter_complex
(concat + drawtext), N precise
tests/test_compiler.py:269:        single_pass=True,
tests/test_compiler.py:305:def
test_single_pass_without_watermark_maps_concat_output_directly(tmp_path):
tests/test_compiler.py:307:        str(tmp_path / "out"), watermark={"enabled": False},
single_pass=True
tests/test_compiler.py:332:def test_single_pass_no_audio_builds_video_only_graph(tmp_path):
tests/test_compiler.py:336:        single_pass=True,
tests/test_compiler.py:361:def test_single_pass_falls_back_to_per_clip_on_failure(tmp_path):
tests/test_compiler.py:367:        single_pass=True,
tests/test_compiler.py:394:def test_single_pass_over_cap_falls_back_to_per_clip(tmp_path):
tests/test_compiler.py:399:        single_pass=True,
tests/test_person_detector.py:118:def test_detections_over_windows_single_pass(mock_cap):
tests/test_worker.py:1004:        seen["encoder"] = stitching.encoder
Binary file tests/__pycache__/test_compiler.cpython-313-pytest-9.0.3.pyc matches
```

125. assistant (2026-07-08T06:51:06.554Z)

Let me check how the worker/dispatch test verifies `encoder` flows through ? since `single_pass_stitch` should ride the same `stitching` spec to the L4 worker, I want to confirm that path is covered.

126. user (2026-07-08T06:51:07.328Z)

```
985             json.dump(spec, f)
986         return spec_uri
987
988
989     def test_cmd_compile_stitches_and_writes_marker(tmp_path, monkeypatch):
990         """cmd_compile reads the staged spec, runs the SHARED _stitch_reel core
(VideoCompiler stubbed so
991         no ffmpeg), mirrors the reel, and writes a compile done-marker for the dispatcher's
bucket-poll."""
992         import backend.orchestration as orch
993
994         bucket = tmp_path / "bucket"
995         bucket.mkdir()
996         source = tmp_path / "src.mp4"
997         source.write_bytes(b"SRC")
998         spec_uri = _write_compile_spec(bucket, "vidReel", source, "reel.mp4")
999         done = str(bucket / "_compile" / "tok.done")
1000
1001        seen = {}
1002
1003        def _fake_stitch(cfg, stitching, video_id, source_ref, time_pairs, out_name, locale,
notify):
1004            seen["encoder"] = stitching.encoder
1005            seen["pairs"] = time_pairs
1006            seen["source"] = source_ref
1007            notify("stitching")
1008            return {
1009                "status": "success",
1010                "video_id": video_id,
1011                "download_url": f"/api/highlights/{video_id}/{out_name}",
1012                "reel_uri": f"gs://b/highlights/{video_id}/{out_name}",
1013            }
1014
1015        monkeypatch.setattr(orch, "_stitch_reel", _fake_stitch)
1016
1017        cfg = tmp_path / "c.json"
1018        cfg.write_text('{"storage": {"backend": "local"}}')
```

```
1019        rc = wmain.cmd_compile(
1020            wmain.build_parser().parse_args(
1021                [
1022                    "compile",
1023                    "--spec-uri",
1024                    spec_uri,
1025                    "--out-uri",
1026                    str(bucket),
1027                    "--done-uri",
1028                    done,
1029                    "--config",
```

127. assistant (2026-07-08T06:51:20.940Z)

Since `single_pass_stitch` rides the compile-spec via `stitching.model_dump()` (same mechanism as `encoder`, already tested end-to-end), let me confirm that serialization path and see whether a focused spec round-trip assertion is cheap to add.

128. user (2026-07-08T06:51:24.059Z)

```
===spec stitching serialization in orchestration===
103:            config.model_dump(mode="json")
104:            if hasattr(config, "model_dump")
485:                "results": [r.model_dump() for r in val_results],
592:                    storage_cfg.model_dump() if storage_cfg is not None else None
613:            video_id, staged_uri, "processed", [r.model_dump() for r in val_results]
702:        storage_cfg.model_dump()  # type: ignore[union-attr]  # hasattr-guarded; mypy can't
narrow
703:        if hasattr(storage_cfg, "model_dump")
743:            "results": [r.model_dump() for r in val_results],
919:                [r.model_dump() for r in val_results],
929:                [r.model_dump() for r in val_results],

===test_compile_dispatch stitching/spec assertions===
42:        vid = self._video_id or self.spec["video_id"]
43:        out_name = self.spec["output_name"]
125:    and the L4 encoder (deployment.cloud_stitch_encoder = h264_nvenc by default) ? so the
worker
134:    assert fake.spec["video_id"] == video_id
135:    assert fake.spec["video_uri"].endswith("source.mp4")
137:    assert fake.spec["time_pairs"] == [[2.0, 6.0], [10.0, 14.0], [20.0, 22.0]]
138:    # the L4 stitch uses NVENC (forwarded from deployment.cloud_stitch_encoder)
139:    assert fake.spec["stitching"]["encoder"] == "h264_nvenc"
183:    def _fake_stitch(c, stitching, vid, source, pairs, name, locale, notify):
```

129. assistant (2026-07-08T06:51:29.750Z)

Let me see the exact spec-building in orchestration and the dispatch test around line 125-145, to add a `single_pass_stitch` round-trip assertion where `encoder` is already checked.

130. user (2026-07-08T06:51:32.629Z)

```
===orchestration: where spec['stitching'] is built===
1122:    stitching = getattr(cfg, "stitching", None) or StitchingConfig()
1135:             "[compile] stitching.min_shot_count=%s: dropped %d/%d segments below the
floor.",
1144:             f"stitching.min_shot_count={stitching.min_shot_count}.",
1203:    with ``stitching.encoder`` (NVENC on the L4, libx264 on the CP), fetches the source,
runs the
1214:    localized_watermark = localize_watermark(stitching.watermark, locale)
1217:        begin_padding=stitching.begin_padding,
1218:        end_padding=stitching.end_padding,
1220:        encoder=getattr(stitching, "encoder", "libx264"),
1221:        single_pass=getattr(stitching, "single_pass_stitch", False),
```

```
1239:            "[compile] stitching failed for video_id=%s: %s", video_id, e, exc_info=True

        video, kept, out_name = _resolve_compile(
            db, cfg, video_id, segment_ids, output_filename
        )
    except _CompileError as e:
        return {"status": "failed", "message": e.detail, "video_id": video_id}

    stitching = getattr(cfg, "stitching", None) or StitchingConfig()
    storage_cfg = getattr(cfg, "storage", None)
    out_root = (getattr(storage_cfg, "output_root", "") or "").strip()
    compute = get_compute_backend(cfg, target_override="cloud_run") if out_root else None
    if compute is None:
        # Compile was placed on the L4 but the cloud dispatch isn't actually wired here (no
output
        # bucket / Cloud Run env). Fall back to the in-process stitch rather than fail ? a slow
reel
        # beats no reel. (The placement guard means this is only reachable in a mis-provisioned
setup.)
        logger.warning(
            "[compile] compile placed on the L4 but no cloud backend available (out_root=%r) ? "
            "stitching in-process on the control-plane instead.",
            out_root,
        )
        time_pairs = [(s["start_time"], s["end_time"]) for s in kept]
        return _stitch_reel(
            cfg, stitching, video_id, video["filepath"], time_pairs, out_name, locale, notify
        )

    # Build the compile-spec the worker consumes. time-pairs are JSON lists (frozen-safe on the
wire);
    # the stitching config is the control-plane's RESOLVED one, with the encoder swapped to the
L4's
    # (deployment.cloud_stitch_encoder, NVENC by default) so the worker hardware-encodes the
reel.
    stitch_dump = stitching.model_dump()
    stitch_dump["encoder"] = getattr(
        getattr(cfg, "deployment", None), "cloud_stitch_encoder", "h264_nvenc"
    )
    spec_payload = {
        "video_id": video_id,
        "video_uri": video["filepath"],
        "output_name": out_name,
        "locale": locale,
        "time_pairs": [
            [float(s["start_time"]), float(s["end_time"])] for s in kept
        ],
        "stitching": stitch_dump,
    }
    token = uuid.uuid4().hex
    spec_uri = f"{out_root.rstrip('/')}/_compile/{token}/spec.json"
    try:
        notify("staging compile spec")
        with tempfile.TemporaryDirectory(prefix="cr-compile-") as td:
```

131. assistant (2026-07-08T06:51:43.693Z)

`single_pass_stitch` rides the spec automatically via `stitching.model_dump()` (line 1326) ? no extra wiring needed. Let me read the dispatch test to lock this round-trip with an assertion:

132. user (2026-07-08T06:51:45.444Z)

```
100             "video_id": video_id,
101             "kind": "compile",
102             "params": json.dumps(
```

```
103                 {"segment_ids": segment_ids, "output_filename": name, "locale": locale}
104             ),
105         }
106 
107 
108     def test_compile_dispatches_to_l4_and_returns_reel(compile_env):
109         db, cfg, video_id, seg_ids, _bucket, monkeypatch = compile_env
110         fake = _FakeCompileClient()
111         _inject_fake(monkeypatch, fake)
112 
113         out = orch._execute_compile(cfg, db, _job(video_id, seg_ids, "myreel.mp4"))
114 
115         assert out["status"] == "success"
116         assert out["video_id"] == video_id
117         assert out["download_url"] == f"/api/highlights/{video_id}/myreel.mp4"
118         assert out["compute"]["path"] == "cloud_run"
119         assert out["compute"]["execution"] == "exec-compile-1"
120         assert len(fake.calls) == 1  # exactly one Cloud Run Job execution
121 
122 
123     def test_compile_dispatch_stages_resolved_spec_with_nvenc(compile_env):
124         """The staged compile-spec carries the RESOLVED time-pairs (padding-agnostic
start/end from the DB)
125         and the L4 encoder (deployment.cloud_stitch_encoder = h264_nvenc by default) ? so
the worker
126         hardware-encodes and needs no control-plane DB."""
127         db, cfg, video_id, seg_ids, _bucket, monkeypatch = compile_env
128         fake = _FakeCompileClient()
129         _inject_fake(monkeypatch, fake)
130 
131         orch._execute_compile(cfg, db, _job(video_id, seg_ids))
132 
133         assert fake.spec is not None
134         assert fake.spec["video_id"] == video_id
135         assert fake.spec["video_uri"].endswith("source.mp4")
136         # 3 resolved (start, end) pairs from the DB rows
137         assert fake.spec["time_pairs"] == [[2.0, 6.0], [10.0, 14.0], [20.0, 22.0]]
138         # the L4 stitch uses NVENC (forwarded from deployment.cloud_stitch_encoder)
139         assert fake.spec["stitching"]["encoder"] == "h264_nvenc"
140         # the argv is a `compile` subcommand carrying the spec + done marker
141         argv = fake.calls[0]
142         assert argv[0] == "compile"
143         assert "--spec-uri" in argv and "--done-uri" in argv
144         joined = " ".join(argv)
```

133. assistant (2026-07-08T06:51:54.113Z)

Let me add a concise assertion locking that `single_pass_stitch` rides the compile-spec to the worker:

134. user (2026-07-08T06:51:57.752Z)

The file C:\Users\avidu\Projects\khelsutra-guru\bhi-wt-cheap-watermark\tests\test_compile_dispatch.py has been updated successfully. (file state is current in your context ? no need to Read it back)

135. assistant (2026-07-08T06:52:01.141Z)

Now let me run the dispatch/worker tests, then the full suite + lint + types per my verification discipline.

136. user (2026-07-08T06:52:09.706Z)

```
..................................................................... [ 90%]
```

```
........                                                                 [100%]
80 passed in 4.77s
```

137. assistant (2026-07-08T06:52:16.756Z)

80 pass. Now the full pre-LGTM gate: lint, types, and the whole suite with coverage. Let me check the CI config for the coverage floor and run ruff + mypy on the changed files.

138. user (2026-07-08T06:52:20.164Z)

```
===cov floor / CI config===
.github/workflows/ci.yml:146:        run: python run_all_tests.py --cov=backend --cov=training
--cov-report=term --cov-fail-under=70
.github/workflows/nightly.yml:141:        run: python run_all_tests.py --cov=backend
--cov=training --cov-report=term --cov-fail-under=70

===ruff===
All checks passed!
exit: 0
```

139. assistant (2026-07-08T06:52:23.843Z)

Ruff clean; coverage floor is 70. Let me run mypy (matching CI) and check its config:

140. user (2026-07-08T06:53:08.949Z)

```
===mypy invocation in CI===
.github/workflows/ci.yml:66:  # Type gate (mypy.ini at repo root): lenient green baseline with
an explicit
.github/workflows/ci.yml:68:  # up. Typed core deps are installed so mypy sees their real
signatures
.github/workflows/ci.yml:79:      - name: Install mypy + typed core deps
.github/workflows/ci.yml:80:        # mypy pinned alongside ruff so a type-checker release can't
redden CI overnight.
.github/workflows/ci.yml:81:        run: python -m pip install mypy==2.1.0 pydantic fastapi
uvicorn python-dotenv google-genai numpy
.github/workflows/ci.yml:83:        run: mypy backend training
.github/workflows/ci.yml:136:        # (pytest, pytest-cov, httpx for the fastapi TestClient,
ruff, mypy).
.github/workflows/nightly.yml:74:  # Type gate (mypy.ini at repo root) ? mirrors
ci.yml::typecheck. Typed core
.github/workflows/nightly.yml:75:  # deps are installed so mypy sees real signatures (without
them
.github/workflows/nightly.yml:86:      - name: Install mypy + typed core deps

===run mypy on changed files===
backend\annotations\base.py:88: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs  [annotation-unchecked]
backend\sports\__init__.py:9: note: By default the bodies of untyped functions are not checked,
consider using --check-untyped-defs  [annotation-unchecked]
backend\pipeline\validators\base.py:123: note: By default the bodies of untyped functions are
not checked, consider using --check-untyped-defs  [annotation-unchecked]
backend\providers\__init__.py:9: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs  [annotation-unchecked]
Success: no issues found in 3 source files
```

141. assistant (2026-07-08T06:53:22.733Z)

mypy clean on changed files (the notes are unrelated info). Let me run the full CI gates: `mypy backend training`, `ruff check .`, then the whole test suite with coverage.

142. user (2026-07-08T06:53:31.429Z)

```
===full ruff===
```

```
All checks passed!

===full mypy (backend training)===
backend\annotations\base.py:88: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs  [annotation-unchecked]
backend\sports\__init__.py:9: note: By default the bodies of untyped functions are not checked,
consider using --check-untyped-defs  [annotation-unchecked]
backend\pipeline\validators\base.py:123: note: By default the bodies of untyped functions are
not checked, consider using --check-untyped-defs  [annotation-unchecked]
backend\providers\__init__.py:9: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs  [annotation-unchecked]
Success: no issues found in 150 source files
```

143. assistant (2026-07-08T06:53:36.761Z)

Full ruff + mypy clean (150 files). Now the full test suite with coverage (matching CI). Running in background as it's the heavy gate.

144. user (2026-07-08T06:53:37.975Z)

Command running in background with ID: bzpqhwdgh. Output is being written to: C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-khelsutra-guru-badminton-highlight-indexer\46eabfd2-0e71-4ff5-a68d-c76f474d4ff3\tasks\bzpqhwdgh.output. You will be notified when it completes. To check interim output, use Read on that file path.

145. assistant (2026-07-08T06:53:55.340Z)

While the full suite runs, let me write the required documentation. Let me re-read the actual §3 in the worktree to edit it precisely.

146. user (2026-07-08T06:53:56.334Z)

```
100     ---
101
102     ## 3. NVENC on the L4 ? and the D5/D16(b) parity note
103
104     The dominant term in the CUJ stitch was the **per-clip 4K re-encode** the branding
watermark forces
105     (~67 s/clip on 2 vCPU with libx264). The L4 has a hardware H.264/HEVC encoder, so #521
adds an encoder
106     knob:
107
108     - `StitchingConfig.encoder` ? `libx264` (default, CPU, works everywhere incl. CI) |
`h264_nvenc` |
109       `hevc_nvenc`. `VideoCompiler` **probes** the requested encoder once per run (a 1-frame
lavfi test
110       encode) and **falls back to libx264** if this ffmpeg build can't run it, so a
misconfig / non-GPU
111       box never breaks. All clips share the one chosen encoder ? the concat stays `-c copy`.
112     - `deployment.cloud_stitch_encoder` (default `h264_nvenc`) ? what the compile-dispatch
forwards onto
113       the worker's `stitching.encoder` for the L4 run. Set it to `libx264` to run the L4
stitch on CPU.
114     - The control-plane's **own** local stitch always uses `stitching.encoder` (libx264 by
default) ? the
115       CP has no GPU.
116
117     **Parity note (honest):** [README.md](./README.md) **D5/D16(b)** kept stitch **encode =
libx264** to
118     protect a byte-exact stitch parity guarantee, and explicitly **deferred NVENC encode
behind a config
119     knob, "revisit only if M7 cost data justifies relaxing parity."** The 2026-07-08 CUJ
(~13 min stitch,
120     a hard scaling wall past ~5 concurrent users) **is** that cost data. #521 lands exactly
```

```
the deferred
121     shape ? NVENC **behind a config knob**, libx264 the default, config-reversible per
phase. Enabling
122     NVENC relaxes byte-identity **at the reel ENCODE layer only** (a cosmetic, shareable
artifact); the
123     **WINDOW/rally-level parity** (the analytical output) is untouched ? detection and
windowing are
124     unchanged. If bit-exact reels are ever required, set `cloud_stitch_encoder=libx264`.
125
126     > Follow-up (tracked, not in #521's first cut): a **pre-rendered / overlay-filter
watermark** so the
127     > stitch isn't a full re-encode at all (the deeper win beyond swapping the encoder).
128
129     ---
```

147. assistant (2026-07-08T06:54:10.993Z)

Now let me replace the deferred follow-up note with the completed, measured Part-3 investigation:

148. user (2026-07-08T06:54:45.669Z)

The file C:\Users\avidu\Projects\khelsutra-guru\bhi-wt-cheap-watermark\docs\COMPUTE_DECOUPLED_SERVING\PHASE_PLACEMENT.md has been updated successfully. (file state is current in your context ? no need to Read it back)

149. assistant (2026-07-08T06:54:53.408Z)

Let me update the §5 Files & tests table to include the Part-3 surface. Let me read it first.

150. user (2026-07-08T06:54:54.722Z)

```
205       **no** `--skip-validation`, so the **worker** validates and rejects a bad clip via a
non-zero rc
206       (surfaced as a failed job).
207
208     **Footgun guard:** a clip can only be validated on the worker if the worker also
**detects** it there.
209     Two protections:
210
211     1. **Config-load guard** (`DeploymentConfig` model-validator): `validation=cloud_run_l4`
requires the
212        effective segment placement to also be `cloud_run_l4` (explicit, or via
`compute_target=cloud_run`)
213        ? else config load fails with an actionable error.
214     2. **Runtime guard** (`_will_detect_on_cloud`): if a per-request `compute='local'`
forces detection
215        in-process (no worker), the control-plane **still validates** ? never leave a clip
unvalidated
216        because nobody ran the gate.
217
218     ---
219
220     ## 5. Files & tests
221
222     | Area | Code | Tests |
223     |---|---|---|
224     | Placement resolver + config | `backend/config/placement.py`,
`backend/config/models.py` (`PhasePlacementConfig`, `DeploymentConfig.phase_placement` /
`cloud_stitch_encoder`, guard) | `tests/test_phase_placement.py`,
`tests/test_config_deployment.py` |
225     | Preset | `backend/config/presets.py` (`cloud-serving` ?
`phase_placement.compile=cloud_run_l4`) | `tests/test_phase_placement.py` |
226     | compile dispatch | `backend/orchestration.py` (`_execute_compile`,
```

`_dispatch_compile_remote`, `_stitch_reel`, `_will_detect_on_cloud`) |
`tests/test_compile_dispatch.py` |

151. user (2026-07-08T06:55:01.714Z)

```
226     | compile dispatch | `backend/orchestration.py` (`_execute_compile`,
`_dispatch_compile_remote`, `_stitch_reel`, `_will_detect_on_cloud`) |
`tests/test_compile_dispatch.py` |
227     | compute backend | `backend/compute/base.py` (`CompileJobSpec`, `run_compile`),
`backend/compute/cloud_run.py` (`run_compile`, `_await_marker`) |
`tests/test_compute_backend.py` |
228     | worker | `backend/worker/__main__.py` (`compile` subcommand, `cmd_compile`) |
`tests/test_worker.py` |
229     | NVENC encoder | `backend/pipeline/stitcher/compiler.py` (`encoder`, `_select_encoder`,
`_probe_encoder_available`) | `tests/test_compiler.py` |
230     | validation delegation | `backend/orchestration.py` (`_maybe_dispatch_remote` skip
flag, `_execute_processing`) | `tests/test_api_cloud_dispatch.py` |
231
232     All GPU-free + cloud-free: a fake `CloudRunJobClient` simulates the worker (reads the
staged spec,
233     writes the reel + done-marker into a local "bucket"), mirroring
`tests/test_api_cloud_dispatch.py`.
234
```

152. user (2026-07-08T06:55:09.971Z)

```
The file C:\Users\avidu\Projects\khelsutra-guru\bhi-wt-cheap-
watermark\docs\COMPUTE_DECOUPLED_SERVING\PHASE_PLACEMENT.md has been updated successfully. (file
state is current in your context ? no need to Read it back)
```

153. assistant (2026-07-08T06:55:12.226Z)

Let me check the full test suite result.

154. user (2026-07-08T06:57:55.271Z)

```
=== output ===
backend\pipeline\validators\base.py                  85      3    96%
backend\pipeline\validators\frame_sampler.py         54      1    98%
backend\providers\__init__.py                        16      1    94%
backend\providers\base.py                             9      1    89%
backend\providers\gemini.py                         137     21    85%
backend\reporting\__init__.py                        26      2    92%
backend\reporting\harvest.py                        111      8    93%
backend\reporting\run_signals.py                     14      0   100%
backend\results.py                                   27      0   100%
backend\server_runtime.py                            57      2    96%
backend\sports\__init__.py                           17      1    94%
backend\sports\badminton.py                          13      1    92%
backend\sports\base.py                               12      3    75%
backend\storage\__init__.py                          85      3    96%
backend\storage\base.py                              16      1    94%
backend\storage\capabilities.py                      59     10    83%
backend\storage\gcs.py                               80      3    96%
backend\storage\gdrive.py                            61      3    95%
backend\storage\gdrive_api.py                       105      6    94%
backend\storage\local.py                             49      3    94%
backend\storage\ref.py                               49      0   100%
backend\storage\s3.py                                47      5    89%
backend\tools\__init__.py                             0      0   100%
backend\tools\export.py                              75      2    97%
backend\tools\rally_slicer.py                       105     47    55%
backend\utils\__init__.py                             0      0   100%
backend\utils\annotation_proxy.py                    64     11    83%
```

```
backend\utils\app_home.py                              40      0   100%
backend\utils\ffmpeg.py                                47      3    94%
backend\utils\freespace.py                             32      2    94%
backend\utils\geometry.py                              29      0   100%
backend\utils\hashing.py                                9      0   100%
backend\utils\logging_setup.py                         11      0   100%
backend\utils\paths.py                                 44      3    93%
backend\utils\redact.py                                16      0   100%
backend\utils\run_telemetry.py                        126     12    90%
backend\utils\single_instance.py                       43     11    74%
backend\utils\video.py                                 34      3    91%
backend\utils\workspace.py                             49      0   100%
backend\worker\__init__.py                              0      0   100%
backend\worker\__main__.py                            371     66    82%
training\__init__.py                                    0      0   100%
training\gen0\__init__.py                               0      0   100%
training\gen0\harness.py                              130      9    93%
-----------------------------------------------------------------------
TOTAL                                               12670   1950    85%
Required test coverage of 70% reached. Total coverage: 84.61%
========================== short test summary info ==========================
FAILED tests/test_import_smoke.py::test_backend_package_has_no_syntax_errors
FAILED tests/test_light_tier_no_torch.py::test_light_tier_imports_without_torch
FAILED tests/test_main_module_budget.py::test_main_py_line_budget - Assertion...
3 failed, 1879 passed, 8 skipped, 25 warnings in 250.92s (0:04:10)
=============================================================================
 RESULT: FAIL - pytest exit 1
=============================================================================
```

155. assistant (2026-07-08T06:58:58.027Z)

?? **Stop ? the worktree state changed underneath me.** The post-run notes show my worktree's `compiler.py` now contains a `_PROBE_SOURCE_SIZE = "256x256"` NVENC-probe fix I did **not** write (that's the separate task I *flagged*), and the shown `__init__` appears to **lack my `single_pass` param**. Meanwhile `test_backend_package_has_no_syntax_errors` FAILED ? a syntax error somewhere in `backend/`. A concurrent process may have written into my supposedly-isolated worktree. I must assess the actual current state before doing anything else.

156. user (2026-07-08T06:59:04.452Z)

```
===git status===

===my markers present in compiler.py?===
single_pass-related lines: 0
===external NVENC fix present?===
37:_PROBE_SOURCE_SIZE = "256x256"
42:    lavfi test encode to null at ``_PROBE_SOURCE_SIZE`` (a resolution ? NVENC's minimum, so
the
62:                f"color=c=black:s={_PROBE_SOURCE_SIZE}:d=0.1",

===syntax check whole backend===
no syntax errors in backend/
```

157. assistant (2026-07-08T06:59:40.532Z)

My uncommitted changes to `compiler.py` are gone ? `git status` is clean, 0 `single_pass` markers remain, but the external NVENC fix (`_PROBE_SOURCE_SIZE`) is present. A concurrent process wrote into my supposedly-isolated worktree. I need the full picture before recovering ? what happened to HEAD, and whether my other files survived.

158. user (2026-07-08T06:59:45.581Z)

```
===HEAD + branch===
```

```
codex/issue-523-sd-reel-review
65ee4c6 fix(serving): probe NVENC at 256x256, not 64x64, so it isn't falsely rejected
(#521/#525) (#527)
008a5d4 fix(compile): fail the L4-worker stitch when the reel mirror-upload fails (#521 follow-
up) (#528)
dac6736 perf(serving): run compile/stitch on the L4 worker; config-driven phase?machine
placement (#521) (#525)
bf97700 docs(runlogs): record live SPA CUJ closing the 2 owner-gated validation-latency cells
(§9) (#522)

===reflog (last 8)===
65ee4c6 HEAD@{0}: reset: moving to 65ee4c64dcab41c74f63613c973e9069d0167b8d
65ee4c6 HEAD@{1}: checkout: moving from perf/521-cheap-watermark to codex/issue-523-sd-reel-
review
65ee4c6 HEAD@{2}: pull --ff-only: Fast-forward
dac6736 HEAD@{3}: reset: moving to HEAD
dac6736 HEAD@{4}: reset: moving to HEAD
dac6736 HEAD@{5}:

===my markers across files===
backend/pipeline/stitcher/compiler.py: 0 single_pass hits
backend/config/models.py: 0 single_pass hits
backend/orchestration.py: 0 single_pass hits
tests/test_compiler.py: 0 single_pass hits
tests/test_compile_dispatch.py: 0 single_pass hits
docs/COMPUTE_DECOUPLED_SERVING/PHASE_PLACEMENT.md: 0 single_pass hits
```

159. assistant (2026-07-08T07:01:08.857Z)

Now I understand the collision. The reflog shows a **concurrent session hijacked my worktree**: `checkout: moving from perf/521-cheap-watermark to codex/issue-523-sd-reel-review`, then `reset: moving to 65ee4c6`. My uncommitted single-pass work (never committed) was wiped from disk. Notably, **my flagged NVENC-probe bug is now fixed and merged (#527)**, and #528 also landed.

Two lessons: (1) I have every edit in my conversation history, so I can reconstruct; (2) I must **commit immediately** after re-applying so a working-tree reset can't destroy it again, and work in a **scratchpad-isolated** worktree siblings can't discover. Let me re-sync and assess the new base.

160. user (2026-07-08T07:01:16.228Z)

```
===origin/master tip===
65ee4c6 fix(serving): probe NVENC at 256x256, not 64x64, so it isn't falsely rejected
(#521/#525) (#527)
008a5d4 fix(compile): fail the L4-worker stitch when the reel mirror-upload fails (#521 follow-
up) (#528)
dac6736 perf(serving): run compile/stitch on the L4 worker; config-driven phase?machine
placement (#521) (#525)

===#527/#528 on origin/master?===
65ee4c6(#527) IS on origin/master

===did #527/#528 touch my files (vs dac6736)?===
 backend/orchestration.py                          | 49 ++++++++++++--
 backend/pipeline/stitcher/compiler.py             | 19 +++--
 docs/COMPUTE_DECOUPLED_SERVING/PHASE_PLACEMENT.md |  8 ++-
 tests/test_compile_dispatch.py                    | 85 ++++++++++++++++++++++++-
 tests/test_compiler.py                            | 62 ++++++++++++++++++
 5 files changed, 211 insertions(+), 12 deletions(-)

===my leftover branch?===
  perf/521-cheap-watermark
```

161. assistant (2026-07-08T07:02:02.769Z)

Now I have the recovery picture. origin/master advanced to `65ee4c6` (#525 + #528 + **#527 = my flagged NVENC fix, now merged**). My changes were never committed, so they're gone from disk ? but I have every edit in history. #527/#528 touched some of the same files, so I'll re-apply against the **new** base and **commit immediately** this time. I'll work in a **scratchpad-isolated** worktree that sibling automations can't discover.

162. user (2026-07-08T07:02:10.171Z)

```
Exit code 1
===models.py touched by #527/#528?===
models.py UNCHANGED since dac6736 (safe verbatim re-apply)

===create fresh isolated worktree in scratchpad off origin/master===
error: unable to create file docs/COMPUTE_DECOUPLED_SERVING/runlogs/DEPLOY_REPORT_cloud-
serving_gpu-resident-nvdec_2026-07-07.md: Filename too long
error: unable to create file docs/COMPUTE_DECOUPLED_SERVING/runlogs/DEPLOY_REPORT_cloud-
serving_validation-latency_2026-07-07.md: Filename too long
fatal: Could not reset index file to revision 'HEAD'.

/usr/bin/bash: line 1: cd: C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-
khelsutra-guru-badminton-highlight-indexer/46eabfd2-0e71-4ff5-a68d-c76f474d4ff3/scratchpad/wt-
sp: No such file or directory
```

163. assistant (2026-07-08T07:02:23.265Z)

Windows MAX_PATH ? the scratchpad path is too long for the repo's deep doc filenames. The existing worktrees used short `C:/kg-wt*` roots for this reason. Let me clean up the partial worktree and use a short path.

164. user (2026-07-08T07:02:34.178Z)

```
===prune broken worktree===
error: failed to delete 'C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/.git/worktrees/a1-master-worktree': Permission denied
error: failed to delete 'C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/.git/worktrees/bhi-nvenc-probe-wt': Permission denied
error: failed to delete 'C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/.git/worktrees/bhi-wt-worker-reval': Permission denied
error: failed to delete 'C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/.git/worktrees/fix-worktree': Permission denied
error: failed to delete 'C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/.git/worktrees/kg-wt521': Permission denied
error: failed to delete 'C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/.git/worktrees/kg-wt525fix': Permission denied
error: failed to delete 'C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/.git/worktrees/master-compare': Permission denied
error: failed to delete 'C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/.git/worktrees/pr465': Permission denied
error: failed to delete 'C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/.git/worktrees/pr465-current-merge': Permission denied
error: failed to delete 'C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/.git/worktrees/pr465-merge': Permission denied
error: failed to delete 'C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/.git/worktrees/pr467': Permission denied
error: failed to delete 'C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/.git/worktrees/pr469': Permission denied
error: failed to delete 'C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/.git/worktrees/wt-465fix': Permission denied
error: failed to delete 'C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/.git/worktrees/wt-a1-runlog': Permission denied
error: failed to delete 'C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/.git/worktrees/wt-claudemd-r25': Permission denied
error: failed to delete 'C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/.git/worktrees/wt-cloudrun-cancel': Permission denied
```

```
error: failed to delete 'C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/.git/worktrees/wt-cloudrun-dispatch': Permission denied
error: failed to delete 'C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/.git/worktrees/wt-cp-rebuild-spec': Permission denied
error: failed to delete 'C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/.git/worktrees/wt-dockerfile-auth': Permission denied
error: failed to delete 'C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/.git/worktrees/wt-docs': Permission denied
error: failed to delete 'C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/.git/worktrees/wt-docs-reconcile': Permission denied
error: failed to delete 'C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/.git/worktrees/wt-drain-concurrency': Permission denied
error: failed to delete 'C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/.git/worktrees/wt-ffmpeg-timeouts': Permission denied
error: failed to delete 'C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/.git/worktrees/wt-gdrive-api-safety': Permission denied
error: failed to delete 'C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/.git/worktrees/wt-idempotent-process': Permission denied
error: failed to delete 'C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/.git/worktrees/wt-input-jail': Permission denied
error: failed to delete 'C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/.git/worktrees/wt-output-dir-config': Permission denied
error: failed to delete 'C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/.git/worktrees/wt-p1-genyaml': Permission denied
error: failed to delete 'C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/.git/worktrees/wt-p2-upload-stream': Permission denied
error: failed to delete 'C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/.git/worktrees/wt-p4-md5cache': Permission denied
error: failed to delete 'C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/.git/worktrees/wt-p5a-pollknob': Permission denied
error: failed to delete 'C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/.git/worktrees/wt-p6-heartbeat': Permission denied
error: failed to delete 'C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/.git/worktrees/wt-pr481-review': Permission denied
error: failed to delete 'C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/.git/worktrees/wt-retention-sweep': Permission denied
error: failed to delete 'C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/.git/worktrees/wt-review-492-mergecheck': Permission denied
error: failed to delete 'C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/.git/worktrees/wt-review-493-mergecheck': Permission denied
error: failed to delete 'C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/.git/worktrees/wt-scratch-reclaim': Permission denied
error: failed to delete 'C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/.git/worktrees/wt-signed-download': Permission denied
error: failed to delete 'C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/.git/worktrees/wt-source-cache': Permission denied
error: failed to delete 'C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/.git/worktrees/wt-upload-dir': Permission denied
error: failed to delete 'C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/.git/worktrees/wt-upload-staging': Permission denied
Deleted branch perf/521-single-pass-stitch (was 65ee4c6).

===create at short path C:/kg-wt-sp===
Preparing worktree (new branch 'perf/521-single-pass-stitch')
branch 'perf/521-single-pass-stitch' set up to track 'origin/master'.
HEAD is now at 65ee4c6 fix(serving): probe NVENC at 256x256, not 64x64, so it isn't falsely
rejected (#521/#525) (#527)
===verify===
perf/521-single-pass-stitch
65ee4c6 fix(serving): probe NVENC at 256x256, not 64x64, so it isn't falsely rejected
(#521/#525) (#527)
(clean if empty)
has #527 probe fix
Shell cwd was reset to C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer
```

165. assistant (2026-07-08T07:03:22.337Z)

New isolated worktree ready at `C:/kg-wt-sp` on `perf/521-single-pass-stitch` @ 65ee4c6 (latest master with #527/#528), clean tree. (The prune "permission denied" noise is just other sessions' locked worktrees ? this environment has dozens of concurrent worktrees, which explains the hijack.) `models.py` is unchanged since my base; the others were touched by #527/#528, so I'll re-apply against the new content and **commit immediately**.

Let me read the new base regions I need to re-apply into.

166. user (2026-07-08T07:03:22.919Z)

```
1       import logging
2       import os
3       import subprocess
4       import tempfile
5       from pathlib import Path
6       from typing import Any, Dict, List, Optional, Tuple, Union
7
8       from backend.utils.ffmpeg import (  # P2a: single ffmpeg/ffprobe resolver
9           ffmpeg_bin,
10          ffmpeg_timeout_sec,
11          ffprobe_bin,
12          ffprobe_timeout_sec,
13          timeout_label,
14      )
15
16      logger = logging.getLogger(__name__)
17
18      # Bundled, commercially-clean default watermark font (Apache-2.0). Shipped so the
overlay
19      # renders out-of-the-box without a system fontconfig setup (a common gap on Windows,
where
20      # `font='Sans'` fails). Overridable via stitching.watermark.font_path. This module lives
at
21      # backend/pipeline/stitcher/, so parents[2] == backend/.
22      _BUNDLED_FONT = (
23          Path(__file__).resolve().parents[2] / "assets" / "fonts" / "RobotoSlab-Regular.ttf"
24      )
25
26      #: Per-encoder availability memo (#521): probing ffmpeg is a subprocess, so a run's
first clip pays
27      #: it and the rest reuse the verdict. Keyed by encoder name; cleared only across
processes.
28      _ENCODER_AVAILABLE: Dict[str, bool] = {}
29
30      #: Frame geometry for the 1-frame NVENC probe encode in ``_probe_encoder_available``. It
MUST stay
31      #: at or above NVENC's minimum encode dimensions: on real GPUs the hardware H.264/HEVC
encoders
32      #: REJECT tiny frames, so a sub-minimum probe (the original 64x64) FALSELY reports NVENC
as
33      #: unavailable and ``_select_encoder`` silently falls back to libx264 ? defeating the
NVENC-on-L4
34      #: re-encode win of #521/#525. Empirically (RTX 2070 Super, ffmpeg 8.1.1) h264_nvenc
rejects
35      #: 64x64 and 128x128 but accepts ~145px and up; 256x256 keeps a safe margin. Do NOT
shrink below
36      #: 256x256 ? guarded by tests/test_compiler.py::test_probe_encoder_uses_safe_resolution.
37      _PROBE_SOURCE_SIZE = "256x256"
38
39
40      def _probe_encoder_available(encoder: str) -> bool:
41          """Can this ffmpeg build actually RUN ``encoder`` (not merely list it)? Runs a tiny
1-frame
```

```
        lavfi test encode to null at ``_PROBE_SOURCE_SIZE`` (a resolution ? NVENC's minimum,
so the
        probe reflects real hardware runnability rather than rejecting an undersized frame).
NVENC is
        often *compiled in* on boxes without an NVIDIA GPU, so a real test encode ? not
``-encoders``
        grep ? is what distinguishes "usable" from "will fail at runtime". Memoized. Any
error /
        timeout ? unavailable (caller falls back to libx264)."""
        if encoder in _ENCODER_AVAILABLE:
            return _ENCODER_AVAILABLE[encoder]
        ok = False
        # The probe is a tiny 1-frame encode ? bound it to a short timeout (? the configured
ffmpeg
        # budget, or 30 s when unbounded) so a wedged encoder can't hang the whole stitch.
        configured = ffmpeg_timeout_sec()
        probe_timeout = min(30.0, float(configured)) if configured else 30.0
        try:
            proc = subprocess.run(
                [
                    ffmpeg_bin(),
                    "-hide_banner",
                    "-f",
                    "lavfi",
                    "-i",
                    f"color=c=black:s={_PROBE_SOURCE_SIZE}:d=0.1",
                    "-frames:v",
                    "1",
                    "-c:v",
                    encoder,
                    "-f",
                    "null",
                    "-",
                ],
                capture_output=True,
                timeout=probe_timeout,
            )
            ok = proc.returncode == 0
        except Exception:  # noqa: BLE001 ? a probe failure/timeout just means "not usable ?
fall back"
            ok = False
        _ENCODER_AVAILABLE[encoder] = ok
        return ok


    class VideoCompiler:
        def __init__(
            self,
            output_dir: str,
            end_padding: float = 1.0,
            begin_padding: float = 0.5,
            watermark: Optional[Any] = None,
            encoder: str = "libx264",
        ):
            self.output_dir = output_dir
            self.end_padding = end_padding
            self.begin_padding = begin_padding
            # Optional branding overlay. Accepts a WatermarkConfig model or a plain dict;
            # normalized to a dict (or None) so this module stays config-package-agnostic.
            self.watermark = self._normalize_watermark(watermark)
            # Video encoder for the per-clip trim re-encode (#521). "libx264" (CPU) is the
default and
            # works everywhere; "h264_nvenc"/"hevc_nvenc" use the L4's HARDWARE encoder for
the 4K
            # re-encode the watermark forces. The requested encoder is PROBED once per run
```

```
            # (_select_encoder) and falls back to libx264 if this ffmpeg build can't run it.
            self.encoder = (encoder or "libx264").strip() or "libx264"
            os.makedirs(output_dir, exist_ok=True)

        @staticmethod
        def _normalize_watermark(watermark: Optional[Any]) -> Optional[Dict[str, Any]]:
            if watermark is None:
                return None
            if hasattr(watermark, "model_dump"):
                return watermark.model_dump()
            if isinstance(watermark, dict):
                return dict(watermark)
            return None

        @staticmethod
        def _ff_quote(path: str) -> str:
            """Make a filesystem path safe to embed inside single quotes in an ffmpeg
            filtergraph operand (drawtext fontfile/textfile, movie= source): forward
            slashes (Windows-friendly), escaped single quotes, AND escaped colons.

            The colon MUST be escaped even inside single quotes: ffmpeg's filtergraph
            option parser treats ':' as the option separator, so a Windows drive-letter
            path like ``C:/dir/font.ttf`` otherwise truncates the operand at ``C`` and
            the encode fails with "Invalid argument". This silently disabled the
            on-by-default bundled-font watermark (and absolute logo paths) on Windows."""
            return path.replace("\\", "/").replace("'", r"\'").replace(":", r"\:")

        @staticmethod
        def _parse_time(val: Union[int, float, str]) -> float:
            """Normalizes a timestamp value to float seconds.
            Handles: float/int (pass-through), plain numeric strings ("12.4"),
            and colon-separated strings ("MM:SS", "HH:MM:SS", "H:MM:SS.ms").
            This mirrors the parse_time logic used in gemini.py to ensure
            the stitcher can always consume whatever format the indexer produced."""
            if isinstance(val, (int, float)):
                return float(val)
            if isinstance(val, str):
                val = val.strip()
                if ":" in val:
                    # Handle MM:SS, HH:MM:SS, etc.
                    parts = val.split(":")
                    parts.reverse()
                    total = 0.0
                    for i, p in enumerate(parts):
                        try:
                            total += float(p) * (60**i)
                        except ValueError:
                            pass
                    return total
                try:
                    return float(val)
                except ValueError:
                    logger.warning(
                        f"Could not parse timestamp value '{val}' as a number. Defaulting to
0.0."
                    )
                    return 0.0
            logger.warning(
                f"Unexpected timestamp type {type(val).__name__} for value '{val}'.
Defaulting to 0.0."
            )
            return 0.0

        @staticmethod
        def _merge_overlapping(
```

```
            segments: List[Tuple[float, float]],
        ) -> List[Tuple[float, float]]:
            """Normalize, sort, and merge overlapping/duplicate (start, end) ranges.

            A highlight reel must never replay the same footage. The same rally can land in
            the segment list more than once ? e.g. two detector runs accumulated for one
            video (add_video is an UPSERT, so re-running without a prune leaves both sets),
or
            chunk-overlap duplicates near boundaries. Stitching those verbatim plays the
rally
            twice. Merging any range that overlaps or abuts the previous one collapses true
            duplicates/overlaps while preserving genuinely distinct (gapped) rallies.
            Degenerate rows (end <= start) are dropped. Timestamps may be float/int/str.
            """
            parsed = []
            for raw_s, raw_e in segments:
                s = VideoCompiler._parse_time(raw_s)
                e = VideoCompiler._parse_time(raw_e)
                if e > s:
                    parsed.append((s, e))
            parsed.sort()
            merged: List[Tuple[float, float]] = []
            for s, e in parsed:
                if merged and s <= merged[-1][1]:  # overlaps/abuts previous -> extend it
                    merged[-1] = (merged[-1][0], max(merged[-1][1], e))
                else:
                    merged.append((s, e))
            return merged

        def _get_video_duration(self, video_path: str) -> float:
            """Probes the video file to get its total duration in seconds.
            Returns 0.0 if it cannot be determined."""
            try:
                timeout = ffprobe_timeout_sec()
                result = subprocess.run(
                    [
                        ffprobe_bin(),
                        "-v",
                        "error",
                        "-show_entries",
                        "format=duration",
                        "-of",
                        "default=noprint_wrappers=1:nokey=1",
                        video_path,
                    ],
                    capture_output=True,
                    text=True,
                    check=True,
                    timeout=timeout,
                )
                return float(result.stdout.strip())
            except subprocess.TimeoutExpired:
                logger.warning(
                    "Could not determine video duration via ffprobe: timed out after %s",
                    timeout_label(ffprobe_timeout_sec()),
                )
                return 0.0
            except Exception as e:
                logger.warning(f"Could not determine video duration via ffprobe: {e}")
                return 0.0

        def _watermark_filter(self, tmp_dir: str) -> Optional[str]:
            """Build the ffmpeg -vf filtergraph that burns the configured watermark (text
            and/or logo PNG) into a clip during the per-clip re-encode. Returns None when
            watermarking is disabled or has nothing to draw. The concat stage stays -c copy
```

```
            because the mark is already baked into every clip identically."""
            wm = self.watermark
            if not wm or not wm.get("enabled"):
                return None

            text = (wm.get("text") or "").strip()
            logo_path = (wm.get("logo_path") or "").strip()
            if logo_path and not os.path.exists(logo_path):
                logger.warning(
                    f"[watermark] logo not found: {logo_path} ? skipping logo overlay."
                )
                logo_path = ""
            if not text and not logo_path:
                return None

            margin = int(wm.get("margin", 24))
            opacity = float(wm.get("opacity", 0.85))
            position = str(wm.get("position", "bottom-right"))
            # drawtext uses lowercase w/h (main) + tw/th (text); overlay uses W/H (main) +
w/h (logo).
            dt_xy = {
                "bottom-right": f"x=w-tw-{margin}:y=h-th-{margin}",
                "bottom-left": f"x={margin}:y=h-th-{margin}",
                "top-right": f"x=w-tw-{margin}:y={margin}",
                "top-left": f"x={margin}:y={margin}",
            }.get(position, f"x=w-tw-{margin}:y=h-th-{margin}")
            ov_xy = {
                "bottom-right": f"W-w-{margin}:H-h-{margin}",
                "bottom-left": f"{margin}:H-h-{margin}",
                "top-right": f"W-w-{margin}:{margin}",
                "top-left": f"{margin}:{margin}",
            }.get(position, f"W-w-{margin}:H-h-{margin}")

            drawtext = ""
            if text:
                ratio = float(wm.get("font_size_ratio") or 0.035)
                divisor = max(1, round(1.0 / ratio))
                # textfile= sidesteps the notorious drawtext text= escaping for arbitrary
strings.
                tf_path = os.path.join(tmp_dir, "watermark.txt")
                with open(tf_path, "w", encoding="utf-8") as fh:
                    fh.write(text)
                # Font precedence: explicit font_path ? bundled Apache-2.0 default (renders
                # without fontconfig) ? fontconfig family (last-resort, environment-
dependent).
                font_path = (wm.get("font_path") or "").strip()
                if not font_path and _BUNDLED_FONT.exists():
                    font_path = str(_BUNDLED_FONT)
                if font_path:
                    font_opt = f"fontfile='{self._ff_quote(font_path)}'"
                else:
                    font_opt = f"font='{wm.get('font', 'Sans')}'"
                box = ""
                if wm.get("box", True):
                    box = f":box=1:boxcolor=black@{min(opacity, 0.5):.2f}:boxborderw=10"
                drawtext = (
                    f"drawtext=textfile='{self._ff_quote(tf_path)}':{font_opt}"
                    f":fontcolor=white@{opacity:.2f}:fontsize=h/{divisor}:{dt_xy}{box}"
                )

            if logo_path and drawtext:
                return
f"movie='{self._ff_quote(logo_path)}'[wm];[in][wm]overlay={ov_xy},{drawtext}"
            if logo_path:
                return f"movie='{self._ff_quote(logo_path)}'[wm];[in][wm]overlay={ov_xy}"
```

```
286             return drawtext
287
288         def _select_encoder(self) -> Tuple[List[str], str]:
289             """Resolve the ``-c:v ?`` args used for EVERY per-clip trim re-encode this run,
plus the
290             chosen encoder name. Chosen ONCE (not per clip) so all clips share one codec and
the final
291             concat can stay ``-c copy``. NVENC (hardware) is the L4 win on the 4K re-encode;
if the
292             requested NVENC encoder can't run in this ffmpeg build (CI / non-GPU box), fall
back to
293             libx264 so a misconfig never nukes the reel."""
294             enc = self.encoder
295             if enc in ("h264_nvenc", "hevc_nvenc") and not _probe_encoder_available(enc):
296                 logger.warning(
297                     "[compile] encoder %r is not runnable in this ffmpeg build ? falling
back to "
298                     "libx264 (CPU). On the L4 worker this should be available.",
299                     enc,
300                 )
301                 enc = "libx264"
302             if enc == "libx264":
303                 # Unchanged historical CPU settings (byte-compatible with the pre-#521
stitch).
304                 return ["-c:v", "libx264", "-preset", "superfast", "-crf", "22"], enc
305             # NVENC (H.264/HEVC): hardware encode. -cq is the CRF-equivalent constant-
quality knob; p4 is
306             # a balanced speed/quality preset. yuv420p keeps the output broadly playable.
307             return (
308                 ["-c:v", enc, "-preset", "p4", "-cq", "23", "-pix_fmt", "yuv420p"],
309                 enc,
310             )
311
312         def _trim_one_segment(
313             self,
314             idx: int,
315             raw_start: Union[int, float, str],
316             raw_end: Union[int, float, str],
317             segments: List[Tuple[float, float]],
318             video_duration: float,
319             tmp_dir: str,
320             raw_video_path: str,
321             watermark_filter: Optional[str],
322             video_codec_args: Optional[List[str]] = None,
323         ) -> Tuple[Optional[str], Optional[Tuple[int, float, float, str]]]:
324             """Trim a single segment into its own clip file (extracted verbatim from the
325             per-segment body of compile_highlights). Returns (clip_path, skip_record): on
326             success the clip path and None; on a skip (invalid/out-of-range/ffmpeg failure)
327             None and the (idx, start, end, reason) record to record in skipped_segments."""
328             # Normalize timestamps to float seconds ? handles float, int,
329             # plain numeric strings, and colon-separated formats like "MM:SS"
330             start = self._parse_time(raw_start)
331             end = self._parse_time(raw_end)
332             segment_label = (
333                 f"Segment #{idx + 1}/{len(segments)} [{start:.2f}s - {end:.2f}s]"
334             )
335
336             # Validate the segment times
337             if start < 0:
338                 logger.warning(
339                     f"{segment_label}: start_time is negative ({start:.2f}s). Clamping to
0."
340                 )
341                 start = 0.0
342
```

```
            if start >= end:
                logger.error(
                    f"{segment_label}: start_time ({start:.2f}s) >= end_time ({end:.2f}s).
Skipping invalid segment."
                )
                return None, (idx, start, end, "start >= end")

            # Check if segment extends beyond video duration
            if video_duration > 0:
                if start >= video_duration:
                    logger.error(
                        f"{segment_label}: start_time ({start:.2f}s) is beyond the video
duration ({video_duration:.2f}s). "
                        f"This segment cannot be extracted ? the video ran out. Skipping."
                    )
                    return None, (idx, start, end, "start beyond video end")

                if end > video_duration:
                    original_end = end
                    end = video_duration
                    logger.warning(
                        f"{segment_label}: end_time ({original_end:.2f}s) exceeds video
duration ({video_duration:.2f}s). "
                        f"Clamping end_time to {video_duration:.2f}s. The segment may be
truncated."
                    )

            clip_path = os.path.join(tmp_dir, f"clip_{idx}.mp4")

            # Precise sub-second trim using fast re-encoding to preserve stream headers
            padded_start = max(0.0, start - self.begin_padding)
            padded_end = end + self.end_padding

            # Clamp padded_end to video duration if known
            if video_duration > 0 and padded_end > video_duration:
                padded_end = video_duration
                logger.info(
                    f"{segment_label}: Padded end ({end + self.end_padding:.2f}s) exceeds
video duration. "
                    f"Clamping to {video_duration:.2f}s (end padding partially applied)."
                )

            trim_duration = padded_end - padded_start
            logger.info(
                f"Trimming {segment_label} -> [{padded_start:.3f}s - {padded_end:.3f}s]
(duration: {trim_duration:.2f}s)"
            )

            # Default to libx264 (the historical settings) when a caller invokes
_trim_one_segment
            # directly without resolving an encoder ? keeps this method's old behaviour
intact.
            codec_args = (
                video_codec_args
                if video_codec_args is not None
                else ["-c:v", "libx264", "-preset", "superfast", "-crf", "22"]
            )
            base_cmd = [
                ffmpeg_bin(),
                "-y",
                "-ss",
                str(round(padded_start, 3)),
                "-to",
                str(round(padded_end, 3)),
                "-i",
```

```
                raw_video_path,
                *codec_args,
                "-c:a",
                "aac",
                "-b:a",
                "128k",
            ]
            vf_args = ["-vf", watermark_filter] if watermark_filter else []

            # Try with the watermark first; if that encode fails (e.g. a bad font/logo
            # path), retry once WITHOUT it so a branding misconfig can never nuke the
            # whole reel. Clips stay codec-identical either way, so concat -c copy holds.
            attempts = [vf_args, []] if vf_args else [[]]
            produced = False
            last_exc: Optional[subprocess.CalledProcessError] = None
            cmd = base_cmd + [clip_path]
            timeout = ffmpeg_timeout_sec()
            for attempt_vf in attempts:
                cmd = base_cmd + attempt_vf + [clip_path]
                try:
                    subprocess.run(
                        cmd, capture_output=True, check=True, timeout=timeout
                    )
                except subprocess.TimeoutExpired:
                    reason = f"ffmpeg timeout after {timeout_label(timeout)}"
                    logger.error(f"{segment_label}: FFmpeg trim timed out.")
                    print(f"  Command: {' '.join(cmd)}")
                    print(f"  {reason}")
                    return None, (idx, start, end, reason)
                except subprocess.CalledProcessError as e:
                    last_exc = e
                    if attempt_vf and vf_args:
                        _err = (
                            e.stderr.decode(errors="replace").strip()[-300:]
                            if getattr(e, "stderr", None)
                            else ""
                        )
                        logger.warning(
                            f"{segment_label}: watermark encode failed; retrying without it.
"
                            f"ffmpeg: {_err or '(no stderr captured)'}"
                        )
                        continue
                    break
                if os.path.exists(clip_path) and os.path.getsize(clip_path) > 0:
                    produced = True
                    if not attempt_vf and vf_args:
                        logger.warning(
                            f"{segment_label}: encoded WITHOUT the watermark (the watermark
filter failed)."
                        )
                    break
                last_exc = None  # ffmpeg succeeded but produced nothing
                break

            if produced:
                return clip_path, None
            elif last_exc is not None:
                stderr_msg = last_exc.stderr.decode(errors="replace").strip()
                logger.error(f"{segment_label}: FFmpeg trim failed.")
                print(f"  Command: {' '.join(cmd)}")
                print(
                    f"  FFmpeg stderr: {stderr_msg[-500:]}"
                )  # Last 500 chars to avoid flooding
                return None, (idx, start, end, f"ffmpeg error: {stderr_msg[-200:]}")
```

```
            else:
                logger.error(
                    f"{segment_label}: FFmpeg exited successfully but output clip is missing
or empty. Skipping."
                )
                return None, (idx, start, end, "empty output clip")

        def compile_highlights(
            self,
            raw_video_path: str,
            segments: List[Tuple[float, float]],
            output_filename: str,
        ) -> str:
            """
            Extracts selected segments from a raw video and stitches them together
            into a seamless highlights file, applying end_padding to prevent abrupt endings.

            .. warning::
               This method uses a two-stage codec contract: per-clip re-encodes must use
               IDENTICAL encoder settings (codec, preset, CRF, pixel format) so the final
               concat can stream-copy (``-c copy``) without re-encoding. Changing the
               clip encode settings without updating the concat assumptions will produce
               broken output.
            """
            if not segments:
                raise ValueError("No highlight segments provided to compile.")

            # Never stitch the same footage twice: collapse overlapping/duplicate ranges
            # (e.g. accumulated re-run segments or chunk-overlap dupes) before trimming.
            original_count = len(segments)
            segments = self._merge_overlapping(segments)
            if len(segments) < original_count:
                logger.info(
                    "Merged %d overlapping/duplicate segment(s) before stitching: %d ->
%d.",
                    original_count - len(segments),
                    original_count,
                    len(segments),
                )
            if not segments:
                raise ValueError("No valid (start < end) highlight segments after merge.")

            if not os.path.exists(raw_video_path):
                raise FileNotFoundError(
                    f"[COMPILER ERROR] Source video file not found: {raw_video_path}"
                )

            # Defense-in-depth: never let the output name escape output_dir (the API also
            # validates this at the request boundary).
            output_path = os.path.join(self.output_dir, os.path.basename(output_filename))

            # Probe video duration so we can detect out-of-range segments
            video_duration = self._get_video_duration(raw_video_path)
            if video_duration > 0:
                logger.info(f"Source video duration: {video_duration:.2f}s")
            else:
                logger.warning(
                    "Video duration unknown. Cannot validate segment boundaries against
video length."
                )

            # We will create temporary clips and stitch them using FFmpeg concat demuxer
            temp_clips = []
            skipped_segments = []

```

```
            # Create a temp directory within output_dir
            with tempfile.TemporaryDirectory(dir=self.output_dir) as tmp_dir:
                logger.info(
                    f"Trimming {len(segments)} segments
(begin_padding={self.begin_padding}s, end_padding={self.end_padding}s)..."
                )
                # Build the (optional) watermark filtergraph once; it is applied during
every
                # per-clip re-encode so the mark is baked into each clip identically.
                watermark_filter = self._watermark_filter(tmp_dir)
                if watermark_filter:
                    logger.info("[watermark] applying branding overlay to each clip.")
                # Resolve the video encoder ONCE (probe NVENC / fall back) so every clip
shares one codec
                # and the final concat stays -c copy (#521).
                video_codec_args, chosen_encoder = self._select_encoder()
                if chosen_encoder != "libx264":
                    logger.info("[compile] encoding clips with %s (hardware).",
chosen_encoder)
                for idx, (raw_start, raw_end) in enumerate(segments):
                    clip_path, skip_record = self._trim_one_segment(
                        idx,
                        raw_start,
                        raw_end,
                        segments,
                        video_duration,
                        tmp_dir,
                        raw_video_path,
                        watermark_filter,
                        video_codec_args,
                    )
                    if clip_path is not None:
                        temp_clips.append(clip_path)
                    if skip_record is not None:
                        skipped_segments.append(skip_record)

                # Report summary of skipped segments
                if skipped_segments:
                    logger.info(
                        f"\n[SUMMARY] {len(skipped_segments)} out of {len(segments)}
segments were SKIPPED:"
                    )
                    for seg_idx, seg_start, seg_end, reason in skipped_segments:
                        logger.info(
                            f"  - Segment #{seg_idx + 1} [{seg_start:.2f}s -
{seg_end:.2f}s]: {reason}"
                        )
                    logger.info("")

                if not temp_clips:
                    raise RuntimeError(
                        f"[COMPILER ERROR] Failed to generate ANY valid highlight clips. "
                        f"All {len(segments)} segments were skipped. "
                        f"Check the errors above for details. "
                        f"Common causes: video file is corrupted, segments are out of range,
or ffmpeg is not installed."
                    )

                logger.info(
                    f"Successfully trimmed {len(temp_clips)}/{len(segments)} clips.
Preparing to stitch..."
                )

                # Create a list file for FFmpeg concat demuxer
                list_file_path = os.path.join(tmp_dir, "clips.txt")
```

```
582                 # encoding pinned: a non-ASCII clip path would crash on Windows (cp1252
default);
583                 # ffmpeg's concat demuxer reads the list as UTF-8 on every OS.
584                 with open(list_file_path, "w", encoding="utf-8") as f:
585                     for clip in temp_clips:
586                         # The concat demuxer resolves relative entries against the list
file's
587                         # directory. Use absolute paths so a temp-dir clip does not become
588                         # output/tmp.../output/tmp.../clip_0.mp4 on Windows.
589                         safe_path = Path(clip).resolve().as_posix().replace("'", "'\\''")
590                         f.write(f"file '{safe_path}'\n")
591
592                 logger.info("Stitching clips together...")
593                 # Stitch them together without re-encoding (since they were encoded with
identical settings)
594                 concat_cmd = [
595                     ffmpeg_bin(),
596                     "-y",
597                     "-f",
598                     "concat",
599                     "-safe",
600                     "0",
601                     "-i",
602                     list_file_path,
603                     "-c",
604                     "copy",
605                     output_path,
606                 ]
607
608                 try:
609                     timeout = ffmpeg_timeout_sec()
610                     subprocess.run(
611                         concat_cmd, capture_output=True, check=True, timeout=timeout
612                     )
613
614                     # Final verification
615                     if not os.path.exists(output_path) or os.path.getsize(output_path) == 0:
616                         raise RuntimeError(
617                             f"[COMPILER ERROR] FFmpeg concatenation appeared to succeed, "
618                             f"but output file is missing or empty: {output_path}"
619                         )
620
621                     output_size_mb = os.path.getsize(output_path) / (1024 * 1024)
622                     logger.info(
623                         f"Dynamic highlight reel saved: {output_path} ({output_size_mb:.1f}
MB)"
624                     )
625                     return output_path
626                 except subprocess.CalledProcessError as e:
627                     stderr_msg = e.stderr.decode(errors="replace").strip()
628                     raise RuntimeError(
629                         f"[COMPILER ERROR] FFmpeg concatenation failed.\n"
630                         f"  Command: {' '.join(concat_cmd)}\n"
631                         f"  FFmpeg stderr: {stderr_msg[-500:]}"
632                     ) from e
633                 except subprocess.TimeoutExpired as e:
634                     raise RuntimeError(
635                         f"[COMPILER ERROR] FFmpeg concatenation timed out after
{timeout_label(timeout)}.\n"
636                         f"  Command: {' '.join(concat_cmd)}"
637                     ) from e
638
```

167. assistant (2026-07-08T07:04:11.566Z)

The new `compiler.py` (with #527's probe fix at lines 30-78) is **identical to my base in every region I edited** ? `__init__`, `_watermark_filter`, `_trim_one_segment`, `compile_highlights` are unchanged. So my exact edits re-apply cleanly. Let me re-apply all 6 compiler edits, then commit immediately.

168. user (2026-07-08T07:04:23.192Z)

The file C:\kg-wt-sp\backend\pipeline\stitcher\compiler.py has been updated successfully. (file state is current in your context ? no need to Read it back)

169. user (2026-07-08T07:04:30.516Z)

The file C:\kg-wt-sp\backend\pipeline\stitcher\compiler.py has been updated successfully. (file state is current in your context ? no need to Read it back)

170. user (2026-07-08T07:04:50.697Z)

The file C:\kg-wt-sp\backend\pipeline\stitcher\compiler.py has been updated successfully. (file state is current in your context ? no need to Read it back)

171. user (2026-07-08T07:05:05.295Z)

The file C:\kg-wt-sp\backend\pipeline\stitcher\compiler.py has been updated successfully. (file state is current in your context ? no need to Read it back)

172. user (2026-07-08T07:05:33.506Z)

The file C:\kg-wt-sp\backend\pipeline\stitcher\compiler.py has been updated successfully. (file state is current in your context ? no need to Read it back)

173. user (2026-07-08T07:05:42.619Z)

The file C:\kg-wt-sp\backend\pipeline\stitcher\compiler.py has been updated successfully. (file state is current in your context ? no need to Read it back)

174. assistant (2026-07-08T07:05:47.653Z)

compiler.py fully re-applied. Now `models.py` (Read required for the new worktree path) and `orchestration.py` (touched by #528 ? need to see the current `_stitch_reel`).

175. user (2026-07-08T07:05:48.684Z)

```
366         min_rally_duration: float = Field(default=0.0, ge=0.0)
367         # Video encoder for the per-clip trim re-encode (the branding watermark forces a re-
encode; the
368         # concat stays -c copy). Default ``libx264`` (CPU, works everywhere incl. CI).
``h264_nvenc`` /
369         # ``hevc_nvenc`` are the L4 HARDWARE encoders ? dramatically faster on the 4K re-
encode that
370         # dominated the 2026-07-08 CUJ stitch (#521). The compiler probes the requested
encoder ONCE and
371         # falls back to libx264 if this ffmpeg build can't run it, so a non-GPU box never
breaks. The
372         # cloud compile-dispatch forwards ``deployment.cloud_stitch_encoder`` onto this for
the L4 run;
373         # the control-plane's own local stitch keeps libx264.
374         encoder: Literal["libx264", "h264_nvenc", "hevc_nvenc"] = "libx264"
375         watermark: WatermarkConfig = Field(default_factory=WatermarkConfig)
376
377
```

176. user (2026-07-08T07:05:49.172Z)

```
1206        returns the ``{status, filepath, download_url, video_id, reel_uri}`` the SPA polls
for. Returns a
1207        ``status='failed'`` result on a stitch error (never raises for an expected ffmpeg
failure).
1208
1209        ``require_mirror`` (the L4-worker caller sets True): on the worker the bucket mirror
is the ONLY
1210        delivery ? the worker's local reel is discarded when the Job exits, so a mirror-
upload failure or a
1211        missing bucket destination MUST fail the stitch (else the dispatcher reports success
for a reel the
1212        SPA can never download ? 404 on a "success" job). The control-plane caller keeps the
default False:
1213        its local reel is a valid fallback, so a mirror miss stays best-effort."""
1214        import backend.main as _main  # call-time seam: VideoCompiler stays patchable on
backend.main
1215
1216        output_dir = (
1217            getattr(getattr(cfg, "output", None), "output_dir", "output") or "output"
1218        )
1219        # Localize the branding watermark to the UI locale the SPA rendered in (recorded on
the job);
1220        # unknown/absent locale keeps the configured default unchanged.
1221        localized_watermark = localize_watermark(stitching.watermark, locale)
1222        compiler = _main.VideoCompiler(
1223            output_dir,
1224            begin_padding=stitching.begin_padding,
1225            end_padding=stitching.end_padding,
1226            watermark=localized_watermark,
1227            encoder=getattr(stitching, "encoder", "libx264"),
```

177. assistant (2026-07-08T07:05:58.526Z)

Both anchors intact (`#528` added a `require_mirror` param but not to the `VideoCompiler` construction). Re-applying models.py and orchestration.py edits:

178. user (2026-07-08T07:06:04.233Z)

The file C:\kg-wt-sp\backend\config\models.py has been updated successfully. (file state is current in your context ? no need to Read it back)

179. user (2026-07-08T07:06:08.597Z)

The file C:\kg-wt-sp\backend\orchestration.py has been updated successfully. (file state is current in your context ? no need to Read it back)

180. assistant (2026-07-08T07:06:32.589Z)

All code re-applied. Given I already lost this work once, let me **commit the implementation immediately** to protect it (syntax-check first, verify branch, stage explicit paths only ? never `-A`), then re-apply tests + docs.

181. user (2026-07-08T07:06:37.287Z)

```
===syntax check===
SYNTAX OK

===branch safety re-verify===
perf/521-single-pass-stitch
65ee4c6 fix(serving): probe NVENC at 256x256, not 64x64, so it isn't falsely rejected
(#521/#525) (#527)

===targeted compiler/watermark tests===
.................................                                        [100%]
```

```
33 passed in 0.72s
Shell cwd was reset to C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer
```

182. assistant (2026-07-08T07:07:04.613Z)

```
Syntax clean, branch correct, 33 tests green. Committing the implementation now to protect it
(explicit paths only):
```

183. user (2026-07-08T07:07:11.142Z)

```
===verify commit is mine and only mine===
650a183 perf(stitch): single-pass concat-filter stitch for a cheaper watermark (#521)
```

184. assistant (2026-07-08T07:07:22.823Z)

```
Implementation committed (`650a183`), and it's the only commit above master ? protected. Now re-
apply tests + docs. Let me read the (#527/#528-modified) regions I need to anchor into.
```

185. user (2026-07-08T07:07:25.585Z)

```
===tail of test_compiler.py (see #527 additions + true EOF)===
    dimensions (>=256px per axis). The original 64x64 is BELOW that minimum, so on real GPUs the
    probe FAILS ("at least one of its streams received no packets") even though NVENC works fine
at
    real resolutions ? making _select_encoder silently fall back to libx264 and defeating the
    NVENC-on-L4 re-encode win of #521/#525. Guard the geometry against regressing to a sub-
minimum
    size."""
    cmd = _capture_probe_cmd(monkeypatch)
    source = cmd[cmd.index("-i") + 1]
    match = re.search(r"s=(\d+)x(\d+)", source)
    assert match, f"probe lavfi source has no s=WxH size spec: {source!r}"
    width, height = int(match.group(1)), int(match.group(2))
    assert width >= 256 and height >= 256, (
        f"probe resolution {width}x{height} is below NVENC's minimum; must be >=256x256 so the "
        "probe reflects real NVENC runnability instead of rejecting an undersized frame
(#521/#525)"
    )


def test_probe_encoder_builds_single_frame_null_encode(monkeypatch):
    """Sanity: the probe stays a real 1-frame test encode to null with the requested codec (not
an
    ``-encoders`` capability grep), so it still distinguishes 'NVENC compiled-in but unusable'
from
    genuinely usable."""
    cmd = _capture_probe_cmd(monkeypatch)
    assert cmd[cmd.index("-c:v") + 1] == "h264_nvenc"  # probes the requested encoder
    assert cmd[cmd.index("-frames:v") + 1] == "1"  # exactly one frame
    assert "null" in cmd  # encodes to the null muxer, discarding output
    assert "-encoders" not in cmd  # a real encode, not a capability grep


def test_probe_encoder_returns_false_on_ffmpeg_failure(monkeypatch):
    """A non-zero ffmpeg exit (the encoder can't run on this box) ? unavailable ? caller falls
back
    to libx264. Previously untested: the existing encoder tests monkeypatch the whole probe
away."""
    from backend.pipeline.stitcher import compiler as comp_mod

    monkeypatch.setattr(comp_mod, "_ENCODER_AVAILABLE", {})
    monkeypatch.setattr(comp_mod.subprocess, "run", lambda *a, **k: MagicMock(returncode=1))
    assert comp_mod._probe_encoder_available("h264_nvenc") is False
```

```
=== #527 probe test present? ===
5:import re
250:def test_probe_encoder_uses_safe_resolution(monkeypatch):
Shell cwd was reset to C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer
```

186. assistant (2026-07-08T07:07:39.857Z)

#527 appended three probe tests (+`import re`) at the end of `test_compiler.py`. My single-pass tests must append after the true EOF. Let me Read the end via the Read tool (required before editing this path), then append.

187. user (2026-07-08T07:07:40.405Z)

```
280         """A non-zero ffmpeg exit (the encoder can't run on this box) ? unavailable ? caller
falls back
281         to libx264. Previously untested: the existing encoder tests monkeypatch the whole
probe away."""
282         from backend.pipeline.stitcher import compiler as comp_mod
283
284         monkeypatch.setattr(comp_mod, "_ENCODER_AVAILABLE", {})
285         monkeypatch.setattr(comp_mod.subprocess, "run", lambda *a, **k:
MagicMock(returncode=1))
286         assert comp_mod._probe_encoder_available("h264_nvenc") is False
287
```

188. user (2026-07-08T07:08:17.082Z)

The file C:\kg-wt-sp\tests\test_compiler.py has been updated successfully. (file state is current in your context ? no need to Read it back)

189. assistant (2026-07-08T07:08:22.946Z)

Now the dispatch test (#528 modified it). Let me read the nvenc-spec assertion region to re-add my one-line check.

190. user (2026-07-08T07:08:25.206Z)

```
123:def test_compile_dispatch_stages_resolved_spec_with_nvenc(compile_env):
139:    assert fake.spec["stitching"]["encoder"] == "h264_nvenc"
Shell cwd was reset to C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer
```

191. user (2026-07-08T07:08:30.874Z)

```
137         assert fake.spec["time_pairs"] == [[2.0, 6.0], [10.0, 14.0], [20.0, 22.0]]
138         # the L4 stitch uses NVENC (forwarded from deployment.cloud_stitch_encoder)
139         assert fake.spec["stitching"]["encoder"] == "h264_nvenc"
140         # the argv is a `compile` subcommand carrying the spec + done marker
141         argv = fake.calls[0]
142         assert argv[0] == "compile"
```

192. user (2026-07-08T07:08:39.316Z)

The file C:\kg-wt-sp\tests\test_compile_dispatch.py has been updated successfully. (file state is current in your context ? no need to Read it back)

193. assistant (2026-07-08T07:08:48.061Z)

Now the doc. #527/#528 added 8 lines to `PHASE_PLACEMENT.md` ? let me read the §3 area to see if my anchor (the follow-up note) survived and how the NVENC section changed.

194. user (2026-07-08T07:08:49.049Z)

```
100
```

```
101     **Fallback:** if compile is placed on the L4 but the box isn't actually cloud-wired (no
output bucket
102     / Cloud Run env), `_dispatch_compile_remote` logs and stitches in-process ? a slow reel
beats no reel.
103
104     ---
105
106     ## 3. NVENC on the L4 ? and the D5/D16(b) parity note
107
108     The dominant term in the CUJ stitch was the **per-clip 4K re-encode** the branding
watermark forces
109     (~67 s/clip on 2 vCPU with libx264). The L4 has a hardware H.264/HEVC encoder, so #521
adds an encoder
110     knob:
111
112     - `StitchingConfig.encoder` ? `libx264` (default, CPU, works everywhere incl. CI) |
`h264_nvenc` |
113       `hevc_nvenc`. `VideoCompiler` **probes** the requested encoder once per run (a 1-frame
lavfi test
114       encode) and **falls back to libx264** if this ffmpeg build can't run it, so a
misconfig / non-GPU
115       box never breaks. All clips share the one chosen encoder ? the concat stays `-c copy`.
116     - `deployment.cloud_stitch_encoder` (default `h264_nvenc`) ? what the compile-dispatch
forwards onto
117       the worker's `stitching.encoder` for the L4 run. Set it to `libx264` to run the L4
stitch on CPU.
118     - The control-plane's **own** local stitch always uses `stitching.encoder` (libx264 by
default) ? the
119       CP has no GPU.
120
121     **Parity note (honest):** [README.md](./README.md) **D5/D16(b)** kept stitch **encode =
libx264** to
122     protect a byte-exact stitch parity guarantee, and explicitly **deferred NVENC encode
behind a config
123     knob, "revisit only if M7 cost data justifies relaxing parity."** The 2026-07-08 CUJ
(~13 min stitch,
124     a hard scaling wall past ~5 concurrent users) **is** that cost data. #521 lands exactly
the deferred
125     shape ? NVENC **behind a config knob**, libx264 the default, config-reversible per
phase. Enabling
126     NVENC relaxes byte-identity **at the reel ENCODE layer only** (a cosmetic, shareable
artifact); the
127     **WINDOW/rally-level parity** (the analytical output) is untouched ? detection and
windowing are
128     unchanged. If bit-exact reels are ever required, set `cloud_stitch_encoder=libx264`.
129
130     > Follow-up (tracked, not in #521's first cut): a **pre-rendered / overlay-filter
watermark** so the
131     > stitch isn't a full re-encode at all (the deeper win beyond swapping the encoder).
132
133     ---
134
135     ## 4. Validation placement + the "nobody validates" guard
136
137     `phase_placement.validation` moves the CPU-bound validator suite off the 2-vCPU control-
plane too,
138     via the existing `--skip-validation` lever (#513):
139
140     - **`control_plane`** (default): the control-plane runs the validators and is the
authoritative gate;
141       the worker **skips** its redundant re-validation (`skip_validation=True`) ? today's
#512/#513
142       behaviour, byte-identical.
143     - **`cloud_run_l4`**: when detection **actually** dispatches to the L4, the control-
plane **delegates**
```

```
144       ? it does NOT run the validator suite (`_execute_processing` skips it), and the
dispatch carries
```

195. assistant (2026-07-08T07:09:06.337Z)

```
The §3 follow-up blockquote is intact ? my anchor works. Since **#527 already fixed** the NVENC
probe bug I'd flagged, I'll re-apply my §3.1 with that note updated from "filed" to "fixed in
#527". Re-applying:
```

196. user (2026-07-08T07:09:31.400Z)

```
The file C:\kg-wt-sp\docs\COMPUTE_DECOUPLED_SERVING\PHASE_PLACEMENT.md has been updated
successfully. (file state is current in your context ? no need to Read it back)
```

197. assistant (2026-07-08T07:09:36.933Z)

```
Now the §5 Files & tests row. Let me read §5 (it may have gained a #527 row):
```

198. user (2026-07-08T07:09:39.296Z)

```
223:## 5. Files & tests
232:| NVENC encoder | `backend/pipeline/stitcher/compiler.py` (`encoder`, `_select_encoder`,
`_probe_encoder_available`) | `tests/test_compiler.py` |
Shell cwd was reset to C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer
```

199. user (2026-07-08T07:09:42.400Z)

```
232     | NVENC encoder | `backend/pipeline/stitcher/compiler.py` (`encoder`, `_select_encoder`,
`_probe_encoder_available`) | `tests/test_compiler.py` |
233     | validation delegation | `backend/orchestration.py` (`_maybe_dispatch_remote` skip
flag, `_execute_processing`) | `tests/test_api_cloud_dispatch.py` |
```

200. user (2026-07-08T07:09:49.662Z)

```
The file C:\kg-wt-sp\docs\COMPUTE_DECOUPLED_SERVING\PHASE_PLACEMENT.md has been updated
successfully. (file state is current in your context ? no need to Read it back)
```

201. assistant (2026-07-08T07:09:54.469Z)

```
All re-applied. Let me run the targeted tests, then fold tests+docs into the commit.
```

202. user (2026-07-08T07:10:05.723Z)

```
.................................................................... [ 82%]
................                                                     [100%]
87 passed in 6.37s
Shell cwd was reset to C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer
```

203. assistant (2026-07-08T07:10:14.805Z)

```
87 pass (my single-pass tests + #527 probe + #528 dispatch tests all green). Let me fold tests +
docs into the commit (explicit paths), then verify the worktree state.
```

204. user (2026-07-08T07:10:18.506Z)

```
===status before staging===
 M docs/COMPUTE_DECOUPLED_SERVING/PHASE_PLACEMENT.md
 M tests/test_compile_dispatch.py
 M tests/test_compiler.py

===amended; files in commit===
8f4e474 perf(stitch): single-pass concat-filter stitch for a cheaper watermark (#521)
 backend/config/models.py                          |  10 +
```

```
 backend/orchestration.py                          |   1 +
 backend/pipeline/stitcher/compiler.py             | 259 +++++++++++++++++++--
 docs/COMPUTE_DECOUPLED_SERVING/PHASE_PLACEMENT.md |  68 +++++-
 tests/test_compile_dispatch.py                    |   2 +
 tests/test_compiler.py                            | 198 +++++++++++++++++
 6 files changed, 521 insertions(+), 17 deletions(-)

===only my commit above master?===
8f4e474 perf(stitch): single-pass concat-filter stitch for a cheaper watermark (#521)
Shell cwd was reset to C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer
```

205. assistant (2026-07-08T07:10:49.118Z)

Commit `8f4e474` now holds all 6 files and is the only commit above master ? the work is durably protected. Now the full pre-LGTM gate. Let me run ruff + mypy, and re-verify the re-applied code end-to-end with real ffmpeg (updating the verify script to point at the new worktree, with a robust luma check this time).

206. assistant (2026-07-08T07:11:04.671Z)

```
"""Re-verify the re-applied single-pass code end-to-end vs REAL ffmpeg (new worktree C:/kg-wt-
sp)."""
import os
import subprocess
import sys

WT = r"C:/kg-wt-sp"
sys.path.insert(0, WT)
SP = os.path.dirname(os.path.abspath(__file__))
SRC = os.path.join(SP, "src_4k_hevc10.mp4")
OUT = os.path.join(SP, "verify_out2")
os.makedirs(OUT, exist_ok=True)
from backend.pipeline.stitcher.compiler import VideoCompiler  # noqa: E402

SEG = [(3.0, 7.0), (12.0, 16.0), (20.0, 24.0)]
WM = {"enabled": True, "text": "Generated by Khelsutra.guru"}
exp = sum((e + 1.0) - max(0.0, s - 0.5) for s, e in SEG)  # VideoCompiler defaults 0.5/1.0


def dur(p):
    o = subprocess.run(["ffprobe", "-v", "error", "-show_entries", "format=duration",
                        "-of", "default=nk=1:nw=1", p], capture_output=True, text=True)
    return float(o.stdout.strip())


def streams(p):
    return subprocess.run(["ffprobe", "-v", "error", "-show_entries",
                           "stream=codec_type,codec_name,width,height", "-of", "csv=p=0", p],
                          capture_output=True, text=True).stdout.replace("\n", " ")


def corner_mean(reel, t):
    """Mean gray of the bottom-right 700x180 corner at time t (robust raw-bytes method)."""
    p = subprocess.run(["ffmpeg", "-v", "error", "-ss", str(t), "-i", reel, "-frames:v", "1",
                        "-vf", "crop=700:180:in_w-700:in_h-180,format=gray", "-f", "rawvideo",
"-"],
                       capture_output=True)
    d = p.stdout
    return (sum(d) / len(d)) if d else -1.0


pc = VideoCompiler(OUT, watermark=WM, single_pass=False).compile_highlights(SRC, SEG, "pc.mp4")
sp = VideoCompiler(OUT, watermark=WM, single_pass=True).compile_highlights(SRC, SEG, "sp.mp4")
spn = VideoCompiler(OUT, watermark={"enabled": False}, single_pass=True).compile_highlights(SRC,
SEG, "spn.mp4")
```

```
print(f"expected reel ~{exp:.2f}s")
print(f"per-clip    : {dur(pc):.2f}s [{streams(pc)}]")
print(f"single-pass : {dur(sp):.2f}s [{streams(sp)}]")
print(f"single nowm : {dur(spn):.2f}s [{streams(spn)}]")

ok = True
def chk(n, c):
    global ok; ok = ok and c; print(f"  [{'PASS' if c else 'FAIL'}] {n}")

chk("single-pass frame-accurate", abs(dur(sp) - exp) < 0.6)
chk("single-pass == per-clip duration", abs(dur(sp) - dur(pc)) < 0.6)
chk("single-pass 4K video+audio", "3840" in streams(sp) and "audio" in streams(sp))
m_wm, m_no = corner_mean(sp, 3.0), corner_mean(spn, 3.0)
chk(f"watermark rendered (corner mean wm={m_wm:.1f} vs nowm={m_no:.1f})", abs(m_wm - m_no) >
2.0)
print("RESULT:", "ALL PASS" if ok else "SOME FAILED")
sys.exit(0 if ok else 1)
```

207. user (2026-07-08T07:11:41.906Z)

```
===ruff===
All checks passed!
===mypy===
backend\pipeline\validators\base.py:123: note: By default the bodies of untyped functions are
not checked, consider using --check-untyped-defs  [annotation-unchecked]
backend\providers\__init__.py:9: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs  [annotation-unchecked]
Success: no issues found in 150 source files
Shell cwd was reset to C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer
```

208. user (2026-07-08T07:11:41.951Z)

```
File created successfully at: C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-
khelsutra-guru-badminton-highlight-
indexer\46eabfd2-0e71-4ff5-a68d-c76f474d4ff3\scratchpad\verify_sp2.py (file state is current in
your context ? no need to Read it back)
```

209. assistant (2026-07-08T07:11:50.500Z)

ruff + mypy clean (150 files). Let me start the full test suite in the background and run the real-ffmpeg re-verification in the foreground.

210. user (2026-07-08T07:11:52.075Z)

```
Command running in background with ID: b0kabecar. Output is being written to:
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-khelsutra-guru-badminton-
highlight-indexer\46eabfd2-0e71-4ff5-a68d-c76f474d4ff3\tasks\b0kabecar.output. You will be
notified when it completes. To check interim output, use Read on that file path.
```

211. user (2026-07-08T07:11:54.357Z)

```
Command running in background with ID: b3dq9wch9. Output is being written to:
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-khelsutra-guru-badminton-
highlight-indexer\46eabfd2-0e71-4ff5-a68d-c76f474d4ff3\tasks\b3dq9wch9.output. You will be
notified when it completes. To check interim output, use Read on that file path.
```

212. assistant (2026-07-08T07:12:00.634Z)

Both running. Let me wait for the real-ffmpeg verification (the quicker one).

213. user (2026-07-08T07:13:31.430Z)

```
=== DONE ===
expected reel ~16.50s
```

```
per-clip    : 16.52s [h264,video,3840,2160 aac,audio ]
single-pass : 16.50s [h264,video,3840,2160 aac,audio ]
single nowm : 16.50s [h264,video,3840,2160 aac,audio ]
  [PASS] single-pass frame-accurate
  [PASS] single-pass == per-clip duration
  [PASS] single-pass 4K video+audio
  [PASS] watermark rendered (corner mean wm=145.2 vs nowm=177.0)
RESULT: ALL PASS
```

214. assistant (2026-07-08T07:13:54.261Z)

Re-verification ALL PASS ? the re-applied code works end-to-end with real ffmpeg: frame-accurate (16.50s), 4K video+audio, watermark rendered (corner mean shifts 177?145 from the box+text).

Since **#527 already merged my flagged NVENC-probe fix**, that task chip is now stale ? let me dismiss it, then wait for the full suite.

215. user (2026-07-08T07:13:54.630Z)

Task task_8a2b7d23 was already started by the user ? it's no longer pending and can't be withdrawn. Nothing was changed.

216. assistant (2026-07-08T07:14:08.531Z)

The task was already started by the user (that's how #527 got made and merged) ? so it's done, not dismissable. Good. Let me wait for the full test suite.

217. user (2026-07-08T07:17:28.296Z)

```
=== DONE ===
Required test coverage of 70% reached. Total coverage: 85.87%
1889 passed, 8 skipped, 25 warnings in 322.57s (0:05:22)
 RESULT: PASS - all tests green
```

218. assistant (2026-07-08T07:17:54.456Z)

Full pre-LGTM gate is GREEN:
- ? Full suite: **1889 passed, 0 failed**, 8 skipped (the 3 failures in the earlier run were artifacts of the hijacked/corrupt worktree ? gone in the clean one).
- ? Coverage **85.87%** (?70 floor; my tests *raised* it).
- ? ruff clean, mypy clean (150 files).
- ? Real-ffmpeg e2e: frame-accurate, valid 4K+audio, watermark rendered.

The task asks for "one PR referencing #521", so let me push and open it. First re-sync and re-verify branch safety (the environment is volatile).

219. user (2026-07-08T07:18:01.359Z)

```
===origin/master tip===
304af7b feat(serving): L4-primary topology design + trust submit-time video_id (#524) (#530)
===my branch: only my commit above master?===
8f4e474 perf(stitch): single-pass concat-filter stitch for a cheaper watermark (#521)
===branch===
perf/521-single-pass-stitch
===push===
remote: Create a pull request for 'perf/521-single-pass-stitch' on GitHub by visiting:
remote:      https://github.com/Khelsutra/badminton-highlight-indexer/pull/new/perf/521-single-pass-stitch
remote:
branch 'perf/521-single-pass-stitch' set up to track 'origin/perf/521-single-pass-stitch'.
To https://github.com/Khelsutra/badminton-highlight-indexer.git
```

```
 * [new branch]      perf/521-single-pass-stitch -> perf/521-single-pass-stitch
Shell cwd was reset to C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer
```

220. assistant (2026-07-08T07:18:31.689Z)

Branch pushed and safe on the remote (durable now). origin/master advanced again to `304af7b` (#524/#530) ? my single clean commit sits on a recent ancestor, no problem for the PR. Let me open the PR referencing #521, with the measured results and verification.

221. assistant (2026-07-08T07:18:49.295Z)

Part 3 of #521 (follow-up to the compile-on-L4 + NVENC work merged in #525): make the branding watermark cheaper so the per-clip stitch isn't a full 4K re-encode. Builds on merged #521/#525 and the NVENC-probe fix #527.

The finding (measured, not assumed)

A burned-in watermark covers **every output frame**, so every output frame **must** be encoded ? that encode is an **irreducible floor**. A stream-copy-only stitch (no watermark) is **~1.1 s** vs **~73 s** with the watermark on a real 4K 10-bit HEVC clip: **~98?99 % of the stitch cost IS the watermark re-encode**. So the levers are (a) a faster encoder (NVENC ? already shipped in #521) and (b) removing **per-clip overhead**. This PR is (b).

Measured ? real 4K 10-bit HEVC clip (3840×2160, 8 segments, RTX 2070S, ffmpeg 8.1, warm)

strategy	libx264	h264_nvenc	frame-accurate?
Baseline (per-clip re-encode + `drawtext` ? concat `-c copy`)	73.1 s	34.7 s	? reel 40.0 s
Option A (pre-rendered PNG overlay)	59.5 s (?19 %)	36.0 s (**+4 %**)	?
Option B (stream-copy trims ? concat re-encode + wm)	65.3 s (?11 %)	28.9 s (?17 %)	? **reel 48.1 s**
Option C ? single-pass (chosen)	**67.4 s (?14 %)**	**34.1 s (?11 %)**	? reel 40.0 s
floor: stream-copy, no watermark	1.1 s	1.2 s	?

Chose Option C: one ffmpeg pass ? precise `-ss/-t` inputs ? concat **filter** ? watermark **once** ? a single encode.

- Unlike the brief's **Option B** (stream-copy trims ? concat re-encode), the concat **filter** keeps cuts **frame-accurate**; Option B's stream-copy snapped to keyframes (48.1 s vs the correct 40.0 s reel ? a *content* regression, not a cosmetic encode relaxation).
- Unlike **Option A**, it helps **NVENC** too (the L4 primary path; A regresses NVENC ~4 %).
- Also produces **smaller** reels (one continuous encode vs N keyframe-forced GOPs) and writes **no intermediate clips**.

Safety

- **Opt-in** `stitching.single_pass_stitch` (default **OFF**). The per-clip path stays the parity-safe default **and** the automatic **fallback**: `_compile_single_pass` returns `None` on any failure (ffmpeg error, timeout, segment-count cap, no valid segments) and `compile_highlights` transparently re-runs the per-clip path ? a slower reel beats no reel.
- Segments validated up front via the shared `_resolve_padded_window` (extracted from `_trim_one_segment`, behaviour-preserving), so an invalid segment is dropped before the graph.
- Rides the compile-spec to the L4 worker exactly like `encoder`.
- **Parity (D5/D16(b)):** identical footage (frame-accurate) + identical watermark; only the *encode structure* differs (one continuous encode vs N GOPs) ? within the reel-ENCODE-layer relaxation D16(b) already allows under NVENC. `single_pass_stitch=false` keeps the old path bit-for-bit.

Verification

- **Full suite: 1889 passed, 0 failed**, 8 skipped; **coverage 85.87 %** (? 70 floor; new tests

raise it).
- **ruff + mypy clean** (150 source files).
- **Real ffmpeg end-to-end:** single-pass reel frame-accurate (16.50 s vs per-clip 16.52 s), valid 4K H.264 + AAC, watermark rendered (verified on the RTX 2070S).
- **Owner-gated:** a confirming run on the **L4 + golden corpus** (per the repo's machine-side split); the transferable result is the relative deltas + frame-accuracy.

Docs: `docs/COMPUTE_DECOUPLED_SERVING/PHASE_PLACEMENT.md` **§3.1** ? full measured before/after and why A/B were rejected.

? Generated with [Claude Code](https://claude.com/claude-code)

222. user (2026-07-08T07:18:49.344Z)

File created successfully at: C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-khelsutra-guru-badminton-highlight-indexer\46eabfd2-0e71-4ff5-a68d-c76f474d4ff3\scratchpad\pr_body.md (file state is current in your context ? no need to Read it back)

223. user (2026-07-08T07:18:57.922Z)

https://github.com/Khelsutra/badminton-highlight-indexer/pull/531
Shell cwd was reset to C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer

224. assistant (2026-07-08T07:19:11.991Z)

PR created: **https://github.com/Khelsutra/badminton-highlight-indexer/pull/531**. Let me verify it's clean (only my commit, mergeable against the advanced master) and check CI kicked off.

225. user (2026-07-08T07:19:17.006Z)

PR # 531 base: master
mergeable: MERGEABLE UNSTABLE
commits: ['perf(stitch): single-pass concat-filter stitch for a cheaper watermar?']
files: 6 -> ['backend/config/models.py', 'backend/orchestration.py', 'backend/pipeline/stitcher/compiler.py', 'docs/COMPUTE_DECOUPLED_SERVING/PHASE_PLACEMENT.md', 'tests/test_compile_dispatch.py', 'tests/test_compiler.py']
Shell cwd was reset to C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer

226. assistant (2026-07-08T07:20:06.358Z)

PR **#531** is clean: **MERGEABLE**, exactly **1 commit** (mine, no sibling commits absorbed), exactly **6 files** (my intended set). `UNSTABLE` just means CI is still running ? and I've already reproduced every CI gate locally (ruff, mypy, full suite, coverage). No conflicts with the twice-advanced master.

Let me check the project memory dir to record a durable pointer for future sessions (without touching the sibling-owned repo-root `SESSION_HANDOFF.md`).

227. user (2026-07-08T07:20:08.048Z)

total 24
drwxr-xr-x 1 avidu 197609   0 Jul 8 12:18 .
drwxr-xr-x 1 avidu 197609   0 Jul 8 12:11 ..
-rw-r--r-- 2 avidu 197609 995 Jul 8 12:18 MEMORY.md
-rw-r--r-- 1 avidu 197609 2920 Jul 8 11:34 pr-525-compile-on-l4.md
-rw-r--r-- 2 avidu 197609 2664 Jul 8 12:18 pr-527-nvenc-probe-resolution.md
-rw-r--r-- 2 avidu 197609 1709 Jul 8 12:18 pr-528-worker-mirror-required.md

228. assistant (2026-07-08T07:20:20.838Z)

Sibling sessions record each PR as a memory file here (pr-525, pr-527, pr-528) ? an active convention. Let me follow it for my PR #531. Reading the index first to append safely:

229. user (2026-07-08T07:20:21.161Z)

```
1       # Memory index
2
3       - [PR #525 ? compile-on-L4 (#521)](pr-525-compile-on-l4.md) ? config-driven
phase?machine placement + L4 NVENC stitch; Part 3 (cheaper watermark) deferred.
4       - [PR #527 ? NVENC probe 64x64?256x256](pr-527-nvenc-probe-resolution.md) ? sub-minimum
probe silently defeated #521's NVENC-on-L4 win; MERGED to master (65ee4c6); STILL awaits owner
live-GPU confirm; monitoring follow-up #529.
5       - [PR #528 ? worker mirror-required](pr-528-worker-mirror-required.md) ? #521 follow-up:
L4-worker stitch now fails on a lost reel mirror-upload (was silent 404-on-success); found by
adversarial review; **MERGED to master** (008a5d4).
6       - **Release baseline (2026-07-08):** master `65ee4c6` carries #521 (compile-on-L4) + its
two follow-up fixes #528 (mirror-required) + #527 (NVENC 256px probe). Owner cut/cutting a
release folding all three. Real L4/NVENC exec still unverified (needs a worker image rebuilt
from this master; deployed `:latest` predated the compile subcommand).
7
```

230. assistant (2026-07-08T07:20:32.771Z)

Let me record PR #531 as a project memory following the established pattern, and add its index pointer:

231. assistant (2026-07-08T07:20:48.399Z)

```
---
name: pr-531-single-pass-stitch
description: PR #531 ? #521 Part-3 (cheaper watermark) via an opt-in single-pass concat-filter
stitch; measured, default-OFF, awaits owner L4/corpus confirm.
metadata:
  type: project
---

**PR [#531](https://github.com/Khelsutra/badminton-highlight-indexer/pull/531)** ?
`perf(stitch): single-pass concat-filter stitch for a cheaper watermark (#521)`. Branch
`perf/521-single-pass-stitch` off master; the deferred Part-3 of [[pr-525-compile-on-l4]].

**What it does:** opt-in `stitching.single_pass_stitch` (default **OFF**). When on,
`VideoCompiler._compile_single_pass` stitches the reel in ONE ffmpeg pass ? precise `-ss/-t`
inputs ? `concat` **filter** ? watermark drawn once ? a single encode ? instead of per-clip re-
encode + concat-copy. Falls back to the per-clip path on any failure; shared
`_resolve_padded_window` validates segments up front; `_SINGLE_PASS_MAX_SEGMENTS` cap guards the
N-input graph. Rides the compile-spec to the L4 like `encoder`.

**Key finding (why the brief's framing was corrected):** a burned-in watermark re-encodes EVERY
frame ? that encode is an irreducible floor (a no-watermark stream-copy stitch is ~1.1 s vs ~73
s with it; ~98?99 % of stitch cost IS the watermark re-encode). So neither pre-render (Option A)
nor stream-copy-then-concat-re-encode (Option B) removes the floor; the win is per-clip
OVERHEAD. Measured on a real 4K 10-bit HEVC clip (RTX 2070S): single-pass (Option C) **?14 %
libx264 / ?11 % NVENC, frame-accurate**, smaller output. Option B was rejected (stream-copy
snaps to keyframes ? reel 48.1 s vs 40.0 s, a content regression); Option A helps only libx264
(+4 % on NVENC).

**Verified:** full suite 1889 passed / coverage 85.87 %; ruff+mypy clean; real-ffmpeg e2e
(frame-accurate, 4K+audio, watermark rendered). **Owner-gated:** confirming run on the L4 +
golden corpus (machine-side split). Doc: `docs/COMPUTE_DECOUPLED_SERVING/PHASE_PLACEMENT.md`
§3.1.

**Process note (2026-07-08):** built in an isolated worktree; a concurrent codex session
HIJACKED the first worktree (checked out `codex/issue-523-sd-reel-review` + reset it), wiping my
UNCOMMITTED changes. Recovered from conversation history, re-applied onto the then-current
master (which had gained #527/#528), and **committed immediately**. Lesson: in this high-
concurrency checkout, commit early ? uncommitted worktree edits are not safe. Related: my
flagged NVENC-probe bug became [[pr-527-nvenc-probe-resolution]] (merged before this PR).
```

232. user (2026-07-08T07:20:48.468Z)

File created successfully at: C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-khelsutra-guru-badminton-highlight-indexer\memory\pr-531-single-pass-stitch.md (file state is current in your context ? no need to Read it back)

233. user (2026-07-08T07:21:00.039Z)

The file C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-khelsutra-guru-badminton-highlight-indexer\memory\MEMORY.md has been updated successfully. (file state is current in your context ? no need to Read it back)

234. assistant (2026-07-08T07:21:09.129Z)

Memory recorded. Let me check the PR's CI status to report accurately, then summarize.

235. user (2026-07-08T07:21:12.567Z)

Golden fixtures cross-OS (macos-latest) pass 22s https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/28924919234/job/85810214347
Golden fixtures cross-OS (windows-latest) pass 53s https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/28924919234/job/85810214398
Leak scan (PII / attribution guard) pass 9s https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/28924919234/job/85810214296
Markdown link check (live docs) pass 6s https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/28924919234/job/85810214322
Mypy (lenient baseline) pass 41s https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/28924919234/job/85810214332
Ruff lint pass 11s https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/28924919234/job/85810214355
Syntax sweep + import smoke (no GPU stack) pass 25s https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/28924919234/job/85810214309
Full test suite (offline, mocked) pending 0 https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/28924919234/job/85810214368
Built wheel smoke (install + launch) pass 1m3s https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/28924919234/job/85810214344
Dependency license scan (no copyleft in runtime deps) pass 33s https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/28924919234/job/85810214330
Golden fixtures cross-OS (ubuntu-latest) pass 26s https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/28924919234/job/85810214352
Shell cwd was reset to C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer

236. assistant (2026-07-08T07:21:52.855Z)

All CI checks **pass** except the full mocked suite which is still **pending** (running) ? and I've reproduced that locally (1889 passed). Notably the "Syntax sweep + import smoke" check **passed**, confirming the 3 failures in my earlier run were artifacts of the hijacked worktree, not real. The work is done.

Summary

Delivered: [PR #531](https://github.com/Khelsutra/badminton-highlight-indexer/pull/531) ? the deferred **Part 3 of #521**: make the branding watermark cheaper so the per-clip stitch isn't a full 4K re-encode.

What the investigation found (the task's core)

I measured, rather than assumed. A burned-in watermark covers **every frame**, so every frame **must** be encoded ? that encode is an **irreducible floor**. On a real 4K 10-bit HEVC clip, a no-watermark stream-copy stitch is **~1.1s** vs **~73s** with the watermark, i.e. **~98?99% of stitch cost is the watermark re-encode**. So the only levers are a faster encoder (NVENC ?

already shipped in #521) and removing **per-clip overhead**.

I benchmarked the brief's Option A and B plus a third variant, on **both** encoders:

strategy	libx264	h264_nvenc	frame-accurate
Baseline (current)	73.1s	34.7s	?
Option A (pre-rendered overlay)	59.5s (?19%)	36.0s (**+4%**)	?
Option B (stream-copy ? concat re-encode)	65.3s	28.9s	? **48.1s reel**
Option C ? single-pass (chosen)	**67.4s (?14%)**	**34.1s (?11%)**	?

Chose Option C ? one ffmpeg pass (precise `-ss/-t` inputs ? `concat` *filter* ? watermark once ? single encode). It beats the brief's Option B, which stream-copies to keyframes and produces a **wrong-length reel** (48.1s vs 40.0s ? a content regression, not the cosmetic encode relaxation D16(b) allows), and beats Option A, which regresses NVENC (the L4 primary path). It's **opt-in** (`stitching.single_pass_stitch`, default OFF), with the robust per-clip path as both the parity-safe default and the automatic fallback.

Verified (pre-LGTM gate, all reproduced locally)
- Full suite **1889 passed, 0 failed**; coverage **85.87%** (?70 floor, my tests raise it) · ruff + mypy clean.
- **Real-ffmpeg end-to-end**: single-pass reel frame-accurate (16.50s vs per-clip 16.52s), valid 4K H.264+AAC, watermark rendered (visually confirmed).
- CI: all checks green except the full suite (still running).
- Documented in `docs/COMPUTE_DECOUPLED_SERVING/PHASE_PLACEMENT.md` **§3.1** with the measured before/after and A/B rejection rationale.
- **Owner-gated** (per the repo's machine-side split): a confirming run on the L4 + golden corpus ? the transferable result is the relative deltas + frame-accuracy on a synthetic clip.

Two things you should know
1. **A latent #521 bug I flagged is already fixed and merged (#527):** `_probe_encoder_available` probed NVENC at `64x64`, below NVENC's minimum encode size, so it *falsely* fell back to libx264 on real GPUs ? silently defeating #521's NVENC-on-L4 win. You (via the task chip) got it fixed to `256x256`. My doc note reflects that it's now fixed.
2. **A concurrent session hijacked my first isolated worktree** ? it checked out `codex/issue-523-sd-reel-review` into it and reset, wiping my *uncommitted* changes mid-way. I recovered from context, re-applied onto the (meanwhile-advanced) master, and **committed immediately** this time. The lesson is baked into a project memory: in this many-worktree checkout, commit early. I deliberately did **not** touch the sibling-owned root `SESSION_HANDOFF.md` (active #523 WIP) ? recorded this thread in the project memory dir instead.

The branch is pushed and durable; my local worktree `C:/kg-wt-sp` holds it and can be pruned after merge.

237. user (2026-07-08T07:21:55.763Z)

please make this as default on so that the next release can pick this up

238. user (2026-07-08T07:22:16.138Z)

[Request interrupted by user]

239. user (2026-07-08T07:53:09.545Z)

merged

240. assistant (2026-07-08T07:55:27.844Z)

#531 is merged ? good. Now you want `single_pass_stitch` **default ON** so the next release picks it up. That's a small, focused default-flip on a fresh branch off the latest master. Let me first check my worktree is still intact (hijack risk), re-sync, and confirm nothing else pins the default OFF.

241. user (2026-07-08T07:55:35.897Z)

```
===worktree state===
branch: perf/521-single-pass-stitch

origin/master: 304af7b feat(serving): L4-primary topology design + trust submit-time video_id
(#524) (#530)

===is #531 (single-pass) merged into master?===
NO ? not on master yet

===does config.json pin it?===
config.json: no single_pass key (Pydantic default governs)

===does cloud-serving preset touch it?===
presets.py: no single_pass key

===any test asserting the CONFIG default is False?===
tests/test_compile_dispatch.py:141:    assert "single_pass_stitch" in fake.spec["stitching"]
Binary file tests/__pycache__/test_compile_dispatch.cpython-313-pytest-9.0.3.pyc matches
Shell cwd was reset to C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer
```

242. assistant (2026-07-08T07:55:56.635Z)

My worktree is intact this time. But like before, my local `origin/master` (304af7b) doesn't yet show #531's `single_pass_stitch` ? likely the same fetch-lag. You said it's merged, so let me re-sync harder and confirm where #531 landed before branching.

243. user (2026-07-08T07:56:01.935Z)

```
===origin/master now===
304af7b feat(serving): L4-primary topology design + trust submit-time video_id (#524) (#530)
65ee4c6 fix(serving): probe NVENC at 256x256, not 64x64, so it isn't falsely rejected
(#521/#525) (#527)
008a5d4 fix(compile): fail the L4-worker stitch when the reel mirror-upload fails (#521 follow-
up) (#528)
dac6736 perf(serving): run compile/stitch on the L4 worker; config-driven phase?machine
placement (#521) (#525)
bf97700 docs(runlogs): record live SPA CUJ closing the 2 owner-gated validation-latency cells
(§9) (#522)

===single_pass_stitch on origin/master?===
ABSENT on master

===search master history for #531===
351a27e perf(eval): single-pass person detection for fusion_golden (~10x faster extraction)
(#125)
Shell cwd was reset to C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer
```

244. assistant (2026-07-08T07:56:21.729Z)

A hard `fetch --prune` still shows #531 **absent** from `origin/master` ? the only "single-pass" hit is an unrelated old PR. Your "merged" and my git view disagree, so let me check the authoritative GitHub state for #531 before I branch (this determines whether I flip on master, or just add the default-on commit to the still-open #531).

245. user (2026-07-08T07:56:25.479Z)

```
state: OPEN
mergedAt: None
mergeCommit: None
base: master head: perf/521-single-pass-stitch
mergeable: MERGEABLE CLEAN
Shell cwd was reset to C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer
```

246. assistant (2026-07-08T07:57:44.451Z)

GitHub's authoritative API says **#531 is OPEN, not merged** (`mergedAt: None`, `mergeable: CLEAN` ? CI went green). So your "merged" and the actual state differ. Since #531 is still open, the cleanest way to make single-pass the default is to add the default-on flip **to #531 itself** (keeps it one PR referencing #521, as intended) rather than a second PR. Let me verify my branch is synced with the PR head, then make the change.

247. user (2026-07-08T07:57:47.642Z)

```
===local vs origin PR head===
local:  8f4e474124ac23e1dbe52838d55d669cca5a3b5b
origin: 8f4e474124ac23e1dbe52838d55d669cca5a3b5b
IN SYNC (safe to add a commit)

===current default line in models.py===
384:    single_pass_stitch: bool = False
Shell cwd was reset to C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer
```

248. assistant (2026-07-08T07:58:29.995Z)

In sync at `8f4e474`. Now let me make the default-on change ? flip the config default and clarify the comments (keeping the `VideoCompiler` constructor default `False` as the conservative primitive + fallback, so the ~10 per-clip unit tests that construct it directly stay valid).

249. user (2026-07-08T07:58:34.726Z)

The file C:\kg-wt-sp\backend\config\models.py has been updated successfully. (file state is current in your context ? no need to Read it back)

250. user (2026-07-08T07:58:43.118Z)

The file C:\kg-wt-sp\backend\pipeline\stitcher\compiler.py has been updated successfully. (file state is current in your context ? no need to Read it back)

251. assistant (2026-07-08T07:58:49.280Z)

Now a config-default test. Let me read the config test section of `test_watermark.py` (this worktree path) to place it:

252. user (2026-07-08T07:58:49.856Z)

```
20      # --- config ----------------------------------------------------------------
21
22
23      def test_watermark_config_defaults_on_with_brand_text():
24          wm = AppConfig().stitching.watermark
25          assert wm.enabled is True  # default-on, with an off switch
26          assert wm.text == "Generated by Khelsutra.guru"
27          assert wm.position == "bottom-right"
28
29
30      def test_watermark_can_be_disabled_via_config():
31          cfg = AppConfig.model_validate({"stitching": {"watermark": {"enabled": False}}})
32          assert cfg.stitching.watermark.enabled is False
33
34
35      def test_font_size_ratio_and_opacity_bounds_enforced():
36          with pytest.raises(ValidationError):
37              WatermarkConfig(opacity=1.5)
```

253. user (2026-07-08T07:58:57.812Z)

The file C:\kg-wt-sp\tests\test_watermark.py has been updated successfully. (file state is current in your context ? no need to Read it back)

254. assistant (2026-07-08T07:59:10.199Z)

Now update the doc §3.1 to reflect default-on. Let me read the exact current lines:

255. user (2026-07-08T07:59:12.658Z)

```
132:> **Status:** ? SHIPPED behind `stitching.single_pass_stitch` (default OFF) · one PR
referencing #521.
176:- `StitchingConfig.single_pass_stitch` ? `false` (default). When **true**, `VideoCompiler`
stitches the
178:- **Default OFF = parity-safe.** The per-clip path stays the default *and* the automatic
**fallback**:
Shell cwd was reset to C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer
```

256. user (2026-07-08T07:59:17.723Z)

```
130     ### 3.1 Part-3 ? cheaper watermark: the single-pass stitch (? shipped, measured)
131
132     > **Status:** ? SHIPPED behind `stitching.single_pass_stitch` (default OFF) · one PR
referencing #521.
133     > Was tracked here as "a pre-rendered / overlay-filter watermark so the stitch isn't a
full re-encode
134     > at all." **Investigated + measured on a real 4K clip; the honest finding reshaped the
fix** (below).
135
```

257. user (2026-07-08T07:59:18.451Z)

```
174     **Shape (mirrors the §3 encoder knob ? opt-in, default-safe, self-healing).**
175
176     - `StitchingConfig.single_pass_stitch` ? `false` (default). When **true**,
`VideoCompiler` stitches the
177       reel in one pass (`_compile_single_pass`). Rides the compile-spec onto the L4 like
`encoder` does.
178     - **Default OFF = parity-safe.** The per-clip path stays the default *and* the automatic
**fallback**:
179       `_compile_single_pass` returns `None` on any failure (ffmpeg error, timeout,
>`_SINGLE_PASS_MAX_SEGMENTS`
180       cap, no valid segments) and `compile_highlights` transparently re-runs the per-clip
path ? a slower
181       reel beats no reel. Segments are validated up front via the shared
`_resolve_padded_window`, so an
182       invalid segment is dropped *before* the graph, never failing the whole encode.
183     - **Parity:** identical footage (frame-accurate) and an identical watermark ? only the
*encode structure*
184       differs (one continuous encode vs N GOPs), which is exactly the reel-ENCODE-layer
relaxation D16(b)
185       already allows under NVENC. WINDOW/rally-level parity is untouched. To force bit-for-
bit the old path,
186       leave `single_pass_stitch=false`.
187
```

258. assistant (2026-07-08T07:59:41.187Z)

Applying the doc §3.1 default-on updates:

259. user (2026-07-08T07:59:44.415Z)

```
The file C:\kg-wt-sp\docs\COMPUTE_DECOUPLED_SERVING\PHASE_PLACEMENT.md has been updated
successfully. (file state is current in your context ? no need to Read it back)
```

260. user (2026-07-08T07:59:55.713Z)

```
The file C:\kg-wt-sp\docs\COMPUTE_DECOUPLED_SERVING\PHASE_PLACEMENT.md has been updated
```

successfully. (file state is current in your context ? no need to Read it back)

261. assistant (2026-07-08T08:00:03.443Z)

Let me update the "Honest limits" note to reflect that default-on raises the priority of the L4 confirmation (while the fallback protects). Let me locate it:

262. user (2026-07-08T08:00:05.299Z)

192:**Honest limits (owner-gated, per the repo's machine-side split).** Numbers are from a **synthetic**
198:`VideoCompiler` NVENC path is correctly enabled. A confirming run on the L4 + golden corpus stays owner-gated.
Shell cwd was reset to C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer

263. user (2026-07-08T08:00:10.963Z)

The file C:\kg-wt-sp\docs\COMPUTE_DECOUPLED_SERVING\PHASE_PLACEMENT.md has been updated successfully. (file state is current in your context ? no need to Read it back)

264. assistant (2026-07-08T08:00:20.613Z)

All default-on edits done. Let me syntax-check and run the targeted tests (watermark/compiler/dispatch + any config default tests) to confirm the flip breaks nothing:

265. user (2026-07-08T08:01:14.437Z)

SYNTAX OK

INFO   backend.pipeline.stitcher.compiler:compiler.py:683 Dynamic highlight reel saved (single-pass): C:\Users\avidu\AppData\Local\Temp\pytest-of-avidu\pytest-1643\test_compile_uses_configured_s0\out\reel.mp4 (0.0 MB)
INFO   httpx:_client.py:1025 HTTP Request: POST http://testserver/api/compile "HTTP/1.1 202 Accepted"
INFO   httpx:_client.py:1025 HTTP Request: GET http://testserver/api/jobs/4b8e39996be74f9e9e1b28a056c2432b "HTTP/1.1 200 OK"
============================== warnings summary ===============================
tests/test_api_endpoints.py: 10 warnings
  C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\site-packages\google\auth_default.py:113: UserWarning: Your application has authenticated using end user credentials from Google Cloud SDK without a quota project. You might receive a "quota exceeded" or "API not enabled" error. See the following page for troubleshooting: https://cloud.google.com/docs/authentication/adc-troubleshooting/user-creds.
    warnings.warn(_CLOUD_SDK_CREDENTIALS_WARNING)

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
========================== short test summary info ==========================
FAILED tests/test_api_endpoints.py::test_compile_uses_configured_stitching_paddings
1 failed, 175 passed, 10 warnings in 47.29s
Shell cwd was reset to C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer

266. assistant (2026-07-08T08:01:36.165Z)

A test failed: `test_compile_uses_configured_stitching_paddings` ? and the log shows it now runs the **single-pass** path (config default flipped), so its assertions about per-clip trim commands no longer match. This is a legitimate test that assumed per-clip; I need to fix it properly, not just paper over it. Let me read it.

267. user (2026-07-08T08:01:37.676Z)

606:def test_compile_uses_configured_stitching_paddings(env):
Shell cwd was reset to C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer

268. user (2026-07-08T08:01:43.243Z)

```
606     def test_compile_uses_configured_stitching_paddings(env):
607         """config.json stitching.{begin,end}_padding must reach the ffmpeg trim boundaries.
608         Regression for the 2026-06-10 audit finding (d): /api/compile constructed
609         VideoCompiler without the paddings, so reels always used the constructor
610         defaults (0.5/1.0) and the config values were silently ignored."""
611         client, db, tmp_path, mp = env
612         cfg = m.global_config
613         cfg.output.output_dir = str(tmp_path / "out")
614         cfg.stitching.begin_padding = 2.0
615         cfg.stitching.end_padding = 3.0
616         cfg.stitching.watermark.enabled = False  # keep the trim command minimal
617         db.add_video("vid", str(_video(tmp_path)), "processed", [])
618         sid = db.add_play_segment("vid", 10.0, 14.0)
619
620         calls = []
621
622         def fake_run(cmd, *a, **k):
623             calls.append(cmd)
624             if cmd[0] == "ffprobe":
625                 return MagicMock(stdout="100.0\n", returncode=0)
626             out = cmd[-1]
627             if isinstance(out, str) and out.endswith(".mp4"):
628                 open(out, "wb").write(b"clip")
629             return MagicMock(returncode=0, stdout=b"", stderr=b"")
630
631         with patch.object(subprocess, "run", side_effect=fake_run):
632             # The inline executor runs the stitch (under this patch) during the POST, so the
ffmpeg
633             # trim commands are captured here; then the job is already 'done'.
634             r = client.post(
635                 "/api/compile",
636                 json={
637                     "video_id": "vid",
638                     "segment_ids": [sid],
639                     "output_filename": "reel.mp4",
640                 },
641             )
642             job = client.get(f"/api/jobs/{r.json()['job_id']}").json()
643         assert r.status_code == 202
644         assert job["status"] == "done" and job["result"]["status"] == "success"
645
646         trim = next(c for c in calls if c[0] == "ffmpeg" and "-ss" in c)
647         # 10.0 - begin_padding 2.0 and 14.0 + end_padding 3.0; the unwired constructor
648         # defaults (0.5/1.0) would have produced 9.5/15.0 here.
649         assert trim[trim.index("-ss") + 1] == "8.0"
650         assert trim[trim.index("-to") + 1] == "17.0"
651
652
653     def test_parity_job_worker_vs_direct_cli_path(env, tmp_path, monkeypatch):
654         """Item 5 positive: same ProcessRequest through job worker path and direct CLI-style
path
655         produce byte-identical DB rows, reports, final play_segments (modulo timestamps).
656         Exercises Failure and empty-run branches.
657         """
658         client, db, tpath, mp = env
659         vid_file = _video(tpath)
660         vid = m.compute_video_id(str(vid_file))
661         mp.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {}))
662         mp.setattr(m.segmenter_registry, "get", lambda name: _FakeSeg)
663         # job worker path
664         rj = client.post(
665             "/api/process", json={"video_path": str(vid_file), "segmenter": "x"}
666         )
```

```
667         jobj = client.get(f"/api/jobs/{rj.json()['job_id']}").json()
668         # direct CLI-style path (using same processing setup as main CLI; inline executor
makes outcomes identical in test harness)
669         rd = client.post(
670             "/api/process", json={"video_path": str(vid_file), "segmenter": "x"}
671         )
672         jobd = client.get(f"/api/jobs/{rd.json()['job_id']}").json()
673         # byte-identical key outcomes
674         assert jobj["result"]["status"] == jobd["result"]["status"]
675         assert len(db.query_play_segments(vid, {})) == len(db.query_play_segments(vid, {}))
```

269. assistant (2026-07-08T08:02:32.354Z)

Clear: the padding regression test now hits the single-pass default, which expresses the window `[8.0, 17.0]` as input seek `-ss 8.0 -t 9.0` rather than `-ss 8.0 -to 17.0`. The paddings *are* correctly applied ? the test's assertion form was per-clip-specific. Since single-pass is what ships now, I'll update it to assert the default path's form:

270. user (2026-07-08T08:02:34.598Z)

The file C:\kg-wt-sp\tests\test_api_endpoints.py has been updated successfully. (file state is current in your context ? no need to Read it back)

271. assistant (2026-07-08T08:02:44.080Z)

Let me confirm that fix and re-run the affected tests, then launch the full suite (authoritative for a default-flip).

272. user (2026-07-08T08:02:52.936Z)

```
.....................................                                    [100%]
42 passed in 3.47s
Shell cwd was reset to C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer
```

273. assistant (2026-07-08T08:02:58.381Z)

Let me run the full suite (background, authoritative for a default-flip) and ruff/mypy (foreground).

274. user (2026-07-08T08:02:59.917Z)

Command running in background with ID: bqyazpqzg. Output is being written to: C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-khelsutra-guru-badminton-highlight-indexer\46eabfd2-0e71-4ff5-a68d-c76f474d4ff3\tasks\bqyazpqzg.output. You will be notified when it completes. To check interim output, use Read on that file path.

275. user (2026-07-08T08:03:07.849Z)

```
===ruff===
All checks passed!
===mypy===
backend\providers\__init__.py:9: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs  [annotation-unchecked]
Success: no issues found in 150 source files
Shell cwd was reset to C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer
```

276. assistant (2026-07-08T08:03:21.093Z)

ruff + mypy clean. Let me wait for the full suite (authoritative for the default flip ? it'll catch any other test that assumed per-clip).

277. user (2026-07-08T08:06:49.888Z)

```
=== DONE ===
Required test coverage of 70% reached. Total coverage: 85.87%
1890 passed, 8 skipped, 25 warnings in 222.22s (0:03:42)
 RESULT: PASS - all tests green
```

278. assistant (2026-07-08T08:07:08.805Z)

Full suite green with the default flipped: **1890 passed, 0 failed** (the one padding test was the only casualty, now fixed to assert the single-pass form), coverage 85.87%, ruff + mypy clean. Committing the default-flip as a distinct, revertable commit on the #531 branch and pushing:

279. user (2026-07-08T08:07:20.849Z)

```
branch: perf/521-single-pass-stitch | in sync with PR head: yes
===my commits above master===
b98209f perf(stitch): default single_pass_stitch ON so the next release picks it up (#521)
8f4e474 perf(stitch): single-pass concat-filter stitch for a cheaper watermark (#521)
===push===
To https://github.com/Khelsutra/badminton-highlight-indexer.git
   8f4e474..b98209f  perf/521-single-pass-stitch -> perf/521-single-pass-stitch
Shell cwd was reset to C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer
```

280. assistant (2026-07-08T08:07:37.724Z)

PR #531 now has 2 commits (feature + default-on), both mine above master. Let me update the PR description to reflect default-on (it currently describes the opt-in version), then update the memory note.

281. user (2026-07-08T08:07:43.931Z)

```
The file C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-khelsutra-guru-
badminton-highlight-indexer\46eabfd2-0e71-4ff5-a68d-c76f474d4ff3\scratchpad\pr_body.md has been
updated successfully. (file state is current in your context ? no need to Read it back)
```

282. user (2026-07-08T08:07:52.120Z)

```
The file C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-khelsutra-guru-
badminton-highlight-indexer\46eabfd2-0e71-4ff5-a68d-c76f474d4ff3\scratchpad\pr_body.md has been
updated successfully. (file state is current in your context ? no need to Read it back)
```

283. assistant (2026-07-08T08:07:57.362Z)

Let me update the verification line to the current suite count and note the flip was verified, then apply the body to the PR:

284. user (2026-07-08T08:08:00.667Z)

```
The file C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-khelsutra-guru-
badminton-highlight-indexer\46eabfd2-0e71-4ff5-a68d-c76f474d4ff3\scratchpad\pr_body.md has been
updated successfully. (file state is current in your context ? no need to Read it back)
```

285. user (2026-07-08T08:08:17.894Z)

```
https://github.com/Khelsutra/badminton-highlight-indexer/pull/531
===PR state + commits===
state: OPEN | mergeable: MERGEABLE UNSTABLE
commits: ['perf(stitch): single-pass concat-filter stitch for a cheaper watermar?',
'perf(stitch): default single_pass_stitch ON so the next release picks?']
Shell cwd was reset to C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer
```

286. assistant (2026-07-08T08:08:31.862Z)

PR #531 updated ? 2 commits (feature + default-on), OPEN, mergeable (CI re-running). Let me update my memory note to reflect default-on:

287. user (2026-07-08T08:08:34.804Z)

<tool_use_error>String to replace not found in file.
String: description: PR #531 ? #521 Part-3 (cheaper watermark) via an opt-in single-pass concat-filter stitch; measured, default-OFF, awaits owner L4/corpus confirm.
(note: Edit also tried swapping \uXXXX escapes and their characters; neither form matched, so the mismatch is likely elsewhere in old_string. Re-read the file and copy the exact surrounding text.)</tool_use_error>

288. assistant (2026-07-08T08:08:39.249Z)

The memory file content differs from what I expect ? let me re-read it (it may have been touched):

289. user (2026-07-08T08:08:39.826Z)

```
1       ---
2       name: pr-531-single-pass-stitch
3       description: PR
4       metadata:
5         node_type: memory
6         type: project
7         originSessionId: 46eabfd2-0e71-4ff5-a68d-c76f474d4ff3
8       ---
9
10      **PR [#531](https://github.com/Khelsutra/badminton-highlight-indexer/pull/531)** ? `perf(stitch): single-pass concat-filter stitch for a cheaper watermark (#521)`. Branch `perf/521-single-pass-stitch` off master; the deferred Part-3 of [[pr-525-compile-on-l4]].
11
12      **What it does:** opt-in `stitching.single_pass_stitch` (default **OFF**). When on, `VideoCompiler._compile_single_pass` stitches the reel in ONE ffmpeg pass ? precise `-ss/-t` inputs ? `concat` **filter** ? watermark drawn once ? a single encode ? instead of per-clip re-encode + concat-copy. Falls back to the per-clip path on any failure; shared `_resolve_padded_window` validates segments up front; `_SINGLE_PASS_MAX_SEGMENTS` cap guards the N-input graph. Rides the compile-spec to the L4 like `encoder`.
13
14      **Key finding (why the brief's framing was corrected):** a burned-in watermark re-encodes EVERY frame ? that encode is an irreducible floor (a no-watermark stream-copy stitch is ~1.1 s vs ~73 s with it; ~98?99 % of stitch cost IS the watermark re-encode). So neither pre-render (Option A) nor stream-copy-then-concat-re-encode (Option B) removes the floor; the win is per-clip OVERHEAD. Measured on a real 4K 10-bit HEVC clip (RTX 2070S): single-pass (Option C) **?14 % libx264 / ?11 % NVENC, frame-accurate**, smaller output. Option B was rejected (stream-copy snaps to keyframes ? reel 48.1 s vs 40.0 s, a content regression); Option A helps only libx264 (+4 % on NVENC).
15
16      **Verified:** full suite 1889 passed / coverage 85.87 %; ruff+mypy clean; real-ffmpeg e2e (frame-accurate, 4K+audio, watermark rendered). **Owner-gated:** confirming run on the L4 + golden corpus (machine-side split). Doc: `docs/COMPUTE_DECOUPLED_SERVING/PHASE_PLACEMENT.md` §3.1.
17
18      **Process note (2026-07-08):** built in an isolated worktree; a concurrent codex session HIJACKED the first worktree (checked out `codex/issue-523-sd-reel-review` + reset it), wiping my UNCOMMITTED changes. Recovered from conversation history, re-applied onto the then-current master (which had gained #527/#528), and **committed immediately**. Lesson: in this high-concurrency checkout, commit early ? uncommitted worktree edits are not safe. Related: my flagged NVENC-probe bug became [[pr-527-nvenc-probe-resolution]] (merged before this PR).
19
```

290. assistant (2026-07-08T08:08:49.347Z)

The memory system normalized the frontmatter (description got truncated to "PR"); the body is

intact. Let me fix the description and update the body for default-on:

291. user (2026-07-08T08:08:52.503Z)

The file C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-khelsutra-guru-badminton-highlight-indexer\memory\pr-531-single-pass-stitch.md has been updated successfully. (file state is current in your context ? no need to Read it back)

292. user (2026-07-08T08:08:58.095Z)

The file C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-khelsutra-guru-badminton-highlight-indexer\memory\pr-531-single-pass-stitch.md has been updated successfully. (file state is current in your context ? no need to Read it back)

293. user (2026-07-08T08:09:07.998Z)

The file C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-khelsutra-guru-badminton-highlight-indexer\memory\pr-531-single-pass-stitch.md has been updated successfully. (file state is current in your context ? no need to Read it back)

294. assistant (2026-07-08T08:09:14.258Z)

Now the MEMORY.md index line for #531. Let me read and update it:

295. user (2026-07-08T08:09:15.070Z)

1 # Memory index
2
3 - [PR #525 ? compile-on-L4 (#521)](pr-525-compile-on-l4.md) ? config-driven phase?machine placement + L4 NVENC stitch; Part 3 (cheaper watermark) deferred.
4 - [PR #527 ? NVENC probe 64x64?256x256](pr-527-nvenc-probe-resolution.md) ? sub-minimum probe silently defeated #521's NVENC-on-L4 win; MERGED to master (65ee4c6); STILL awaits owner live-GPU confirm; monitoring follow-up #529.
5 - [PR #528 ? worker mirror-required](pr-528-worker-mirror-required.md) ? #521 follow-up: L4-worker stitch now fails on a lost reel mirror-upload (was silent 404-on-success); found by adversarial review; **MERGED to master** (008a5d4).
6 - [PR #531 ? single-pass stitch (#521 Part-3)](pr-531-single-pass-stitch.md) ? cheaper watermark via opt-in single-pass concat-filter stitch (default OFF); measured ?11??14% both encoders, frame-accurate; OPEN, awaits owner L4/corpus confirm.
7 - **Release baseline (2026-07-08):** master `65ee4c6` carries #521 (compile-on-L4) + its two follow-up fixes #528 (mirror-required) + #527 (NVENC 256px probe). Owner cut/cutting a release folding all three. Real L4/NVENC exec still unverified (needs a worker image rebuilt from this master; deployed `:latest` predated the compile subcommand).
8

296. user (2026-07-08T08:09:21.173Z)

The file C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-khelsutra-guru-badminton-highlight-indexer\memory\MEMORY.md has been updated successfully. (file state is current in your context ? no need to Read it back)

297. assistant (2026-07-08T08:09:31.084Z)

Memory updated. Let me do a final CI check on the updated PR, then summarize.

298. user (2026-07-08T08:09:34.568Z)

pass Built wheel smoke (install + launch)
pass Dependency license scan (no copyleft in runtime deps)
pass Golden fixtures cross-OS (macos-latest)
pass Golden fixtures cross-OS (ubuntu-latest)
pass Golden fixtures cross-OS (windows-latest)
pass Leak scan (PII / attribution guard)

```
pass    Markdown link check (live docs)
pass    Mypy (lenient baseline)
pass    Ruff lint
pass    Syntax sweep + import smoke (no GPU stack)
pending Full test suite (offline, mocked)
Shell cwd was reset to C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer
```

299. assistant (2026-07-08T08:09:56.599Z)

Done. `single_pass_stitch` is now **default ON**, pushed to PR #531 as a distinct, revertable second commit.

One correction you'll want to know: GitHub's API says **PR #531 is still OPEN, not merged** (`mergedAt: none`, and `single_pass_stitch` isn't on `origin/master` even after a hard fetch). So "merged" didn't match the actual state. Because it's still open, I put the default-flip **into #531 itself** rather than spin up a second PR ? that keeps it "one PR referencing #521" and means **merging #531 ships single-pass as the default**. If you actually merged something else or want the flip as its own PR, say so and I'll re-cut it.

What changed (commit `b98209f`):
- `StitchingConfig.single_pass_stitch: False ? True` ? single-pass is now the product default, so the next release picks it up.
- The per-clip path remains the **automatic fallback** on any single-pass failure (ffmpeg error, timeout, segment cap) ? that's what makes default-on safe.
- The `VideoCompiler` *constructor* arg stays `False` (the conservative primitive default; orchestration passes the config value explicitly) ? this keeps the ~10 unit tests that construct `VideoCompiler` directly to exercise the per-clip/fallback path valid, no churn.
- One test genuinely needed updating: a padding regression test asserted the per-clip `-ss/-to` form; it now asserts the single-pass default's `-ss 8.0 -t 9.0` input-seek form for the same `[8.0, 17.0]` window.
- Doc `PHASE_PLACEMENT.md` §3.1 updated (status default-ON; fallback keeps it safe; L4/corpus confirmation stays owner-gated and matters a bit more now it's the default).

Verified (default-flip re-run):
- Full suite **1890 passed, 0 failed**, coverage **85.87%** (?70) · ruff + mypy clean.
- CI on the updated PR: **all checks green** except the full mocked suite still running (reproduced locally).

Residual (unchanged, honest): the confirming run on the real **L4 + golden corpus** stays owner-gated. Since single-pass is now the *default*, that confirmation matters slightly more ? but if it ever misbehaves on real hardware, the per-clip fallback still produces a reel, so it degrades rather than breaks.

PR: https://github.com/Khelsutra/badminton-highlight-indexer/pull/531

300. user (2026-07-08T08:26:24.335Z)

merged

301. assistant (2026-07-08T08:26:37.076Z)

Let me verify authoritatively this time ? the last two "merged" signals didn't match GitHub's state, so I'll check the API and confirm it actually landed on master.

302. user (2026-07-08T08:26:43.733Z)

```
===PR #531 state (GitHub API)===
state: MERGED
mergedAt: 2026-07-08T08:17:45Z
mergeCommit: 5880c26bb0599ef37a3251209a4bd7764bd91be4

origin/master: 5880c26 perf(stitch): single-pass concat-filter stitch for a cheaper watermark
(#521) (#531)
===single_pass_stitch default on master now?===
```

386:    single_pass_stitch: bool = True
Shell cwd was reset to C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer

303. assistant (2026-07-08T08:27:06.378Z)

Confirmed ? this time it genuinely merged: **PR #531 is MERGED** (`mergedAt 2026-07-08T08:17:45Z`, merge commit `5880c26`), `origin/master` now carries it, and **`single_pass_stitch: bool = True`** is on master. The default-on landed; the next release will pick it up.

Let me update the memory record to MERGED and clean up my worktree.

304. user (2026-07-08T08:27:11.355Z)

The file C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-khelsutra-guru-badminton-highlight-indexer\memory\MEMORY.md has been updated successfully. (file state is current in your context ? no need to Read it back)

305. user (2026-07-08T08:27:20.230Z)

The file C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-khelsutra-guru-badminton-highlight-indexer\memory\pr-531-single-pass-stitch.md has been updated successfully. (file state is current in your context ? no need to Read it back)

306. assistant (2026-07-08T08:27:41.378Z)

Now let me clean up my merged worktree and branch (good hygiene in this heavily-worktree'd repo):

307. user (2026-07-08T08:27:45.914Z)

```
===remove my merged worktree===
removed (or already gone)
===delete merged local branch===
Deleted branch perf/521-single-pass-stitch (was b98209f).
===confirm my worktree/branch gone===
worktree gone ?
branch gone ?
```

308. assistant (2026-07-08T08:27:57.914Z)

All wrapped up. ?

Merged and landed: PR #531 is in master (`5880c26`), and **`single_pass_stitch = True` is now the default** ? so the next release ships the single-pass concat-filter stitch by default (~11?14% faster, frame-accurate, both encoders), with the per-clip path as the automatic fallback if it ever fails.

Cleaned up my merged worktree and local branch. Memory record updated to MERGED.

One honest residual carried forward (tracked in the doc §3.1 and project memory): the confirming run on the real **L4 + golden corpus** is still owner-gated. It matters a little more now that single-pass is the default ? but the fallback means a failure on real hardware degrades to the old per-clip stitch rather than breaking. That's the one thing worth closing when you next have the GPU box + corpus in hand.